UNIVERSITÀ DELLA CALABRIA

Anno accademico 2026/2027

Algorithmic Game Theory

Appunti del corso: teoria e laboratori

Corso tenuto da Sebastiano A. Piccolo

Ultima compilazione: 5 novembre 2026

Indice

1 Introduzione all'Algorithmic Game Theory **1**

1.1 Informazioni sul corso . . . 1

1.2 Che cos'è la teoria dei giochi . . . 2

1.3 Che cos'è l'Algorithmic Game Theory . . . 2

1.4 Panoramica delle aree dell'AGT . . . 2

1.4.1 Giochi non cooperativi . . . 2

1.4.2 Giochi coalizionali . . . 4

1.4.3 Scelta sociale computazionale . . . 5

1.4.4 Mechanism design . . . 5

1.4.5 Matching markets . . . 6

1.5 Giochi combinatori . . . 6

1.5.1 Dalla teoria dei giochi al giocare . . . 6

1.5.2 L'albero di gioco . . . 6

1.5.3 Un'IA per il tris . . . 7

1.5.4 L'algoritmo Minimax . . . 7

1.5.5 I limiti di Minimax . . . 8

1.5.6 Potatura Alpha-Beta . . . 8

1.5.7 Oltre alpha-beta . . . 10

1.6 Laboratorio: Minimax per il tris . . . 11

1.6.1 Rappresentazione e condizioni di vittoria . . . 11

1.6.2 Minimax . . . 11

1.6.3 Scegliere la mossa migliore . . . 12

1.6.4 Minimax contro sé stesso . . . 13

1.6.5 Minimax con potatura alpha-beta . . . 13

1.6.6 Giocare contro l'IA . . . 14

2 Teoria dell'utilità **15**

2.1 Concetti di base . . . 15

2.1.1 Alternative . . . 15

2.1.2 Preferenze . . . 16

2.2 Rappresentare le preferenze: utilità ordinale 16
2.2.1 Le condizioni sono necessarie 16
2.2.2 Le condizioni sono sufficienti 17
2.2.3 Esempi: l'utilità ordinale non è interpretabile 17
2.3 Oltre l'ordinale: le lotterie 18
2.3.1 Lotterie composte 18
2.3.2 Valore atteso 18
2.3.3 Utilità attesa 19
2.4 L'utilità di Von Neumann–Morgenstern 19
2.4.1 Necessità degli assiomi 20
2.4.2 Sufficienza: dimostrazione del teorema VNM 20
2.5 Il paradosso di Allais 21
2.6 Atteggiamento verso il rischio 22
2.7 Applicazioni 23
2.7.1 Condivisione del rischio (*risk sharing*) 24
2.7.2 Assicurazione 24
2.8 Riepilogo sulla teoria VNM 25
2.9 Laboratorio: Battaglia navale e ricerca bayesiana 26
2.9.1 Regole e impostazione 26
2.9.2 Strategia 1: tiro casuale 27
2.9.3 Strategia 2: *hunt and target* 27
2.9.4 La teoria della ricerca bayesiana 28
2.9.5 Strategia 3: ricerca bayesiana per la battaglia navale 30
2.9.6 Risultati misurati 32
2.9.7 Analisi in teoria dell'informazione 33
2.9.8 Che cosa portarsi dietro 34

3 Giochi non cooperativi **35**

3.1 Giochi strategici e forma normale 35
3.1.1 Giochi a due giocatori come matrici 36
3.2 Strategie dominanti 36
3.2.1 Eliminazione iterata delle strategie strettamente dominate 36
3.3 Best response ed equilibrio di Nash in strategie pure 37
3.3.1 IESDS ed equilibrio di Nash 38

3.3.2 Come leggere l'equilibrio di Nash . . . 38
3.3.3 Equilibri con utilità analitiche: la tragedia dei beni comuni . . . 39
3.4 Strategie miste . . . 40
3.5 Best response ed equilibrio di Nash misti . . . 41
3.5.1 Condizione generale per una best response . . . 42
3.6 Equilibri misti nei giochi 2×2 . . . 43
3.7 Giochi a somma zero e minimax . . . 43
3.7.1 Esempio: sasso-carta-forbici come programma lineare . . . 45
3.8 Laboratorio: Best response, equilibri e giochi potenziali . . . 47
3.8.1 Best response ed equilibrio di Nash . . . 47
3.8.2 Minimax nei giochi a somma zero . . . 48
3.8.3 Equilibri misti nei giochi 2×2 . . . 49
3.8.4 Giochi potenziali e rilevamento di comunità . . . 49

4 Equilibri alternativi e apprendimento 53
4.1 Esistenza dell'equilibrio di Nash misto . . . 53
4.1.1 Il teorema del punto fisso di Brouwer . . . 53
4.1.2 La costruzione di Nash . . . 54
4.2 Complessità del calcolo degli equilibri . . . 55
4.2.1 Equilibri misti: PPAD-completezza . . . 55
4.2.2 Equilibri puri: NP-difficoltà . . . 56
4.2.3 Il quadro complessivo . . . 57
4.3 Equilibri correlati . . . 57
4.3.1 L'intuizione: il semaforo . . . 57
4.3.2 Definizione . . . 58
4.3.3 Le disuguaglianze nell'esempio . . . 58
4.3.4 Calcolo con la programmazione lineare . . . 59
4.4 Altri concetti di soluzione . . . 59
4.5 Apprendimento nei giochi . . . 60
4.5.1 Fictitious play . . . 60
4.5.2 Regret matching . . . 60
4.6 Laboratorio: Equilibri correlati e apprendimento . . . 63
4.6.1 Equilibri correlati con `linprog` . . . 63
4.6.2 Fictitious play . . . 64

4.6.3 Regret matching 65

5 Giochi coalizionali **67**

5.1 Dai giochi non cooperativi a quelli coalizionali 67

5.2 Giochi in funzione caratteristica 68

5.3 Struttura di coalizione 69

5.3.1 Il problema è difficile 69

5.3.2 Programmazione dinamica 69

5.4 Dividere il payoff 71

5.4.1 Superadditività 71

5.5 Stabilità: il core 71

5.5.1 Due cattive notizie 72

5.6 Il nucleolo 73

5.6.1 L'algoritmo di Kopelowitz 73

5.6.2 Il nucleolo può essere iniquo 74

5.7 Equità: il valore di Shapley 74

5.7.1 Caratterizzazione assiomatica 76

5.8 Laboratorio: Core e valore di Shapley 77

5.8.1 Rappresentare la funzione caratteristica 77

5.8.2 Superadditività 77

5.8.3 Core 77

5.8.4 Valore di Shapley 78

5.8.5 In più: struttura di coalizione e nucleolo 79

6 Giochi coalizionali: aspetti computazionali **81**

6.1 Il problema computazionale 81

6.2 Il gioco dell'aeroporto 82

6.3 Giochi su sottografo indotto 84

6.4 Rappresentazioni compatte 85

6.4.1 Marginal Contribution Nets 85

6.4.2 MC-Nets con formule generali 86

6.5 Approssimazione Monte Carlo del valore di Shapley 88

6.6 Riepilogo delle tecniche 89

6.7 Laboratorio: Aeroporto, Monte Carlo, ISG e MC-Nets 90

6.7.1 Il gioco dell'aeroporto 90

6.7.2 Approssimazione Monte Carlo 91
6.7.3 Giochi su sottografo indotto 92
6.7.4 MC-Nets 93

LEZIONE 1

Introduzione all'Algorithmic Game Theory

TEORIA

1.1 Informazioni sul corso

- **Lezioni**: mercoledì 10:30–13:30 (teoria), giovedì 11:30–13:30 (laboratorio).
- **Teams**: *Algorithmic Game Theory 2026/27* (il codice per entrare è nelle slide).
- **Esame**: un **progetto** in cui si deve *formalizzare* un problema e *implementarne* una soluzione usando gli strumenti dell'AGT.

> **Osservazione (conseguenza pratica).** Dato che l'esame è un progetto, contano due cose: saper *modellare* un problema (chi sono i giocatori, quali azioni hanno, quali utilità) e saper *scrivere codice* che calcola il concetto di soluzione giusto. I laboratori vanno quindi studiati quanto la teoria.

Programma (provvisorio)

1. Giochi combinatori (*Combinatorial Games*)
2. Teoria dell'utilità (*Utility theory*)
3. Teoria dei giochi non cooperativi (*Non-cooperative game theory*)
4. Teoria dei giochi coalizionali (*Coalitional game theory*)
5. Scelta sociale computazionale (*Computational Social Choice*)
6. Mercati di abbinamento (*Matching Markets*)
7. Progettazione di meccanismi (*Mechanism Design*)
8. Divisione equa (*Fair division*)

Testi di riferimento

Non esiste un unico libro che copra tutto il corso; queste sono le risorse gratuite indicate:

- Y. Shoham, K. Leyton-Brown, *Multiagent Systems*;
- N. Nisan, T. Roughgarden, É. Tardos, V. Vazirani, *Algorithmic Game Theory*;
- A. Karlin, Y. Peres, *Game Theory, Alive*;
- F. Brandt et al., *Handbook of Computational Social Choice*.

1.2 Che cos'è la teoria dei giochi

> **Definizione 1.1 (Teoria dei giochi).** La **teoria dei giochi** studia le *decisioni prese da più soggetti* (*multiperson decision making*), con lo scopo di prevedere come più agenti **razionali** interagiranno e si comporteranno in situazioni di **conflitto** e/o di **cooperazione**.

I quattro ingredienti della definizione:

1. **Conflitto**: le azioni di un giocatore influenzano gli altri.
2. **Cooperazione**: i giocatori possono unire le forze per raggiungere un obiettivo comune.
3. **Comportamento razionale**: ogni giocatore vuole *massimizzare* la propria **utilità attesa** (*expected utility*).
4. **Predizione**: vogliamo sapere cosa succederà. Lo si fa tramite i **concetti di soluzione** (*solution concepts*), come l'equilibrio di Nash o il valore di Shapley.

1.3 Che cos'è l'Algorithmic Game Theory

> **Definizione 1.2 (Algorithmic Game Theory).** L'**Algorithmic Game Theory** (AGT) è l'area all'intersezione tra *teoria dei giochi* e *progettazione di algoritmi*, il cui obiettivo è progettare algoritmi in *ambienti strategici*. [Nisan et al. 2007]

In concreto, l'AGT si occupa di progettare algoritmi per:

- **calcolare** in modo efficiente i concetti di soluzione (es. trovare un equilibrio);
- **aggregare** voti e preferenze;
- **allocare** risorse scarse.

1.4 Panoramica delle aree dell'AGT

1.4.1 Giochi non cooperativi

I giochi non cooperativi modellano situazioni di **conflitto**: i giocatori competono tra loro e non possono necessariamente cooperare. Esempi: scacchi, sasso-carta-forbici, dilemma del prigioniero.

L'obiettivo è modellare una situazione e capire cosa succede. Il concetto di soluzione tipico è l'**equilibrio**.

Il dilemma del prigioniero

Due giocatori scelgono simultaneamente se *cooperare* (C) o *tradire* (D, *defect*). I numeri sono le utilità (anni di carcere con segno negativo): in ogni cella il primo valore è di P1, il secondo di P2.

		P2	
		Cooperate	Defect
P1	Cooperate	$-1, -1$	$-5, 0$
	Defect	$0, -5$	$-3, -3$

Tabella 1.1: Dilemma del prigioniero in *forma normale* (gioco bi-matrice).

Questa rappresentazione a tabella si chiama **forma normale** (o gioco *bi-matrice*): una matrice di utilità per ciascuno dei due giocatori.

Definizione 1.3 (Ottimo di Pareto). Un esito è **Pareto-ottimo** se non esiste un altro esito in cui *qualcuno* sta meglio e *nessuno* sta peggio. In altre parole: non si può migliorare la situazione di un giocatore senza peggiorare quella di qualcun altro.

Definizione 1.4 (Equilibrio di Nash, idea). Un profilo di strategie è un **equilibrio di Nash** se nessun giocatore può migliorare la propria utilità cambiando *da solo* la propria scelta.

Esempio (analisi del dilemma). Nash. Per P1, tradire conviene *qualunque* cosa faccia P2: se P2 coopera, $0 > -1$; se P2 tradisce, $-3 > -5$. Lo stesso vale per P2. Quindi (D, D) è l'unico equilibrio di Nash, con utilità $(-3, -3)$. Da (C, C), invece, ciascuno ha interesse a deviare (passa da -1 a 0). **Pareto.** (C, C) con $(-1, -1)$ è Pareto-ottimo. L'equilibrio (D, D) *non* lo è: (C, C) fa stare meglio entrambi.

Il dilemma mostra la tensione centrale della teoria dei giochi:

$$\textbf{Ottimalità di Pareto} \Rightarrow \textbf{efficienza} \qquad \textbf{Equilibrio di Nash} \Rightarrow \textbf{stabilità}$$

L'esito efficiente non è stabile e l'esito stabile non è efficiente.

Attenzione (definizione nelle slide). Le slide definiscono l'ottimo di Pareto come *"nessun giocatore può deviare senza peggiorare qualcun altro"*. È una formulazione informale: l'ottimo di Pareto *non* riguarda le deviazioni unilaterali (quelle sono il criterio di Nash), ma il confronto tra *esiti*. Nel dilemma sono Pareto-ottimi anche (C, D) e (D, C): passando da $(-5, 0)$ a qualunque altro esito P2 starebbe peggio. L'unico esito *non* Pareto-ottimo è (D, D).

Applicazioni

- algoritmi per giocare: scacchi, poker, . . . ;
- *Generative Adversarial Networks* (GAN): generatore e discriminatore sono due giocatori;
- sicurezza: pattugliamento, sorveglianza, . . . ;
- decisioni sequenziali robuste (*robust sequential decision making*).

1.4.2 Giochi coalizionali

Nei giochi coalizionali (o cooperativi) i giocatori **uniscono le forze** per un obiettivo comune, ad esempio mettono insieme i soldi per comprare un bene. Le domande sono due:

- quale coalizione è più probabile che si formi?
- come si divide il bene (il *payoff*) tra i membri?

Concetto di soluzione principale: il **valore di Shapley**.

Esempio (tre imprese). Tre imprese A, B, C possiedono ciascuna una parte della tecnologia per sviluppare un prodotto. Due imprese insieme guadagnano 100; una da sola guadagna 0; tutte e tre insieme guadagnano comunque 100. La **funzione caratteristica** v assegna a ogni coalizione il suo valore:

$$\begin{aligned} v(\{A\}) = v(\{B\}) = v(\{C\}) &= 0, \\ v(\{A,B\}) = v(\{A,C\}) = v(\{B,C\}) &= 100, \\ v(\{A,B,C\}) &= 100. \end{aligned}$$

- Se si forma la coalizione di tutti e tre e si divide in parti uguali: 33,33 a testa.
- Ma B e C da soli ottengono 100 e, dividendo, 50 a testa: stanno meglio abbandonando A. La divisione in parti uguali della grande coalizione non è stabile.

Esempio (variante: un giocatore più importante). Ora $v(\{A,B\}) = v(\{A,C\}) = 100$ ma $v(\{B,C\}) = 0$ (il resto invariato). Senza A non si produce nulla: A è più "importante" e la divisione in parti uguali non è più equa. Come dividere in modo equo? Con il **valore di Shapley**.

Approfondimento (calcolo del valore di Shapley nella variante). Il valore di Shapley (sarà definito formalmente più avanti) dà a ogni giocatore il suo *contributo marginale medio* su tutti gli ordini in cui i giocatori possono entrare nella coalizione. Con 3 giocatori gli ordini sono $3! = 6$:

Ordine	contributo di A	contributo di B	contributo di C
A, B, C	0	100	0
A, C, B	0	0	100
B, A, C	100	0	0
B, C, A	100	0	0
C, A, B	100	0	0
C, B, A	100	0	0
Media	66,67	16,67	16,67

Esempio di lettura: nell'ordine B, C, A, A entra quando ci sono già B e C, e porta il valore da $v(\{B,C\}) = 0$ a $v(\{A,B,C\}) = 100$: contributo 100. Il totale è 100, cioè tutto il valore della grande coalizione. Nel caso simmetrico di prima si ottiene invece 33,33 a testa.

Applicazioni

- divisione equa (*fair division*);
- classifiche di importanza, potere di voto, . . . ;
- spiegabilità dell'IA: **SHAP** (valori di Shapley applicati alle feature di un modello);
- *network science*: massimizzazione dell'influenza, centralità, . . .

1.4.3 Scelta sociale computazionale

La **scelta sociale computazionale** (*Computational Social Choice*) si occupa del calcolo dei risultati delle regole di voto e dei problemi legati a rappresentare e raccogliere le preferenze dei giocatori. Domande tipiche:

- come si decide il vincitore di un'elezione?
- a quali condizioni la democrazia è possibile?

Esempio (paradosso di Condorcet). Tre votanti ordinano tre candidati:

Votante	1ª preferenza	2ª preferenza	3ª preferenza
Votante 1	A	B	C
Votante 2	B	C	A
Votante 3	C	A	B

Chi dovrebbe vincere? Confrontiamo i candidati a coppie, a maggioranza:

- A batte B (votanti 1 e 3);
- B batte C (votanti 1 e 2);
- C batte A (votanti 2 e 3).

La preferenza collettiva è **ciclica**: $A \succ B \succ C \succ A$. Anche se ogni votante ha preferenze coerenti (transitive), la maggioranza non le ha, e nessun candidato batte tutti gli altri.

Applicazioni

Elezioni, decisioni collettive, e nel machine learning i *random forest* e gli *ensemble* di modelli (ogni modello "vota" una predizione).

1.4.4 Mechanism design

Il **mechanism design** è una sorta di teoria dei giochi *al contrario*: invece di prendere un gioco come dato e analizzarne gli esiti, si vuole *progettare* un gioco che incentivi i giocatori a comportarsi nel modo desiderato.

Esempio (aste). In un'asta si vuole che le persone dichiarino il *vero* prezzo a cui comprerebbero un oggetto o venderebbero un servizio.

Casi di successo: eBay, Google Ads, appalti pubblici e aste.

1.4.5 Matching markets

Nei **mercati di abbinamento** si vogliono abbinare agenti ad altri agenti, o agenti a oggetti, beni, servizi, garantendo qualche proprietà sociale desiderabile. Esempi: studenti e alloggi universitari, pazienti e donatori di rene, lavoratori e compiti. Nel corso si studierà un algoritmo premiato con il Nobel per lo **stable matching**.

1.5 Giochi combinatori

Definizione 1.5 (Gioco combinatorio). Un **gioco combinatorio** ha le seguenti proprietà:

1. **due giocatori**, di solito chiamati Giocatore 1 e Giocatore 2, oppure MAX e MIN;
2. **informazione perfetta**: entrambi conoscono l'intero stato del gioco;
3. **nessun elemento casuale**: niente dadi, mazzi mescolati o eventi aleatori;
4. **finito**: il gioco termina sempre (niente partite infinite);
5. **turni alternati**: i giocatori muovono uno alla volta;
6. **somma zero**: il guadagno di un giocatore è esattamente la perdita dell'altro.

Esempi: scacchi, dama, go, tris (*tic-tac-toe*).

1.5.1 Dalla teoria dei giochi al giocare

Problema centrale: dato lo stato attuale del gioco, qual è la mossa *migliore*?

La mossa migliore è quella che porta al miglior esito possibile *assumendo che anche l'avversario giochi in modo perfetto*. L'idea chiave è modellare l'avversario come intelligente e capace di dare sempre la contromossa migliore.

Osservazione (perché assumere un avversario perfetto). Se un giocatore vince contro la **risposta migliore** (*best response*) dell'avversario, vince anche contro qualunque risposta non ottimale. Ragionare sul caso peggiore dà quindi una *garanzia*: il risultato non può essere peggiore di quello calcolato.

1.5.2 L'albero di gioco

Definizione 1.6 (Albero di gioco). Ogni gioco di questo tipo si può rappresentare come un **albero di gioco**:

- i **nodi** sono stati del gioco (una configurazione della scacchiera);
- gli **archi** sono mosse, che portano da uno stato a un altro;
- la **radice** è lo stato attuale;

- le **foglie** sono stati terminali (vittoria di qualcuno o pareggio).

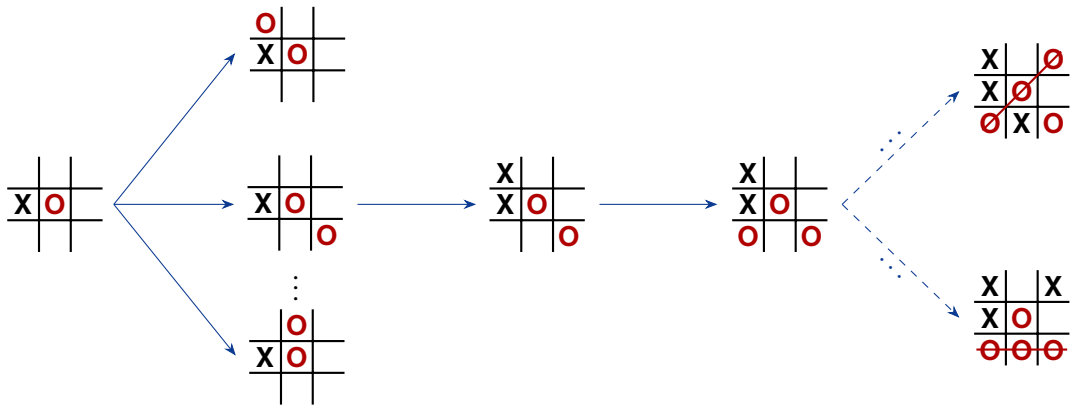


Figura 1.1: Una porzione dell'albero di gioco del tris. A destra due foglie in cui vince O.

1.5.3 Un'IA per il tris

Servono tre elementi:

1. un modo per **rappresentare** la scacchiera e lo stato del gioco;
2. una **funzione di valutazione** per sapere chi vince;
3. una **strategia di ricerca** nell'albero di gioco.

Funzione di punteggio. Il gioco è piccolo e a somma zero, quindi si assegna un punteggio agli stati finali:

$$\text{valore}(s) = \begin{cases} +1 & \text{se vince MAX,} \\ -1 & \text{se vince MIN,} \\ 0 & \text{in caso di pareggio.} \end{cases}$$

La grandezza dei numeri non conta: conta solo l'*ordine* ($+1 > 0 > -1$).

Rappresentazione. La scacchiera è una matrice 3×3 di numeri: X $\to 1$, O $\to 2$, cella vuota $\to 0$. Servono una funzione che controlla se c'è un tris su una riga, colonna o diagonale, e una che controlla se la scacchiera è piena.

1.5.4 L'algoritmo Minimax

Minimax è un algoritmo ricorsivo che esplora l'albero di gioco per determinare la mossa ottimale.

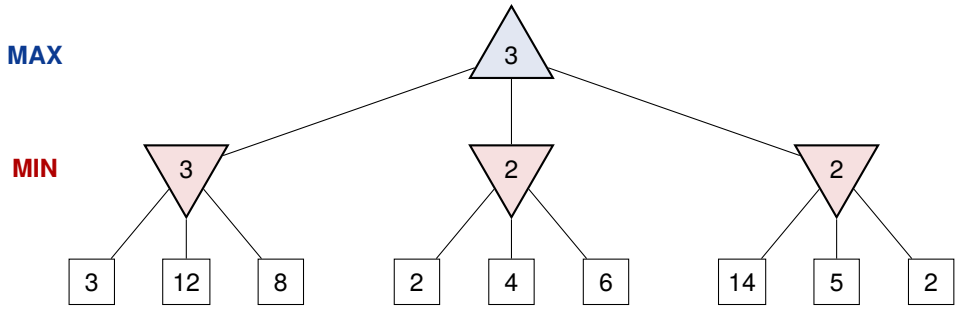
- Il giocatore **MAX** vuole *massimizzare* il punteggio: sceglie la mossa con il valore più alto restituito dalla chiamata ricorsiva.
- Il giocatore **MIN** vuole *minimizzare* il punteggio (che è quello di MAX): sceglie la mossa con il valore più basso.

```
def minimax(board, player):
    if board is leaf node:
        return value(board)
    if player = MAX:
        best = −∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MIN)
            best = max(val, best)
        return best
    else:
        best = +∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MAX)
            best = min(val, best)
        return best
```

In formula, il valore minimax di uno stato s è:

$$\mathrm{MM}(s) = \begin{cases} \text{valore}(s) & \text{se } s \text{ è terminale,} \\ \max_{s' \in \text{figli}(s)} \mathrm{MM}(s') & \text{se in } s \text{ muove MAX,} \\ \min_{s' \in \text{figli}(s)} \mathrm{MM}(s') & \text{se in } s \text{ muove MIN.} \end{cases}$$

Approfondimento (esempio numerico). Un albero a due livelli (MAX alla radice, MIN al livello sotto). Si parte dalle foglie e si risale: ogni nodo MIN prende il minimo dei figli, la radice MAX il massimo.



I tre nodi MIN valgono $\min(3, 12, 8) = 3$, $\min(2, 4, 6) = 2$, $\min(14, 5, 2) = 2$; la radice vale $\max(3, 2, 2) = 3$. MAX sceglie quindi la prima mossa.

1.5.5 I limiti di Minimax

Minimax nella versione base esplora *tutto* l'albero. Già per il tris l'albero completo ha 549 946 nodi (partendo dalla scacchiera vuota). Molti rami però sono inutili: se un ramo è già sicuramente peggiore di un'alternativa trovata prima, non serve esplorarlo.

1.5.6 Potatura Alpha-Beta

Si tengono traccia di due valori, i migliori trovati finora lungo il percorso:

- α: il valore migliore (più *alto*) che **MAX** può garantirsi al livello corrente o sopra;
- β: il valore migliore (più *basso*) che **MIN** può garantirsi al livello corrente o sopra.

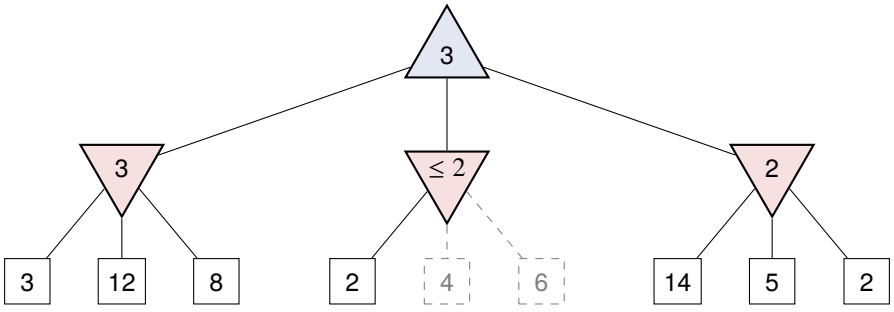
All'inizio $\alpha = -\infty$ e $\beta = +\infty$. Appena $\alpha \geq \beta$, il nodo corrente non potrà mai essere raggiunto con gioco ottimale e i figli restanti si possono **potare** (`break`).

```
def minimax(board, player, alpha, beta):
    if board is leaf node:
        return value(board)
    if player = MAX:
        best = −∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MIN, alpha, beta)
            best = max(val, best)
            alpha = max(best, alpha)
            if alpha >= beta:
                break
        return best
    else:
        best = +∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MAX, alpha, beta)
            best = min(val, best)
            beta = min(best, beta)
            if alpha >= beta:
                break
        return best
```

Approfondimento (alpha-beta sull'esempio numerico). Riprendiamo l'albero di prima, esplorando i figli da sinistra a destra.

1. Primo nodo MIN ($\alpha = -\infty, \beta = +\infty$): vede 3, 12, 8 e restituisce 3. La radice aggiorna $\alpha = 3$.
2. Secondo nodo MIN ($\alpha = 3$): la prima foglia vale 2, quindi $\beta = 2$. Ora $\alpha = 3 \geq \beta = 2$: **potatura**. Le foglie 4 e 6 non vengono visitate. *Perché è corretto*: questo nodo MIN varrà al massimo 2, ma MAX ha già un'alternativa che vale 3, quindi non sceglierà mai questo ramo.
3. Terzo nodo MIN ($\alpha = 3$): vede 14 ($\beta = 14$), 5 ($\beta = 5$), 2 ($\beta = 2$) e restituisce 2.

Il risultato è 3, identico a Minimax, ma con due foglie in meno. Il risultato di alpha-beta è *sempre* identico a quello di Minimax: cambia solo quanto lavoro si fa.



Osservazione (l'ordine delle mosse conta). Quanto si pota dipende dall'ordine in cui si esplorano i figli. Se le mosse migliori vengono esplorate per prime, α e β si stringono subito e si pota molto di più. Nel caso ideale alpha-beta visita circa $b^{m/2}$ nodi invece di b^m (b = numero di mosse per stato, m = profondità): a parità di tempo si può cercare circa due volte più in profondità.

1.5.7 Oltre alpha-beta

Alpha-beta può ridurre l'albero di oltre il 99%, ma alcuni giochi restano troppo grandi (scacchi, go, . . .). Tecniche usate in pratica:

- **ricerca a profondità fissa** / *iterative deepening*: ci si ferma a una certa profondità, aumentandola finché c'è tempo;
- **valutazione euristica** degli stati: una stima del valore di stati non terminali;
- *quiescence search*: si continua a cercare oltre il limite solo nelle posizioni "instabili" (es. in mezzo a uno scambio di pezzi);
- *hashing* e *transposition tables*: si memorizza il valore di stati già visti, raggiungibili con sequenze di mosse diverse;
- *move ordering*: si provano prima le mosse promettenti, per potare di più;
- **basi di conoscenza**: aperture e tabelle dei finali negli scacchi.

1.6 Laboratorio: Minimax per il tris

LAB

[Lab_Minimax_blank.ipynb]

Obiettivo: implementare Minimax (e poi alpha-beta) per il tris, farlo giocare contro sé stesso e contro un umano. Il codice qui sotto è una soluzione completa ed è stata verificata: le asserzioni del notebook passano e le due IA pareggiano sempre.

1.6.1 Rappresentazione e condizioni di vittoria

La scacchiera è una lista di liste 3×3: `0` = vuota, `1` = X, `2` = O. `check_winner` restituisce `1` o `2` se quel giocatore ha un tris, altrimenti `0`. `is_full` dice se non ci sono celle vuote.

```python
def check_winner(board):
    lines = []
    lines.extend(board)                                  # righe
    lines.extend([[board[i][j] for i in range(3)]
                  for j in range(3)])                    # colonne
    lines.append([board[i][i] for i in range(3)])        # diagonale
    lines.append([board[i][2 - i] for i in range(3)])    # antidiagonale
    for line in lines:
        if line[0] != 0 and line[0] == line[1] == line[2]:
            return line[0]
    return 0

def is_full(board):
    return all(cell != 0 for row in board for cell in row)
```

> **Attenzione (la cella vuota).** Il controllo `line[0] != 0` è essenziale: senza, una riga vuota `[0,0,0]` verrebbe vista come "tre uguali" e la funzione restituirebbe un vincitore inesistente.

1.6.2 Minimax

X è MAX, O è MIN. La funzione `score` traduce il vincitore nel punteggio $+1/-1/0$. Invece di copiare la scacchiera a ogni mossa, si *modifica sul posto* e poi si **annulla la mossa** (*rollback*) prima di provare la successiva.

```python
from math import inf

def score(board):
    w = check_winner(board)
    if w == 1:
        return 1       # vince X (MAX)
    if w == 2:
        return -1      # vince O (MIN)
    return 0           # pareggio

def minimax(board, is_maximizing):
    if check_winner(board) or is_full(board):   # nodo foglia
```

```
        return score(board)
    if is_maximizing:
        best = -inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 1                          # prova la mossa
                    best = max(best, minimax(board, False))
                    board[i][j] = 0                          # annulla la mossa
        return best
    else:
        best = inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 2
                    best = min(best, minimax(board, True))
                    board[i][j] = 0
        return best
```

Attenzione (pseudocodice del notebook). Nel notebook, nel ramo MAX, il `return best` è indentato *dentro* il ciclo `for`: così la funzione restituirebbe il valore dopo aver provato solo la prima mossa. Il `return` va dopo il ciclo, come nelle slide e nel codice qui sopra.

1.6.3 Scegliere la mossa migliore

`minimax` restituisce un *valore*, non una mossa. `find_best_move` prova ogni mossa legale, valuta la posizione risultante con `minimax` e tiene quella con il valore più alto (se muove il giocatore 1) o più basso (se muove il giocatore 2).

```
def find_best_move(board, player):
    move = (-1, -1)                      # nessuna mossa se la scacchiera è piena
    best = -inf if player == 1 else inf
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                board[i][j] = player
                # dopo la mossa di 'player' tocca all'avversario:
                # se ha mosso X (1) ora minimizza O, e viceversa
                val = minimax(board, player == 2)
                board[i][j] = 0
                if (player == 1 and val > best) or (player == 2 and val < best):
                    best, move = val, (i, j)
    return move
```

Attenzione (errore tipico). Dopo aver piazzato la mossa di `player` bisogna chiamare `minimax` con il turno dell'*avversario*. Chiamarla con `is_maximizing = (player == 1)` è un errore comune: l'IA valuterebbe le posizioni come se toccasse di nuovo a lei.

1.6.4 Minimax contro sé stesso

Il notebook fornisce il ciclo di gioco: i giocatori si alternano chiamando `find_best_move` finché qualcuno vince o la scacchiera è piena. Con due giocatori ottimali la partita finisce **sempre in pareggio**. Con l'implementazione sopra la partita è:

$$(0,0),\ (1,1),\ (0,1),\ (0,2),\ (2,0),\ (1,0),\ (1,2),\ (2,1),\ (2,2) \quad \Rightarrow \quad \text{pareggio}$$

X	X	O
O	O	X
X	O	X

> **Approfondimento (perché il pareggio).** In ogni gioco combinatorio finito si può calcolare il valore di ogni stato risalendo dalle foglie (**induzione all'indietro**, *backward induction*), che è esattamente ciò che fa Minimax. Ne segue che uno dei due giocatori ha una strategia che garantisce la vittoria, oppure entrambi possono garantirsi almeno il pareggio (teorema di Zermelo). Per il tris vale la seconda: il valore minimax della scacchiera vuota è 0.

1.6.5 Minimax con potatura alpha-beta

Si aggiungono i parametri `alpha` e `beta` e il controllo di potatura dopo ogni figlio. Il resto del codice (incluso `find_best_move`) non cambia, salvo chiamare `minimax_ab` al posto di `minimax`.

```python
def minimax_ab(board, is_maximizing, alpha=-inf, beta=inf):
    if check_winner(board) or is_full(board):
        return score(board)
    if is_maximizing:
        best = -inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 1
                    best = max(best, minimax_ab(board, False, alpha, beta))
                    board[i][j] = 0
                    alpha = max(alpha, best)
                    if alpha >= beta:
                        return best              # potatura
        return best
    else:
        best = inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 2
                    best = min(best, minimax_ab(board, True, alpha, beta))
                    board[i][j] = 0
                    beta = min(beta, best)
                    if alpha >= beta:
                        return best              # potatura
        return best
```

Attenzione (`break` con due cicli annidati). Qui le mosse si scorrono con *due* cicli (`i` e `j`). Un `break` uscirebbe solo dal ciclo interno e l'esterno continuerebbe a esplorare le righe successive, annullando parte della potatura. Per questo si usa `return best` (va bene perché la mossa è già stata annullata). In alternativa si può prima costruire la lista delle celle vuote e usare un solo ciclo.

Risultati misurati. Stessa partita (pareggio), stesse mosse, ma molto meno lavoro:

	Minimax	Alpha-beta
Nodi visitati dalla scacchiera vuota	549 946	18 297
Nodi visitati in un'intera partita IA contro IA	618 175	36 684
Tempo della partita (Python)	circa 2,9 s	circa 0,17 s

Alpha-beta visita circa il 3% dei nodi di Minimax, cioè ne pota circa il 97%.

1.6.6 Giocare contro l'IA

L'ultima cella del notebook permette di sfidare l'algoritmo inserendo le mosse come `riga colonna` (indici da 0). Se l'implementazione è corretta **non si può mai vincere**: al massimo si pareggia.

Riepilogo della lezione

- Teoria dei giochi = decisioni tra più agenti razionali; AGT = progettare algoritmi in ambienti strategici.
- Aree del corso: giochi non cooperativi (equilibrio di Nash), coalizionali (valore di Shapley), scelta sociale (paradosso di Condorcet), mechanism design, matching.
- Dilemma del prigioniero: efficienza (Pareto) e stabilità (Nash) possono non coincidere.
- Giochi combinatori: 2 giocatori, informazione perfetta, niente caso, finiti, a turni, a somma zero. Si rappresentano con l'albero di gioco.
- Minimax: valore di uno stato = max (MAX) o min (MIN) dei valori dei figli.
- Alpha-beta: stesso risultato, pota i rami con $\alpha \geq \beta$. Nel tris visita 18 297 nodi invece di 549 946.

Prossima lezione: teoria dell'utilità e decisioni in condizioni di incertezza; giochi strategici e formalizzazione della *best response*.

LEZIONE 2

Teoria dell'utilità

TEORIA

Nella lezione precedente le utilità erano numeri dati ($+1$, 0, -1 per il tris). Qui si fa il passo indietro: *da dove vengono* quei numeri? La risposta è che non sono un dato primitivo, ma una **rappresentazione** delle preferenze del giocatore. Il percorso della lezione è:

1. concetti di base: alternative e preferenze;
2. rappresentare preferenze ordinali (utilità **ordinale**);
3. oltre l'ordinale: lotterie e utilità **cardinale** (VNM);
4. atteggiamento verso il rischio;
5. applicazioni e introduzione alla teoria delle decisioni.

2.1 Concetti di base

2.1.1 Alternative

Un agente sceglie all'interno di un insieme X di **alternative**, che devono essere:

- **mutuamente esclusive**: se ne realizza una sola;
- **esaustive**: coprono tutti i casi possibili.

Esempio (scelta dei corsi). Le opzioni sono {Deep Learning, Algorithmic Game Theory}, ma queste *non* sono le alternative: si possono seguire entrambi i corsi o nessuno. L'insieme corretto è

$$X = \{\,\mathrm{DL},\ \mathrm{AGT},\ \mathrm{DLAGT},\ \mathrm{N}\,\},$$

dove DLAGT = entrambi e N = nessuno dei due.

Osservazione (è un passo di modellazione, non una formalità). Costruire X *bene* è la prima cosa da fare in un modello. Con n opzioni combinabili liberamente le alternative sono 2^n: già qui si vede che la dimensione dello spazio delle alternative è un problema *computazionale*, non solo concettuale.

2.1.2 Preferenze

Definizione 2.1 (Relazione di preferenza). Le **preferenze** sono una relazione $\succsim$ su X, cioè un sottoinsieme di $X \times X$. Si legge $x \succsim y$ come "x è *almeno buono quanto* y". La relazione è:

- **completa** se $\forall x, y \in X$ vale $x \succsim y$ oppure $y \succsim x$;
- **transitiva** se $\forall x, y, z \in X$, $[x \succsim y \text{ e } y \succsim z] \Rightarrow x \succsim z$.

Una preferenza è una **relazione di preferenza** se e solo se è completa e transitiva.

Da $\succsim$ si derivano altre due relazioni:

$$\begin{aligned} \text{preferenza } \textbf{stretta}: &\qquad x \succ y \iff [\, x \succsim y \text{ e } y \not\succsim x \,]; \\ \textbf{indifferenza}: &\qquad x \sim y \iff [\, x \succsim y \text{ e } y \succsim x \,]. \end{aligned}$$

Osservazione (completezza non vuol dire "decidere sempre"). La completezza non dice che l'agente preferisce sempre qualcosa a qualcos'altro: dice che sa sempre *confrontare* due alternative. Il caso "mi sono indifferenti" è ammesso ed è proprio $x \sim y$. Quello che la completezza esclude è l'incomparabilità ("non so dire").

2.2 Rappresentare le preferenze: utilità ordinale

Definizione 2.2 (Funzione di utilità). Una relazione di preferenza $\succsim$ è **rappresentata** da una funzione di utilità $u : X \to \mathbb{R}$ nel senso seguente:

$$x \succsim y \iff u(x) \geq u(y) \qquad \forall x, y \in X.$$

Se un giocatore trova x almeno buono quanto y, allora $u(x)$ deve essere almeno alto quanto $u(y)$. In questo senso il giocatore si comporta *come se* (*as if*) stesse massimizzando $u(\cdot)$.

Osservazione (la direzione della spiegazione). L'ultima frase va presa alla lettera e sarà ripresa alla fine della lezione: l'utilità è un *modo di descrivere* le preferenze, non la loro causa. Non si sceglie x perché ha utilità più alta; ha utilità più alta perché è ciò che si sceglie.

Teorema 2.3 (Rappresentazione ordinale). Sia X *finito*. Una preferenza può essere rappresentata da una funzione di utilità se e solo se è completa e transitiva. Inoltre, se $u : X \to \mathbb{R}$ rappresenta $\succsim$ e $f : \mathbb{R} \to \mathbb{R}$ è **strettamente crescente**, allora anche $f \circ u$ rappresenta $\succsim$.

Per l'ultima affermazione queste utilità si dicono **ordinali**: conta solo l'*informazione d'ordine*.

2.2.1 Le condizioni sono necessarie

Transitività. Sia $X = \{a, b, c\}$ e si supponga il ciclo $a \succ b \succ c \succ a$. Se esistesse u si avrebbe $u(a) > u(b) > u(c) > u(a)$, cioè $u(a) > u(a)$: assurdo.

Approfondimento (la *money pump*). Filosofi ed economisti direbbero che violare la transitività genera una **pompa di soldi**. Se l'agente ha il ciclo sopra e possiede c, allora preferisce strettamente b a c: gli si può vendere b in cambio di c più un piccolo importo ε. Poi preferisce a a b: altro scambio, altro ε. Poi preferisce c ad a: terzo scambio. L'agente è tornato al punto di partenza avendo pagato 3ε, e il ciclo si può ripetere all'infinito. È l'argomento "pratico" a favore della transitività: chi la viola si fa svuotare le tasche.

Completezza. Si assuma di avere una funzione di utilità $u : X \to \mathbb{R}$. Dati due numeri $u(x)$ e $u(y)$, sono ammissibili solo due possibilità: $u(x) \geq u(y)$ oppure $u(y) \geq u(x)$. Quindi $x \succsim y$ o $y \succsim x$: la completezza è una necessità matematica, ereditata dal fatto che l'ordine $\geq$ su $\mathbb{R}$ è totale.

2.2.2 Le condizioni sono sufficienti

Dimostrazione del teorema. Sia $X = \{x_1, \ldots, x_n\}$. Grazie a completezza e transitività si possono partizionare gli elementi di X in k **classi di indifferenza** $C_1, \ldots, C_k$ (ogni classe raccoglie alternative tra loro indifferenti) ordinate in modo che $C_1 \succ C_2 \succ \ldots \succ C_k$. Si definisce allora

$$u(x) = k - i + 1 \qquad \text{per ogni } x \in C_i,$$

cioè $u = k$ sulla classe migliore, $u = k - 1$ sulla successiva, fino a $u = 1$ su C_k. Per costruzione $u(x) \geq u(y) \iff x \succsim y$. □

Attenzione (dove serve che X sia finito). L'ipotesi "X finito" non è decorativa: serve a poter costruire le classi di indifferenza e a numerarle. Con X infinito completezza e transitività *non* bastano. Il controesempio classico sono le preferenze **lessicografiche** su $X = \mathbb{R}^2$, dove $(a_1, a_2) \succsim (b_1, b_2)$ se $a_1 > b_1$, oppure se $a_1 = b_1$ e $a_2 \geq b_2$: la relazione è completa e transitiva ma non ammette alcuna rappresentazione con una u a valori reali.

2.2.3 Esempi: l'utilità ordinale non è interpretabile

Con $X = \{a, b, c\}$ e preferenze $a \succ b \succ c$, tutte e tre queste funzioni rappresentano le *stesse* preferenze:

	u_1	u_2	u_3
$u(a)$	-1	30	90
$u(b)$	-2	20	2
$u(c)$	-3	10	1

Il valore numerico dell'utilità ordinale **non è interpretabile**: non dice con quale *intensità* un giocatore preferisce a a b. Con u_2 le differenze sono uguali, con u_3 il salto tra a e b sembra enorme, ma le preferenze rappresentate sono identiche.

Osservazione (perché non basta). Finché si sceglie tra alternative *certe*, l'ordine basta: si prende la migliore. Ma appena entra in gioco l'**incertezza** (una scelta dà a con probabilità 0,5 e c con probabilità 0,5) serve sapere *quanto* b sta tra a e c: serve un'utilità **cardinale**. È il problema che risolve la teoria di Von Neumann–Morgenstern.

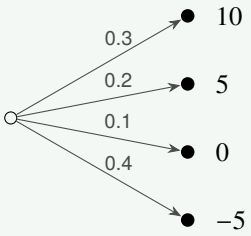
2.3 Oltre l'ordinale: le lotterie

Definizione 2.4 (Lotteria semplice). Una **lotteria semplice** è una tupla

$$L = (p_1, x_1;\ p_2, x_2;\ \ldots;\ p_n, x_n)$$

dove $x_1, \ldots, x_n \in X \subseteq \mathbb{R}$ sono **premi monetari** e $(p_1, \ldots, p_n)$ è una distribuzione di probabilità ($p_i \geq 0$, $\sum_i p_i = 1$). Si indica con $\mathcal{L}$ l'insieme delle lotterie semplici.

Esempio (una lotteria). $L = (0{,}3,\ 10;\ 0{,}2,\ 5;\ 0{,}1,\ 0;\ 0{,}4,\ -5)$.



2.3.1 Lotterie composte

Una lotteria può avere come premi altre lotterie. Siano

$$L_1 = (0{,}9:\ -1\$,\ 0{,}1:\ 5\$), \qquad L_2 = (0{,}75:\ -1\$,\ 0{,}25:\ 2\$).$$

La **lotteria composta** $L_c = (0{,}5:\ L_1,\ 0{,}5:\ L_2)$ equivale alla lotteria semplice che si ottiene moltiplicando le probabilità lungo i rami e sommando quelle che portano allo stesso premio:

$$L_e = \big(0{,}5\times0{,}9+0{,}5\times0{,}75:\ -1\$,\quad 0{,}5\times0{,}25:\ 2\$,\quad 0{,}5\times0{,}1:\ 5\$\big) = (0{,}825:\ -1\$,\ 0{,}125:\ 2\$,\ 0{,}05:\ 5\$).$$

Osservazione (ipotesi di riduzione). Questo passaggio è un'*assunzione* del modello: si suppone che al giocatore interessi solo la distribuzione finale sui premi, non il modo in cui ci si arriva (due estrazioni invece di una). Grazie a essa basta studiare $\mathcal{L}$, l'insieme delle lotterie *semplici*.

2.3.2 Valore atteso

Definizione 2.5 (Valore atteso). Per $L = (p_1, x_1; \ldots; p_n, x_n)$ il **valore atteso** è

$$\mathbb{E}(L) = \sum_{i=1}^{n} p_i\, x_i.$$

Per la lotteria dell'esempio:

$$\mathbb{E}(L) = 0{,}3 \times 10 + 0{,}2 \times 5 + 0{,}1 \times 0 + 0{,}4 \times (-5) = 2.$$

Giocando ripetutamente quella lotteria ci si aspetta di vincere in media 2.

Attenzione (il valore atteso non basta a decidere). Verrebbe da usare $\mathbb{E}$ come criterio di scelta, ma è sbagliato. Si consideri il **paradosso di San Pietroburgo**: si lancia una moneta finché non esce testa; se esce al lancio k si vincono 2^k euro. Il valore atteso è $\sum_{k\geq 1} 2^{-k} \cdot 2^k = \infty$, eppure nessuno pagherebbe più di poche decine di euro per giocare. Serve applicare l'utilità *ai premi*, non ai soldi.

2.3.3 Utilità attesa

Definizione 2.6 (Utilità attesa e funzione di Bernoulli). Data una relazione di preferenza $\succsim$ su $\mathcal{L}$, una funzione di utilità $U : \mathcal{L} \to \mathbb{R}$ è una **funzione di utilità attesa** se può essere scritta come

$$U(L) = \sum_{i=1}^{n} p_i \, u(x_i)$$

per qualche funzione $u : \mathbb{R} \to \mathbb{R}$. La funzione u si chiama **funzione di utilità di Bernoulli**.

Attenzione (due utilità diverse, due simboli diversi). u è definita sui *premi* (numeri di euro), U sulle *lotterie*. La forma di U è vincolata: deve essere **lineare nelle probabilità**. Non ogni U che rappresenta preferenze su $\mathcal{L}$ ha questa forma; il teorema VNM dice esattamente quando ce l'ha.

2.4 L'utilità di Von Neumann–Morgenstern

Von Neumann e Morgenstern (VNM) hanno fornito le condizioni sotto cui una relazione di preferenza $\succsim$ su $\mathcal{L}$ è rappresentabile da una funzione di utilità attesa.

Definizione 2.7 (Assiomi VNM).

Assioma 1 (**ordine di preferenza**) $\succsim$ è completa e transitiva.

Assioma 2 (**continuità**) se $L \succ M \succ N$, esiste $p \in [0,1]$ tale che $pL + (1-p)N \sim M$.

Assioma 3 (**indipendenza**) per ogni lotteria N e ogni $p \in [0,1]$,

$$L \succsim M \iff pL + (1-p)N \succsim pM + (1-p)N.$$

Osservazione (come leggere gli assiomi 2 e 3). La **continuità** dice che non esistono premi "infinitamente" migliori di altri: se M sta tra L e N, c'è una miscela di L e N esattamente equivalente a M. L'**indipendenza** dice che un esito *comune* alle due lotterie non deve cambiare il confronto: N compare con la stessa probabilità $1-p$ da entrambe le parti, quindi deve essere irrilevante. È l'assioma più forte dei tre e, come si vedrà con Allais, è quello che gli esseri umani violano.

Teorema 2.8 (Von Neumann–Morgenstern). Una relazione binaria $\succsim$ su $\mathcal{L}$ ammette una rappresentazione di utilità attesa se e solo se soddisfa gli assiomi 1–3. Inoltre, se U e V sono due rappresentazioni di utilità attesa di $\succsim$, allora esistono costanti $a, b \in \mathbb{R}$ con $a > 0$ tali che

$$U(\cdot) = a\,V(\cdot) + b.$$

L'ultima affermazione dice che la rappresentazione VNM è unica **a meno di trasformazioni affini**.

Osservazione (ordinale contro cardinale). È qui la differenza con il Teorema 1. Lì si poteva applicare a *u qualunque* funzione strettamente crescente; qui solo $ax + b$ con $a > 0$. Il margine di libertà è molto più stretto, e questo rende i numeri *interpretabili*: sono fissati l'origine e l'unità di misura, come per la temperatura in gradi Celsius. Ha senso dire che il salto da a a b è il doppio del salto da b a c; non ha senso dire che a vale il doppio di b. Si parla di **scala a intervalli**.

2.4.1 Necessità degli assiomi

Ordine di preferenza. Già dimostrato per il Teorema 1.

Continuità. Sia $L \succ M \succ N$; allora $u(L) > u(M) > u(N)$. Si ponga

$$p = \frac{u(M) - u(N)}{u(L) - u(N)} \in (0, 1).$$

Per la definizione di utilità attesa:

$$\begin{aligned} u\big(pL + (1-p)N\big) &= p\,u(L) + (1-p)\,u(N) \\ &= p\,u(L) + u(N) - p\,u(N) \\ &= p\big(u(L) - u(N)\big) + u(N) \\ &= \frac{u(M) - u(N)}{u(L) - u(N)}\big(u(L) - u(N)\big) + u(N) \\ &= u(M) - u(N) + u(N) = u(M). \end{aligned}$$

Quindi $pL + (1-p)N \sim M$: la continuità è necessaria.

Indipendenza. Siano $L_1 = pL + (1-p)N$ e $L_2 = pM + (1-p)N$. Per linearità dell'attesa:

$$\begin{aligned} u(L_1) &= p\,u(L) + (1-p)\,u(N), \\ u(L_2) &= p\,u(M) + (1-p)\,u(N). \end{aligned}$$

Sottraendo membro a membro, il termine comune in N si cancella:

$$u(L_1) - u(L_2) = p\big(u(L) - u(M)\big).$$

Se $L \succ M$ allora $u(L) > u(M)$, dunque $u(L_1) > u(L_2)$, cioè $L_1 \succ L_2$. L'indipendenza è necessaria.

Osservazione (da dove viene la "forza" dell'indipendenza). La cancellazione del termine $(1-p)u(N)$ funziona *perché* U è lineare nelle probabilità. Indipendenza e linearità sono in sostanza la stessa proprietà vista da due lati: uno comportamentale (l'esito comune non conta), l'altro algebrico.

2.4.2 Sufficienza: dimostrazione del teorema VNM

La dimostrazione si divide in due parti:

- **Parte 1**: si mostra che $U(L) = \sum_i p_i\,u(x_i)$;
- **Parte 2**: si mostra che $L \succ M \iff U(L) > U(M)$ per ogni $L, M \in \mathcal{L}$.

Parte 1. Si supponga di avere n esiti $o_1, \dots, o_n$. Per completezza e transitività si possono ordinare dal peggiore al migliore: $o_1 \precsim \dots \precsim o_n$ (nel senso o_n migliore). Si fissano i due estremi della scala:

$$u(o_1) = 0, \qquad u(o_n) = 1.$$

Si definisce la famiglia di lotterie semplici

$$L(p) = p \cdot o_n + (1 - p) \cdot o_1,$$

cioè "il premio migliore con probabilità p, il peggiore con probabilità $1 - p$". Per l'assioma di continuità, per ogni esito o_i esiste una probabilità $q_i \in [0, 1]$ tale che $L(q_i) \sim o_i$; si pone $u(o_i) = q_i$. Ne segue che l'utilità di una lotteria $M = \sum_i p_i o_i$ è l'attesa di u:

$$u(M) = u\Big(\sum_i p_i o_i\Big) = \sum_i p_i \, u(o_i) = \sum_i p_i \, q_i.$$

> **Osservazione (che cosa è davvero q_i).** q_i è un modo operativo di *misurare* l'utilità: è la probabilità di vincere il premio migliore che rende il giocatore indifferente a ricevere o_i con certezza. È una definizione che si può realizzare come esperimento, chiedendo alla persona di aggiustare q finché non è indifferente, ed è esattamente questo che rende la scala cardinale invece che ordinale: q_i posiziona o_i *tra* i due estremi.

Parte 2. Per l'assioma di continuità il giocatore è indifferente tra l'esito o_i e la lotteria $q_i \cdot o_n + (1 - q_i) \cdot o_1$. Insieme all'assioma di indipendenza (si sostituisce un esito alla volta dentro la lotteria) il giocatore è indifferente tra le due lotterie

$$M = \sum_i p_i o_i \qquad \text{e} \qquad M' = \sum_i p_i \big[\, q_i \cdot o_n + (1 - q_i) \cdot o_1 \,\big].$$

Ma M' coinvolge solo i due esiti estremi, quindi si può riscrivere come lotteria semplice:

$$M' = \Big(\sum_i p_i q_i\Big) \cdot o_n + \Big(\sum_i p_i (1 - q_i)\Big) \cdot o_1 = U(M) \cdot o_n + \big(1 - U(M)\big) \cdot o_1.$$

Ogni lotteria è dunque equivalente a una lotteria sui soli due estremi, la cui unica caratteristica è la probabilità $U(M)$ di ottenere il premio migliore.

Si supponga ora $L \succ M$ e si definiscano

$$L' = U(L) \cdot o_n + \big(1 - U(L)\big) \cdot o_1,$$
$$M' = U(M) \cdot o_n + \big(1 - U(M)\big) \cdot o_1.$$

Vale l'ordine $L' \sim L \succ M \sim M'$, quindi $L' \succ M'$. Ma L' e M' differiscono solo per la probabilità del premio migliore, e tra due lotterie di questa famiglia si preferisce quella con la probabilità più alta: dunque $U(L) > U(M)$. Vale anche il viceversa, quindi $L \succ M \iff U(L) > U(M)$. □

2.5 Il paradosso di Allais

Si risponda d'istinto a due domande. *Prima*: quale lotteria si preferisce?

$$L = (1\text{: } 1.000.000\$), \qquad M = (0{,}89\text{: } 1.000.000\$,\ \ 0{,}10\text{: } 5.000.000\$,\ \ 0{,}01\text{: } 0\$).$$

Seconda: e tra queste?

$$N = (0{,}89\colon 0\$,\ \ 0{,}11\colon 1.000.000\$), \qquad O = (0{,}90\colon 0\$,\ \ 0{,}10\colon 5.000.000\$).$$

La maggior parte delle persone sceglie L nella prima coppia (il milione sicuro) e O nella seconda. Questa combinazione è **incompatibile** con la teoria dell'utilità attesa.

Approfondimento (perché è una contraddizione). Si normalizzi $u(0) = 0$ e si scrivano $u_1 = u(1\text{M})$, $u_5 = u(5\text{M})$.

- $L \succ M$ significa $u_1 > 0{,}89\,u_1 + 0{,}10\,u_5$, cioè $0{,}11\,u_1 > 0{,}10\,u_5$;
- $O \succ N$ significa $0{,}10\,u_5 > 0{,}11\,u_1$.

Le due disuguaglianze si contraddicono: *nessuna* funzione di utilità u può giustificare entrambe le scelte.
Visto con l'assioma di indipendenza è ancora più netto. Si scriva ogni lotteria come miscela con $p = 0{,}11$:

	parte con probabilità 0,11	parte con probabilità 0,89
L	1M certo	1M
M	$(\frac{10}{11}\colon 5\text{M},\ \frac{1}{11}\colon 0)$	1M
N	1M certo	0
O	$(\frac{10}{11}\colon 5\text{M},\ \frac{1}{11}\colon 0)$	0

Le coppie (L, M) e (N, O) differiscono nella prima colonna esattamente allo stesso modo; la seconda colonna è un esito *comune* (dentro ogni coppia). Per l'indipendenza l'esito comune non deve contare, quindi chi preferisce L a M deve preferire N a O.

Osservazione (che cosa insegna il paradosso). Allais non dimostra che la teoria VNM è sbagliata: dimostra che è **normativa** e non **descrittiva**. Dice come si *dovrebbe* decidere per essere coerenti, non come le persone decidono davvero. L'effetto in gioco si chiama *certainty effect*: il salto da 0,99 a 1 (certezza) pesa psicologicamente molto più del salto da 0,10 a 0,11. Le teorie descrittive alternative (per esempio la *prospect theory* di Kahneman e Tversky) nascono proprio da qui.

2.6 Atteggiamento verso il rischio

Definizione 2.9 (Lotteria equa). Una lotteria $L = (p\colon x,\ (1-p)\colon y)$ è **equa** se il suo valore atteso è nullo:

$$\mathbb{E}(L) = p\,x + (1-p)\,y = 0.$$

Attenzione (notazione delle slide). Le slide scrivono "$L = p \cdot x + (1-p) \cdot y = 0$". La lotteria L è un oggetto (una distribuzione), non un numero: a essere nullo è il suo *valore atteso* $\mathbb{E}(L)$. Si tratta di un abuso di notazione, ma la sostanza è "giocare questa lotteria non costa e non rende nulla, in media".

Definizione 2.10 (Atteggiamenti verso il rischio). Un giocatore con funzione di utilità di Bernoulli u è:

- **neutrale al rischio** se u è **lineare**, $u(x) = ax + b$ (con $a > 0$): è indifferente di fronte alle lotterie eque;
- **avverso al rischio** se u è **concava**: se strettamente concava, non gioca *nessuna* lotteria equa;
- **amante del rischio** (*risk seeking*) se u è **convessa**: se strettamente convessa, gioca *tutte* le lotterie eque.

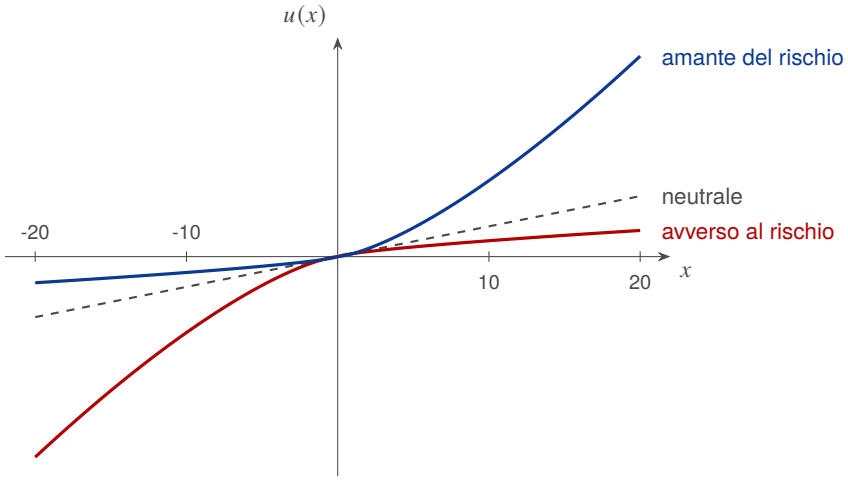


Figura 2.1: Le tre forme della funzione di utilità sui guadagni. L'avverso al rischio soffre una perdita più di quanto goda un guadagno della stessa entità; per l'amante del rischio vale il contrario.

Approfondimento (perché concavità significa avversione al rischio). È la **disuguaglianza di Jensen**. Se u è concava, per ogni lotteria L

$$\mathbb{E}\big[u(L)\big] \;\leq\; u\big(\mathbb{E}[L]\big),$$

cioè l'utilità del gioco è al più l'utilità di incassare con certezza il valore atteso. Chi è avverso al rischio preferisce quindi sempre il certo all'aleatorio di pari media; per una lotteria equa aggiunta alla propria ricchezza, non giocare è almeno altrettanto buono che giocare.
Si definisce **equivalente certo** (*certainty equivalent*) $CE(L)$ il premio certo che dà la stessa utilità della lotteria, cioè $u(CE(L)) = \mathbb{E}[u(L)]$. La differenza

$$\mathbb{E}[L] - CE(L)$$

è il **premio per il rischio**: quanto il giocatore è disposto a rinunciare per eliminare l'incertezza. Per un avverso al rischio è positivo, ed è esattamente la quantità su cui si regge il mercato assicurativo.

2.7 Applicazioni

In entrambe le applicazioni la funzione di Bernoulli è $u(x) = \sqrt{x}$, che è strettamente concava: il giocatore è avverso al rischio.

2.7.1 Condivisione del rischio (*risk sharing*)

Due giocatori avversi al rischio, con $u(x) = \sqrt{x}$, possiedono due attività rischiose *indipendenti*

$$A_1 = A_2 = (0{,}5,\ 100;\ 0{,}5,\ 0).$$

L'utilità di ciascuna attività, presa da sola, è

$$u(A_1) = u(A_2) = 0{,}5 \times \sqrt{100} + 0{,}5 \times \sqrt{0} = 5.$$

Se i due giocatori formano un **fondo comune** in cui ciascuno possiede metà del totale, ogni giocatore si ritrova con l'attività

$$A_m = (0{,}25,\ 100;\ 0{,}5,\ 50;\ 0{,}25,\ 0),$$

la cui utilità è

$$u(A_m) = 0{,}25 \times \sqrt{100} + 0{,}5 \times \sqrt{50} + 0{,}25 \times \sqrt{0} \approx 2{,}5 + 3{,}54 = 6{,}04 > 5.$$

Entrambi i giocatori stanno **meglio** pur non avendo cambiato il valore atteso.

> **Approfondimento (da dove viene A_m e perché funziona).** Le due attività sono indipendenti, quindi i quattro casi (vince A_1 sì/no, A_2 sì/no) hanno probabilità 0,25 ciascuno. Il montante totale è 200, 100, 100, 0; metà del fondo vale quindi 100, 50, 50, 0, da cui $A_m = (0{,}25, 100;\ 0{,}5, 50;\ 0{,}25, 0)$. Il valore atteso è rimasto 50 in entrambi i casi: quello che è cambiato è la *varianza*, scesa da 2500 a 1250. Mettere insieme rischi indipendenti li **diversifica**, e per un giocatore con u concava meno varianza a parità di media significa più utilità. È il principio su cui si reggono i fondi comuni di investimento e le mutue assicurative.

2.7.2 Assicurazione

Un giocatore avverso al rischio con $u(x) = \sqrt{x}$ possiede l'attività $A = (0{,}5,\ 100;\ 0{,}5,\ 0)$. Esiste una compagnia di assicurazione **neutrale al rischio** con molto capitale.

Quanto è disposto a pagare il giocatore? Il giocatore paga un premio P e in cambio riceve con certezza 100; il suo patrimonio certo è quindi $100 - P$. Accetta se questo gli dà almeno l'utilità della lotteria:

$$u(100 - P) \ \geq\ 0{,}5 \times u(100) + 0{,}5 \times u(0) \implies \sqrt{100 - P} \geq 5 \implies 100 - P \geq 25 \implies P \leq 75.$$

Quanto vuole la compagnia? Essendo neutrale al rischio, le basta che il premio copra il valore atteso del risarcimento:

$$P \ \geq\ 0{,}5 \times 100 + 0{,}5 \times 0 \implies P \geq 50.$$

Conclusione. Entrambe le parti guadagnano se il contratto si stipula a un premio

$$P \in [50,\ 75].$$

> **Osservazione (lettura con l'equivalente certo).** $\sqrt{100 - P} \geq 5$ significa $100 - P \geq 25$: l'equivalente certo della lotteria è $CE(A) = 25$, mentre il suo valore atteso è $\mathbb{E}(A) = 50$. Il premio per

il rischio è $50 - 25 = 25$, ed è esattamente l'ampiezza dell'intervallo $[50, 75]$: il **surplus** che le due parti si dividono. L'assicurazione crea valore perché sposta il rischio da chi lo odia a chi gli è indifferente.

Attenzione (il patrimonio iniziale conta). Qui si è assunto implicitamente che il giocatore non abbia altra ricchezza: l'utilità si calcola su $100 - P$ e su 100 o 0. Se avesse un patrimonio iniziale w, le utilità sarebbero $u(w + 100 - P)$ e $u(w + 100)$, $u(w)$, e il premio massimo accettabile sarebbe diverso (più basso in termini relativi: più si è ricchi, meno si è avversi al rischio con $u(x) = \sqrt{x}$). La funzione di Bernoulli si applica alla *ricchezza finale*, non al guadagno della singola scommessa.

2.8 Riepilogo sulla teoria VNM

- La teoria VNM fornisce un modo per costruire funzioni di utilità **cardinali**: i valori numerici sono interpretabili su una **scala a intervalli** e, in certe situazioni, su una scala di rapporti.
- La teoria VNM fornisce una teoria **normativa** della decisione: massimizzare l'utilità attesa. È molto usata in applicazioni finanziarie ed economiche.
- La teoria VNM **non descrive** il comportamento umano. Un'alternativa non è preferita perché ha un'utilità più alta; piuttosto, le si associa un'utilità più alta perché è l'alternativa preferita.

2.9 Laboratorio: Battaglia navale e ricerca bayesiana

LAB

[Battleship_Blank.ipynb, Lab_2_Battleship.pdf]

Obiettivo: costruire un'IA per la **battaglia navale**, partendo da strategie euristiche semplici e arrivando a una strategia **adattiva** che a ogni turno spara dove l'*utilità attesa* è massima. Quest'ultima tecnica si chiama **ricerca bayesiana** (*Bayesian search*) ed è uno degli algoritmi più efficaci per i problemi di ricerca e di "caccia al tesoro".

Il collegamento con la teoria della lezione è diretto: non c'è un avversario strategico, ma c'è **incertezza**. Si deve scegliere l'azione (la cella su cui sparare) che massimizza l'utilità attesa, dove la probabilità viene aggiornata a ogni colpo con le informazioni raccolte.

2.9.1 Regole e impostazione

Il campo è una griglia 10×10 su cui sono disposte cinque navi, in orizzontale o in verticale, senza sovrapposizioni e senza uscire dal bordo:

Nave	Celle
Portaerei (*Carrier*)	5
Corazzata (*Battleship*)	4
Incrociatore (*Cruiser*)	3
Sommergibile (*Submarine*)	3
Cacciatorpediniere (*Destroyer*)	2
Totale	**17**

Non essendoci un avversario vero, le strategie non si valutano in partite vinte ma in **numero di turni necessari ad affondare tutte le navi**: 17 è il minimo teorico (nessun colpo a vuoto), 100 il massimo.

Lo stato della conoscenza si tiene in una matrice `shot_grid` con `0` = cella inesplorata, `1` = colpo a vuoto, `2` = colpo a segno.

> **Attenzione (un bug nel generatore fornito).** La funzione `create_random_disposition` del notebook estrae le coordinate così:
>
> ```
> if horizontal:
>     x = np.random.randint(0, grid_size - 1)        # riga
>     y = np.random.randint(0, grid_size - length)   # colonna di partenza
> else:
>     x = np.random.randint(0, grid_size - length)
>     y = np.random.randint(0, grid_size - 1)
> ```
>
> `np.random.randint(0, n)` esclude `n`. Quindi per una nave orizzontale la riga sta in $[0, 8]$ (la riga 9 non esce mai) e la colonna di partenza in $[0,\ 9 - \texttt{length}]$, cioè l'ultima cella occupata arriva al più alla colonna 8. Simmetricamente per le navi verticali. Il risultato è che **l'ultima riga e l'ultima colonna non sono mai occupate**: su 500 partite generate, le celle della riga 9 e della colonna 9 non contengono mai una nave, mentre con il generatore corretto sono occupate rispettivamente 642 e

660 volte. Gli estremi corretti sarebbero `randint(0, grid_size)` e `randint(0, grid_size - length + 1)`.
Conseguenza pratica: con il generatore del notebook il 19% del campo è di fatto acqua garantita e le cifre misurate sono un po' ottimistiche. Più sotto i risultati sono riportati con entrambi i generatori: la differenza arriva a circa 3 turni.

2.9.2 Strategia 1: tiro casuale

La strategia più semplice: si spara a caso, ignorando del tutto gli esiti. Non è una vera strategia, ma serve da **riferimento** (*baseline*): battere il caso è il minimo sindacale.

```python
def random_attack_strategy(grid):
    shot_grid = np.zeros((10, 10), dtype=int)
    cells = [(i, j) for i in range(10) for j in range(10)]
    np.random.shuffle(cells)
    hits = turns = 0
    for (i, j) in cells:
        shot_grid[i, j] = 2 if grid[i, j] else 1
        turns += 1
        if grid[i, j]:
            hits += 1
            if hits == 17:              # tutte le navi affondate
                break
    return shot_grid, turns
```

Osservazione (mescolare invece di estrarre). Sparare "a caso" si può implementare estraendo una cella e riprovando se è già stata colpita, ma verso la fine della partita quasi tutte le celle sono esplorate e il ciclo di riprova diventa lento. Mescolare una volta la lista delle 100 celle e scorrerla dà la stessa distribuzione senza riprove.

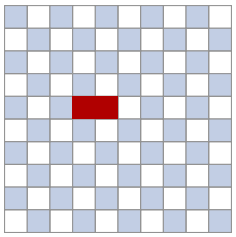
2.9.3 Strategia 2: *hunt and target*

La strategia casuale va male perché tratta tutte le celle come equiprobabili e *butta via* l'informazione dei colpi a segno. L'idea di **hunt and target** è alternare due modalità:

- **hunt** (caccia): si cerca a caso finché non si colpisce qualcosa;
- **target** (bersaglio): appena si colpisce, si spara nelle celle adiacenti (sopra, sotto, destra, sinistra) non ancora esplorate, finché la nave è affondata.

È il classico compromesso tra **esplorazione** e **sfruttamento**.

In più, in modalità hunt non serve sparare ovunque: la nave più piccola occupa 2 celle, quindi una nave non può nascondersi dentro una sola "casella nera" di una **scacchiera**. Basta sparare sulle celle con $(i + j)$ pari: 50 celle invece di 100, con la garanzia di intercettare comunque ogni nave.



Le celle della scacchiera (in blu) bastano: qualunque nave da 2 celle (in rosso) ne tocca almeno una.

```python
def hunt_and_target_strategy(grid):
    shot_grid = np.zeros((10, 10), dtype=int)
    hunt = [(i, j) for i in range(10) for j in range(10) if (i + j) % 2 == 0]
    np.random.shuffle(hunt)
    targets = []                        # pila delle celle da verificare
    hits = turns = 0
    while hits < 17:
        if targets:
            i, j = targets.pop()
        else:
            i, j = hunt.pop()
        if shot_grid[i, j]:             # gia' esplorata: salta il turno
            continue
        shot_grid[i, j] = 2 if grid[i, j] else 1
        turns += 1
        if grid[i, j]:
            hits += 1
            for di, dj in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                a, b = i + di, j + dj
                if 0 <= a < 10 and 0 <= b < 10 and shot_grid[a, b] == 0:
                    targets.append((a, b))
    return shot_grid, turns
```

Attenzione (la pila può contenere celle già colpite). Una cella può finire nella pila `targets` due volte (da due colpi a segno diversi), oppure essere colpita nel frattempo. Il controllo `if shot_grid[i, j]: continue` prima di sparare è indispensabile: senza, si conterebbero turni per colpi mai sparati e il risultato sarebbe gonfiato.

Approfondimento (si può fare meglio in modalità target). L'implementazione sopra è "la più semplice": spara alle quattro celle adiacenti senza ragionare. Un miglioramento classico è sfruttare l'**orientamento**: dopo due colpi a segno allineati si sa che la nave è orizzontale o verticale, quindi si possono scartare le celle adiacenti nell'altra direzione. La ricerca bayesiana della prossima sezione ottiene questo effetto *automaticamente*, senza regole scritte a mano.

2.9.4 La teoria della ricerca bayesiana

[Lab_2_Battleship.pdf]

L'idea della strategia 3 viene dalla **teoria della ricerca bayesiana** (*Bayesian search theory*), nata negli anni '40 per ritrovare relitti di navi, aerei e sottomarini. Si sovrappone una griglia all'area in cui si trova

l'oggetto e si guida la ricerca calcolando e ricalcolando la posizione *più probabile*: è un algoritmo iterativo e adattivo basato sul **massimo a posteriori**.

Definizione 2.11 (Ricerca bayesiana, schema generale).

1. **Inizializzazione**: si calcola la distribuzione a priori con l'informazione disponibile.
2. **A ogni iterazione**:
   - *scelta dell'azione*: si sceglie la cella di massima utilità;
   - *osservazione*: si esegue l'azione e se ne osserva l'esito;
   - *aggiornamento*: si aggiorna la distribuzione a posteriori in base all'esito.

Il caso base: un oggetto in n scatole

Ci sono n scatole $S = \{1, \dots, n\}$ e un solo oggetto. Il vettore a priori è $\mathbf{p}^{(0)} = (p_1^{(0)}, \dots, p_n^{(0)})$, dove $p_i^{(0)}$ è la probabilità che l'oggetto sia nella scatola i. Ispezionare la scatola i costa c_i (tempo o denaro) e la ricerca è **imperfetta**: se l'oggetto è in i, ispezionando i lo si trova con probabilità $q_i \in (0, 1]$ (**probabilità di rilevazione**).

Si supponga di ispezionare al passo t la scatola k senza trovare nulla (evento F_k). La probabilità di questo evento è $\Pr(F_k) = 1 - p_k^{(t)} q_k$, e con il teorema di Bayes:

$$\text{scatola ispezionata } k: \quad p_k^{(t+1)} = \Pr(\text{oggetto in } k \mid F_k) = \frac{\Pr(F_k \mid \text{oggetto in } k)\, p_k^{(t)}}{\Pr(F_k)} = \frac{(1 - q_k)\, p_k^{(t)}}{1 - p_k^{(t)} q_k},$$

$$\text{altre scatole } j \neq k: \quad p_j^{(t+1)} = \Pr(\text{oggetto in } j \mid F_k) = \frac{p_j^{(t)}}{1 - p_k^{(t)} q_k}.$$

Osservazione (come leggere l'aggiornamento). Un'ispezione fallita *abbassa* la probabilità della scatola ispezionata (non la azzera, se $q_k < 1$: l'oggetto potrebbe esserci e non essere stato visto) e *alza* in proporzione quella di tutte le altre. Il denominatore comune serve solo a rinormalizzare: le probabilità tornano a sommare 1.

Definizione 2.12 (Indice di ricerca). L'utilità di ispezionare la scatola i al tempo t è la probabilità di trovare l'oggetto *per unità di costo*:

$$I_i^{(t)} = \frac{p_i^{(t)} q_i}{c_i}.$$

Teorema 2.13 (ottimalità della ricerca golosa). Quando si cerca *un solo* oggetto in n scatole *indipendenti*, la strategia **golosa** (*greedy*) che a ogni passo ispeziona la scatola con l'indice $I_i^{(t)}$ più alto è ottimale.

Esempio (il volo Air France 447). Il 1° giugno 2009 il volo AF 447 (Rio de Janeiro–Parigi) scomparve sull'Atlantico. Le slide mostrano l'area di ricerca: l'ultima posizione nota, la rotta prevista e un cerchio di 40 miglia nautiche attorno, con il prior e un posteriore disegnati sulla griglia. Secondo il paper di Stone et al. incluso nel materiale, il prior era una *miscela* di tre distribuzioni: uni-

> forme nel cerchio, basata sui dati di incidenti precedenti, e ottenuta ripercorrendo all'indietro la deriva dei rottami recuperati. Le campagne di ricerca degli anni precedenti, fallite, sono state usate come ispezioni senza successo per aggiornarlo, tenendo conto anche della possibilità che i segnalatori acustici delle scatole nere non avessero funzionato. Nel 2011 la ricerca cominciò dal centro del posteriore e trovò il relitto in una settimana. [Search_for_the_Wreckage_of_Air_France_Flight_AF_447.pdf]

2.9.5 Strategia 3: ricerca bayesiana per la battaglia navale

Nella battaglia navale i costi sono uniformi ($c_i = 1$) e la rilevazione è perfetta ($q_i = 1$: se si spara su una nave la si colpisce). Ma ci sono **5 bersagli** e le caselle **non sono indipendenti** (una nave occupa celle contigue): l'ottimalità della strategia golosa non vale più. La ricerca bayesiana resta però un'ottima euristica:

1. si calcola una mappa di probabilità sulla griglia per *ciascuna* nave (5 mappe di densità);
2. le si somma in un'unica mappa (il *prior*);
3. a ogni colpo si sa se è a segno o a vuoto e si aggiorna la mappa (il *posteriore*).

Come si costruisce la mappa. Per ogni cella e per ogni nave **non ancora affondata** si contano le **configurazioni valide**, cioè le pose (orizzontali o verticali) della nave che coprono quella cella senza uscire dal bordo e senza toccare celle incompatibili. Su un campo vuoto il conteggio favorisce il centro: una cella d'angolo è coperta da poche pose, una centrale da molte. Sommando le cinque navi si ottiene il prior:

10	15	19	21	22	22	21	19	15	10
15	20	24	26	27	27	26	24	20	15
19	24	28	30	31	31	30	28	24	19
21	26	30	32	33	33	32	30	26	21
22	27	31	33	34	34	33	31	27	22
22	27	31	33	34	34	33	31	27	22
21	26	30	32	33	33	32	30	26	21
19	24	28	30	31	31	30	28	24	19
15	20	24	26	27	27	26	24	20	15
10	15	19	21	22	22	21	19	15	10

Prior: numero di pose valide che coprono ogni cella, sommato sulle cinque navi (calcolato col codice sotto).

> **Osservazione (da dove vengono 10 e 34).** Nell'angolo $(0, 0)$ ogni nave ha esattamente 2 pose che lo coprono (una orizzontale, una verticale): $5 \times 2 = 10$. Al centro una nave da ℓ celle ha ℓ pose orizzontali e ℓ verticali: $2(5 + 4 + 3 + 3 + 2) = 34$. Il rapporto 3,4 tra centro e angolo è il motivo per cui conviene cominciare dal centro.

Aggiornamento dopo un colpo a vuoto. Una nave non può stare su una cella risultata vuota: si scartano tutte le pose che la coprono e si ricontano.

Aggiornamento dopo un colpo a segno. Si entra in **modalità target** per affondare la nave colpita: si contano *solo* le configurazioni valide che **passano per il colpo a segno**. Si resta in modalità target finché c'è almeno una nave colpita ma non ancora affondata.

Affondamento. Quando una nave affonda, l'avversario dichiara quale: la si toglie dall'elenco di quelle da cercare e le sue celle smettono di contare come colpi a segno "aperti".

```python
SHIPS = {1: 5, 2: 4, 3: 3, 4: 3, 5: 2}       # id nave -> lunghezza
UNEXPLORED, MISS, HIT, SUNK = 0, 1, 2, 3

def placements(length):
    out = []
    for horizontal in (True, False):
        for i in range(10 if horizontal else 10 - length + 1):
            for j in range(10 - length + 1 if horizontal else 10):
                out.append([(i, j + k) if horizontal else (i + k, j)
                            for k in range(length)])
    return out

PLACEMENTS = {L: placements(L) for L in set(SHIPS.values())}

def likelihood_map(shot, remaining):
    post = np.zeros((10, 10))
    targeting = (shot == HIT).any()          # c'e' una nave colpita e non affondata
    for length in remaining:                 # solo le navi ancora a galla
        for cells in PLACEMENTS[length]:
            if any(shot[a, b] in (MISS, SUNK) for a, b in cells):
                continue                     # posa incompatibile
            n_hit = sum(shot[a, b] == HIT for a, b in cells)
            if targeting and n_hit == 0:
                continue                     # in target: solo pose per i colpi
            for a, b in cells:
                if shot[a, b] == UNEXPLORED:
                    post[a, b] += n_hit if targeting else 1
    return post

def bayesian_search_strategy(grid):
    shot = np.zeros((10, 10), dtype=int)
    remaining = list(SHIPS.values())
    hits_on = {sid: 0 for sid in SHIPS}
    turns = 0
    while remaining:
        post = likelihood_map(shot, remaining)
        i, j = np.unravel_index(np.argmax(post), post.shape)
        turns += 1
        sid = grid[i, j]
        if sid == 0:
            shot[i, j] = MISS
            continue
        shot[i, j] = HIT
        hits_on[sid] += 1
        if hits_on[sid] == SHIPS[sid]:       # affondata: l'avversario lo dichiara
            shot[grid == sid] = SUNK
            remaining.remove(SHIPS[sid])
    return shot, turns
```

Osservazione (il peso in modalità target). Ogni posa che passa per h colpi a segno viene contata h volte, una per ciascun colpo attraverso cui "passa". È esattamente quello che si ottiene seguendo alla lettera le slide ("per ogni colpo a segno, conta le configurazioni che ci passano") e ha l'effetto desiderato: dopo due colpi allineati, le pose lungo quella linea pesano doppio e la strategia prosegue nella direzione giusta senza regole scritte a mano.

Osservazione (dov'è il teorema di Bayes). Il nome "bayesiana" è giustificato anche se nel codice non compare la formula. Il prior è uniforme su tutte le pose di ogni nave; una cella risultata a vuoto ha verosimiglianza 0 per ogni posa che la occupa, quindi tutte quelle pose vengono eliminate dal posteriore. Contare le pose sopravvissute è calcolare la probabilità a posteriori marginale di ogni cella, a meno della costante di normalizzazione, che non serve perché si cerca solo l'*argmax*. È il caso $q_i = 1$ dell'aggiornamento visto sopra: una ricerca fallita azzera la cella.

Attenzione (le navi si contano una alla volta, non insieme). Il conteggio tratta ogni nave come se fosse sola sul campo: due navi possono "sovrapporsi" nel conteggio anche se nella realtà non possono. Il calcolo esatto richiederebbe di enumerare le disposizioni *congiunte* delle cinque navi compatibili con tutti i dati, che sono troppe per essere contate a ogni turno (vedi l'analisi informativa più sotto). Sommare le mappe delle singole navi è un'approssimazione: è il motivo per cui le slide la chiamano *euristica*.

Attenzione (senza le navi affondate il programma si blocca). Una prima versione "ingenua" di questa strategia, che *non* tiene traccia delle navi affondate e cerca sempre tutte e cinque, entra in **ciclo infinito**. Succede così: una nave da 2 celle viene colpita tutta e si trova circondata da celle già esplorate. I suoi colpi a segno restano "aperti", quindi si è in modalità target; ma l'unica posa che li spiega non tocca alcuna cella inesplorata e quindi non assegna voti a nessuno. Se tutti i colpi aperti sono in questa condizione il posteriore è *tutto zero*, `argmax` restituisce sempre $(0, 0)$ e si spara all'infinito sulla stessa cella.
Con la traccia delle navi affondate il problema scompare da solo: un colpo a segno aperto appartiene sempre a una nave ancora a galla, e la posa *vera* di quella nave è valida e tocca almeno una cella inesplorata, quindi il posteriore non può essere nullo. Nella simulazione lo si è controllato con un `assert post[i, j] > 0` a ogni turno, su tutte le partite.

Approfondimento (una semplificazione della simulazione). Quando una nave affonda, il codice marca come `SUNK` esattamente le sue celle leggendole dalla griglia vera (`shot[grid == sid] = SUNK`). Un giocatore reale sa *quale* nave è affondata ma non sempre *quali* dei suoi colpi a segno le appartenessero (due navi possono toccarsi). Di solito lo si deduce, ma non sempre: la simulazione è quindi leggermente ottimistica rispetto al gioco vero.

2.9.6 Risultati misurati

Le quattro strategie (le tre del notebook più la versione bayesiana senza navi affondate, per confronto) sono state eseguite sulle *stesse* 500 partite, generate sia con il generatore del notebook sia con quello corretto.

	Casuale	Hunt and target	Bayes senza affondate	Bayes con affondate
Generatore del notebook (riga 9 e colonna 9 sempre vuote)				
Turni medi	95,2	60,9	54,6	**41,3**
Deviazione standard	5,1	9,1	9,4	9,0
Mediana	97	62	54	41
Minimo – massimo	68–100	34–79	27–77	23–67
Partite entro 50 turni	0%	16,0%	33,8%	84,4%
Generatore corretto				
Turni medi	95,6	61,3	56,1	**44,3**
Deviazione standard	4,4	8,9	8,6	8,4
Mediana	97	62	56	43
Minimo – massimo	76–100	31–80	31–78	27–68
Partite entro 50 turni	0%	12,0%	25,0%	76,8%

- Il primo salto (dal caso a *hunt and target*) fa risparmiare circa 35 turni.
- Il secondo è la ricerca bayesiana *fatta bene*: altri 20 turni in meno, con oltre tre partite su quattro chiuse entro 50 colpi. I 41–44 turni misurati sono in linea con i 40–44 dichiarati nelle slide del docente.
- Senza la traccia delle navi affondate la ricerca bayesiana guadagna solo 6 turni su hunt and target: **l'informazione dell'affondamento vale da sola circa 13 turni**.
- Il bug del generatore abbassa di circa 3 turni la media della ricerca bayesiana (41,3 contro 44,3) e di meno di un turno quella di hunt and target. Con le navi confinate in una griglia 9×9 le configurazioni possibili sono meno, quindi c'è meno informazione da scoprire: ne approfitta di più la strategia che sfrutta meglio l'informazione.

2.9.7 Analisi in teoria dell'informazione

Quanta informazione c'è in una partita. Su una griglia 10×10 una nave da 2 celle si può posare in $2 \times 10 \times 9 = 180$ modi, una da 3 in 160, una da 4 in 140, una da 5 in 120. Le configurazioni possibili delle cinque navi sono quindi

$$\Omega = 180 \times 160 \times 160 \times 140 \times 120 = 7{,}74 \cdot 10^{10} \approx 2^{36{,}17}.$$

Circa metà sono inammissibili perché le navi si sovrappongono: le configurazioni legali sono circa 2^{35}, quindi una disposizione contiene circa **35 bit** di informazione.

Quanta ne porta un colpo. L'informazione media di un colpo è la sua **entropia di Shannon**, $H = -\sum_i p_i \log_2 p_i$. Se si colpisce una nave con probabilità $17/100$ e si manca con probabilità $83/100$:

$$H(\text{colpo}) = -\left(\tfrac{17}{100}\log_2\tfrac{17}{100} + \tfrac{83}{100}\log_2\tfrac{83}{100}\right) \approx 0{,}66 \text{ bit}, \qquad \mathbb{E}(\#\text{colpi}) \approx \frac{35 \text{ bit}}{0{,}66 \text{ bit/colpo}} \approx 53.$$

La ricerca bayesiana ne impiega 40–44, il 17% in meno: estrae circa $35/42 \approx 0{,}8$ bit per colpo, perché sceglie le celle in modo da *livellare* il posteriore e rendere ogni tiro il più informativo possibile.

Attenzione (53 non è un limite inferiore). Le slide dicono che la ricerca bayesiana "migliora il limite del 17%". Un limite che si può battere non è un limite: 53 è il numero di colpi che servirebbero se *ogni* colpo avesse probabilità di successo $17/100$, cioè se fosse informativo quanto un tiro a caso. Un buon giocatore sceglie celle con probabilità di successo più vicina a $1/2$, e un colpo con due esiti porta al più 1 bit quando i due esiti sono equiprobabili. Il vero limite informativo è quindi attorno ai *35 colpi* (ignorando l'informazione extra degli annunci di affondamento), oltre al vincolo banale di almeno 17 colpi. La cifra di 53 va letta come "quanto vale un tiro poco informato", non come soglia.

2.9.8 Che cosa portarsi dietro

- Le tre strategie sono tre livelli di uso dell'informazione: nessuno (casuale), locale (hunt and target), globale (bayesiana).
- La ricerca bayesiana non ha regole scritte a mano: massimizza l'utilità attesa rispetto a una distribuzione aggiornata. Il comportamento "intelligente" (cominciare dal centro, seguire l'orientamento della nave dopo due colpi) **emerge** dal calcolo.
- Ogni informazione disponibile va usata: tenere conto delle navi affondate vale da solo una dozzina di turni.
- È lo stesso schema delle applicazioni reali di *search theory*, come la ricerca del relitto del volo AF 447.

Riepilogo della lezione

- Le preferenze sono una relazione $\succsim$ su X; se è completa e transitiva è una *relazione di preferenza*, e da essa si derivano $\succ$ e $\sim$.
- **Teorema 1**: con X finito, $\succsim$ è rappresentabile da $u : X \to \mathbb{R}$ se e solo se è completa e transitiva. L'utilità è *ordinale*: invariante per trasformazioni strettamente crescenti, quindi i numeri non si interpretano.
- Con l'incertezza si passa alle *lotterie*. Una lotteria composta si riduce a una semplice moltiplicando le probabilità lungo i rami.
- **Teorema VNM**: $\succsim$ su $\mathcal{L}$ ammette utilità attesa $U(L) = \sum_i p_i u(x_i)$ se e solo se vale ordine di preferenza, continuità e indipendenza. L'utilità è *cardinale*: unica a meno di trasformazioni affini $aU + b$ con $a > 0$.
- Il *paradosso di Allais* mostra che le persone violano l'indipendenza: VNM è una teoria normativa, non descrittiva.
- Atteggiamento verso il rischio = curvatura di u: lineare $\to$ neutrale, concava $\to$ avverso, convessa $\to$ amante. Con $u(x) = \sqrt{x}$ la condivisione del rischio e l'assicurazione creano valore per entrambe le parti (premio $P \in [50, 75]$ nell'esempio).
- **Lab**: la ricerca bayesiana sceglie a ogni turno la cella di massima utilità attesa, aggiornando il posteriore con colpi a vuoto, a segno e affondamenti. Media misurata: 41–44 colpi, contro 61 di hunt and target e 95 del caso.

Prossima lezione: giochi non cooperativi in forma normale, *best response* ed equilibrio di Nash.

LEZIONE 3

Giochi non cooperativi

TEORIA

Nella lezione 1 i giochi erano combinatori: due giocatori, a turni, a somma zero, con informazione perfetta. Qui si toglie quasi tutto. I giocatori possono essere n, muovono *simultaneamente*, ognuno ha la propria utilità (non necessariamente opposta a quella degli altri) e le utilità sono quelle di VNM della lezione 2.

Il percorso della lezione:

1. giochi in forma normale;
2. strategie dominanti;
3. strategie pure e *best response*;
4. equilibrio di Nash in strategie pure;
5. strategie miste e *best response*;
6. equilibrio di Nash in strategie miste;
7. equilibri misti nei giochi 2×2;
8. giochi a somma zero e minimax.

3.1 Giochi strategici e forma normale

Nei **giochi strategici**:

- i giocatori sono in **competizione** tra loro;
- ognuno è **egoista** (*self-interested*): vuole massimizzare il proprio payoff;
- ognuno deve decidere quale azione intraprendere.

Ci sono due modi di rappresentarli:

- **forma normale** (o *forma matriciale*): elenca il payoff di ciascun giocatore in funzione delle azioni di tutti, *come se* si muovesse simultaneamente;
- **forma estesa**: include l'ordine delle mosse e ciò che ogni giocatore sa quando muove. Si rappresenta come un albero (lo si è visto nella lezione 1). Negli scacchi il bianco muove e il nero risponde; nel poker conta anche che cosa un giocatore *vede* quando punta.

Definizione 3.1 (Gioco in forma normale). Un gioco in forma normale è una tupla $\langle N, A, u \rangle$ dove:

- $N = \{1, 2, \ldots, n\}$ è l'insieme dei **giocatori**;
- $A = \{A_1, A_2, \ldots, A_n\}$ raccoglie le **azioni** di tutti: A_i è l'insieme delle azioni del giocatore i,

e $a = (a_1, \ldots, a_n) \in A_1 \times \cdots \times A_n$ è un **profilo di azioni**;

- $u_i : A_1 \times \cdots \times A_n \to \mathbb{R}$ è la **funzione di utilità** (payoff) del giocatore i, e $u = (u_1, \ldots, u_n)$ è il profilo delle utilità.

3.1.1 Giochi a due giocatori come matrici

Con due giocatori si usa una tabella: il giocatore 1 sceglie la **riga**, il giocatore 2 la **colonna**; in ogni cella il primo numero è il payoff della riga, il secondo quello della colonna.

	A	B
C	2, 1	3, 4
D	1, 3	4, 4

$$A = \begin{pmatrix} 2 & 3 \\ 1 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 4 \\ 3 & 4 \end{pmatrix}$$

I giocatori sono 1 (riga) e 2 (colonna), gli insiemi di azioni sono $\{C, D\}$ e $\{A, B\}$, e per esempio $u_1(C, A) = 2$, $u_2(C, A) = 1$. Equivalentemente si usano due matrici: A con i payoff della riga e B con quelli della colonna. È la rappresentazione che si usa nel codice.

3.2 Strategie dominanti

Una **strategia pura** per il giocatore i è la scelta di un'azione $a \in A_i$. Un giocatore razionale non gioca una strategia che non gli dà il risultato migliore *qualunque cosa facciano gli altri*.

Definizione 3.2 (Strategia strettamente e debolmente dominata). Siano s_i' e s_i'' due strategie del giocatore i. La strategia s_i' è **strettamente dominata** da s_i'' se, per *ogni* combinazione delle strategie degli altri giocatori, il payoff di i con s_i' è strettamente minore che con s_i'':

$$u_i(s_1, \ldots, s_{i-1}, s_i', s_{i+1}, \ldots, s_n) < u_i(s_1, \ldots, s_{i-1}, s_i'', s_{i+1}, \ldots, s_n).$$

La strategia s_i' è **debolmente dominata** da s_i'' se vale la stessa cosa con $\leq$ al posto di $<$, *e almeno una disuguaglianza è stretta*.

Attenzione (la dominanza debole nelle slide). Le slide definiscono la dominanza debole con il solo $\leq$ ovunque. Così ogni strategia dominerebbe debolmente sé stessa, e due strategie con payoff identici si dominerebbero a vicenda. La definizione standard richiede $\leq$ per ogni profilo degli avversari e $<$ per almeno uno.

3.2.1 Eliminazione iterata delle strategie strettamente dominate

Una strategia strettamente dominata non verrà mai giocata da un giocatore razionale, quindi la si può *eliminare* dal gioco. Nel gioco ridotto possono comparire nuove strategie dominate: si ripete. Il procedimento si chiama **IESDS** (*Iterated Elimination of Strictly Dominated Strategies*).

Esempio (il dilemma del prigioniero).

	C	D
C	−1, −1	−4, 0
D	0, −4	−3, −3

Per il giocatore 1, *Cooperate* è strettamente dominata da *Defect* ($-1 < 0$ e $-4 < -3$): la si elimina. Lo stesso per il giocatore 2. Sopravvive solo (D, D): *Defect* è una **strategia strettamente dominante**. L'esito è razionale ma non socialmente ottimo: (C, C) farebbe stare meglio entrambi.
Un gioco che si risolve completamente con IESDS si dice **risolvibile per dominanza** (*dominance solvable*).

Osservazione (numeri diversi dalla lezione 1). Nella lezione 1 il payoff di chi coopera da solo era −5, qui è −4. Non cambia nulla: quello che conta nel dilemma del prigioniero è l'*ordine* dei payoff, $a > b > c > d$ con a = tradire da soli, b = cooperare entrambi, c = tradire entrambi, d = cooperare da soli (qui $0 > -1 > -3 > -4$).

Esempio (un gioco senza strategie dominate).

	T	V	C
L	0, 4	4, 0	5, 3
M	4, 0	0, 4	5, 3
R	3, 5	3, 5	8, 8

Nessuna strategia si può eliminare. Per il giocatore 1, L è la migliore se l'altro gioca V, M se gioca T, R se gioca C; per il giocatore 2, T è la migliore contro L, V contro M, C contro R. Ogni strategia è la migliore in almeno un caso, quindi nessuna è dominata: **non tutti i giochi si risolvono con IESDS**.

3.3 Best response ed equilibrio di Nash in strategie pure

Si immagini che il giocatore i sappia che cosa giocheranno tutti gli altri:

$$a_{-i} = \langle a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_n \rangle.$$

Definizione 3.3 (Best response pura). La **best response pura** ad a_{-i} è l'azione che massimizza il payoff di i:

$$a_i^* \in \mathrm{BR}(a_{-i}) \iff \forall a_i \in A_i,\ u_i(a_i^*, a_{-i}) \geq u_i(a_i, a_{-i}).$$

$\mathrm{BR}(a_{-i})$ è un *insieme*: le best response possono essere più di una, in caso di pareggio.

Definizione 3.4 (Equilibrio di Nash puro). Un profilo di azioni è un **equilibrio di Nash puro** (PNE) se ogni giocatore sta giocando una best response:

$$a = \langle a_1, \ldots, a_n \rangle \text{ è un NE} \iff \forall i,\ a_i \in \mathrm{BR}(a_{-i}).$$

Esempio (trovare l'equilibrio puro). Nel gioco 3×3 precedente si segnano, per ogni colonna, il miglior payoff del giocatore 1 e, per ogni riga, il miglior payoff del giocatore 2:

	T	V	C
L	0, **4**	**4**, 0	5, 3
M	**4**, 0	0, **4**	5, 3
R	3, 5	3, 5	**8**, **8**

L'unica cella in cui *entrambi* i giocatori stanno rispondendo al meglio all'altro è (R, C): è l'unico equilibrio di Nash puro.

Osservazione (il metodo è generale). "Segnare le best response e cercare le celle con entrambi i segni" funziona per qualunque gioco a due giocatori in forma matriciale, ed è esattamente quello che fa il codice del laboratorio: per ogni cella controlla che la riga sia ottima nella sua colonna e la colonna ottima nella sua riga.

3.3.1 IESDS ed equilibrio di Nash

- Se IESDS elimina tutti i profili tranne uno, quel profilo è anche l'**unico** equilibrio di Nash puro.
- Il viceversa non vale: alcune strategie sopravvivono a IESDS senza far parte di un equilibrio di Nash (nel gioco 3×3 sopravvivono tutte, ma l'equilibrio è uno).

Approfondimento (perché le strategie eliminate non stanno in un equilibrio). Se s_i' è strettamente dominata da s_i'', allora contro *qualunque* a_{-i} vale $u_i(s_i'', a_{-i}) > u_i(s_i', a_{-i})$, quindi s_i' non è mai una best response e non può comparire in un equilibrio di Nash. Applicando l'argomento a ogni passo dell'eliminazione, nessun equilibrio di Nash viene mai eliminato. Se alla fine resta un solo profilo, e un equilibrio esiste, deve essere quello.

3.3.2 Come leggere l'equilibrio di Nash

Il concetto di equilibrio di Nash non dice *come* i giocatori lo raggiungano, ma la logica è questa:

- se i giocatori non sono in equilibrio, almeno uno ha l'opportunità di deviare, e lo farà;
- se sono in equilibrio, nessuno ha interesse a cambiare strategia;
- questa logica ragiona **un giocatore alla volta**: non considera deviazioni congiunte di più giocatori.

Per questo gli equilibri di Nash si dicono **stabili** e **auto-rinforzanti**.

Esempio (la battaglia dei sessi: due equilibri).

	A	R
A	2, 1	0, 0
R	0, 0	1, 2

Ci sono due equilibri puri, (A, A) e (R, R), e ciascuno è sbilanciato a favore di uno dei due giocatori.

Un'opzione migliore per entrambi sarebbe *accordarsi* e giocare (A, A) e (R, R) con probabilità 0,5 ciascuno: payoff atteso 1,5 a testa.

Attenzione (l'accordo non è una strategia mista). Giocare "(A, A) metà delle volte e (R, R) l'altra metà" richiede che i due giocatori *si coordinino*, per esempio lanciando insieme una moneta. Se ciascuno randomizzasse per conto suo al 50%, metà delle volte si ritroverebbero in (A, R) o (R, A) con payoff nullo. Un accordo di questo tipo non è un equilibrio di Nash ma un **equilibrio correlato**: è l'argomento della lezione 4.

Esempio (i calci di rigore: nessun equilibrio puro).

	L	R
L	$1, -1$	$-1, 1$
R	$-1, 1$	$1, -1$

Il portiere (riga) vince se indovina il lato, il tiratore (colonna) se lo sbaglia. Da qualunque cella uno dei due ha interesse a cambiare: non esiste un equilibrio puro. Si "risolve" solo randomizzando tra L e R con probabilità 0,5.

Attenzione ("il miglior esito per entrambi"). Le slide dicono che randomizzare al 50% dà "il miglior esito per entrambi". In un gioco a somma zero questa frase non ha senso: ciò che uno guadagna l'altro lo perde, quindi non esiste un esito migliore per entrambi. Il senso corretto è che 50/50 è l'unico profilo *stabile*: ciascuno si garantisce payoff atteso 0 e nessuno dei due può fare meglio cambiando da solo. È la strategia minimax, che si ritrova alla fine della lezione.

3.3.3 Equilibri con utilità analitiche: la tragedia dei beni comuni

Le azioni non devono per forza essere un insieme finito. Due agricoltori pompano acqua dalla stessa falda:

- $x_1, x_2 \geq 0$ sono i livelli di estrazione e $X = x_1 + x_2$ il totale;
- il ricavo per unità d'acqua è $A - BX$ con $A, B > 0$: vale A quando non si è estratto nulla e cala di B per ogni unità estratta in totale;
- l'utilità dell'agricoltore i è $u(x_i, x_j) = x_i\big(A - B(x_i + x_j)\big)$.

Best response. L'agricoltore i sceglie x_i prendendo x_j come dato. Condizione del primo ordine:

$$\frac{\partial u}{\partial x_i} = \frac{\partial}{\partial x_i}\big(Ax_i - Bx_i^2 - Bx_ix_j\big) = A - 2Bx_i - Bx_j = 0 \quad \Longrightarrow \quad \mathrm{BR}_i(x_j) = x_i = \frac{A - Bx_j}{2B}.$$

Equilibrio di Nash. Il gioco è simmetrico, quindi si cerca $x_i^* = x_j^* = x^*$:

$$x^* = \frac{A - Bx^*}{2B} \;\Rightarrow\; 2Bx^* = A - Bx^* \;\Rightarrow\; x^* = \frac{A}{3B}, \qquad X^{\mathrm{Nash}} = 2x^* = \frac{2A}{3B}.$$

Ottimo sociale. Un pianificatore massimizza il benessere totale $W(X) = X(A - BX)$:

$$\frac{dW}{dX} = A - 2BX = 0 \;\Rightarrow\; X^{\text{Opt}} = \frac{A}{2B}, \qquad x^{\text{plan}} = \frac{A}{4B} \text{ a testa.}$$

	Estrazione totale	Benessere totale	Utilità a testa
Equilibrio di Nash	$\frac{2A}{3B}$	$\frac{2A^2}{9B} \approx 0{,}222\,\frac{A^2}{B}$	$\frac{A^2}{9B}$
Ottimo sociale	$\frac{A}{2B}$	$\frac{A^2}{4B} = 0{,}25\,\frac{A^2}{B}$	$\frac{A^2}{8B}$

In competizione gli agricoltori estraggono *più* acqua del necessario e ottengono *meno* utilità ciascuno. È la **tragedia dei beni comuni**: i giocatori sono incentivati a sfruttare una risorsa comune a danno di tutti. Il meccanismo è lo stesso del dilemma del prigioniero, con un continuo di azioni al posto di due.

Approfondimento (i passaggi che le slide saltano).

- La condizione del primo ordine individua un *massimo* perché $\partial^2 u/\partial x_i^2 = -2B < 0$: l'utilità è concava in x_i.
- La best response vale solo finché $A - Bx_j \geq 0$; se l'altro estrae più di A/B la risposta ottima è $x_i = 0$. All'equilibrio il vincolo non è attivo.
- L'utilità a testa all'equilibrio è $x^*(A - BX^{\text{Nash}}) = \frac{A}{3B} \cdot \frac{A}{3} = \frac{A^2}{9B}$; con il piano è $\frac{A}{4B} \cdot \frac{A}{2} = \frac{A^2}{8B}$, più alta.
- Il piano non è un equilibrio: se l'altro estrae $A/4B$, la best response è $\text{BR}(A/4B) = \frac{A - A/4}{2B} = \frac{3A}{8B} > \frac{A}{4B}$. Ognuno ha interesse a estrarre di più.

Attenzione ("ottimo di Pareto" nel titolo delle slide). Le slide confrontano l'equilibrio con l'"ottimo di Pareto", ma quello che calcolano è l'ottimo *utilitaristico*, cioè il massimo della somma delle utilità. L'ottimo utilitaristico è Pareto-ottimo, ma non è l'unico esito Pareto-ottimo: per esempio un agricoltore che estrae $A/2B$ da solo, con l'altro a zero, è anch'esso Pareto-ottimo.

3.4 Strategie miste

Definizione 3.5 (Strategia mista). Una **strategia** σ_i del giocatore i è una distribuzione di probabilità su A_i.

- È **pura** se una sola azione ha probabilità 1.
- È **mista** se più azioni hanno probabilità positiva. Le azioni con probabilità positiva formano il **supporto** della strategia, $\text{supp}(\sigma_i)$.

S_i è l'insieme delle strategie di i, $S = S_1 \times \cdots \times S_n$ quello dei profili, e $\sigma = (\sigma_1, \ldots, \sigma_n) \in S$ è un profilo di strategie miste.

Con le strategie miste le utilità non si leggono più nella tabella: si usa l'**utilità attesa** della lezione 2. La probabilità di un profilo di azioni è il prodotto delle probabilità con cui ogni giocatore sceglie la propria

azione (i giocatori randomizzano in modo indipendente):

$$u_i(\sigma) = \sum_{a \in A} u_i(a)\ \Pr(a \mid \sigma), \qquad \Pr(a \mid \sigma) = \prod_{j \in N} \sigma_j(a_j).$$

In forma matriciale, con σ_r e σ_c vettori riga: $u_r = \sigma_r A \sigma_c^T$ e $u_c = \sigma_r B \sigma_c^T$.

Esempio (matching pennies).

	H	T
H	$1, -1$	$-1, 1$
T	$-1, 1$	$1, -1$

Con $\sigma_r = \sigma_c = (\frac{1}{2}, \frac{1}{2})$ ogni cella ha probabilità 0,25:

$$u_r = 1 \cdot 0{,}25 - 1 \cdot 0{,}25 - 1 \cdot 0{,}25 + 1 \cdot 0{,}25 = 0.$$

Con $\sigma_r = \sigma_c = (\frac{4}{5}, \frac{1}{5})$:

$$u_r = 1 \cdot (0{,}8 \cdot 0{,}8) - 1 \cdot (0{,}8 \cdot 0{,}2) - 1 \cdot (0{,}2 \cdot 0{,}8) + 1 \cdot (0{,}2 \cdot 0{,}2) = 0{,}64 - 0{,}16 - 0{,}16 + 0{,}04 = 0{,}36.$$

Attenzione (perché servono le utilità di VNM). Calcolare il valore atteso dei payoff ha senso solo se i payoff sono utilità *cardinali*, cioè di VNM (lezione 2). Con utilità solo ordinali un equilibrio misto non è nemmeno definito: trasformando i numeri con una funzione crescente non lineare, l'equilibrio misto cambierebbe, mentre gli equilibri puri resterebbero gli stessi.

3.5 Best response ed equilibrio di Nash misti

Definizione 3.6 (Best response mista ed equilibrio di Nash misto). La **best response mista** a σ_{-i} è la strategia che massimizza il payoff atteso di i:

$$\sigma_i^* \in \mathrm{BR}(\sigma_{-i}) \iff \forall \sigma_i \in S_i,\ u_i(\sigma_i^*, \sigma_{-i}) \geq u_i(\sigma_i, \sigma_{-i}).$$

Un profilo $\sigma = \langle \sigma_1, \ldots, \sigma_n \rangle$ è un **equilibrio di Nash misto** (MNE) se $\forall i,\ \sigma_i \in \mathrm{BR}(\sigma_{-i})$.

Attenzione (un refuso nella definizione). Le slide scrivono "$\forall \sigma_i \in A_i$". Le strategie miste stanno in S_i (l'insieme delle distribuzioni su A_i), non in A_i.

Teorema 3.7 (Nash, 1950). Ogni gioco finito (numero finito di giocatori e di azioni) ha almeno un equilibrio di Nash misto.

La dimostrazione, tramite il teorema del punto fisso di Brouwer, è nella lezione 4.

3.5.1 Condizione generale per una best response

Teorema 3.8 (Condizione del supporto). In un gioco a due giocatori $(A, B) \in \mathbb{R}^{m\times n}$, la strategia σ_r della riga è una best response a σ_c se e solo se

$$\sigma_r(i) > 0 \implies (A\sigma_c^T)_i = u = \max_k (A\sigma_c^T)_k \qquad \forall i \in A_1.$$

In parole: **una strategia mista è una best response se e solo se mette probabilità solo su azioni pure che sono a loro volta best response**.

Dimostrazione. $(A\sigma_c^T)_i$ è l'utilità che la riga ottiene giocando la sua i-esima azione. Sia $u = \max_k (A\sigma_c^T)_k$. L'utilità di σ_r è

$$\begin{aligned}\sigma_r A\sigma_c^T &= \sum_{i=1}^{m} \sigma_r(i)\,(A\sigma_c^T)_i = \sum_{i=1}^{m} \sigma_r(i)\big[u - u + (A\sigma_c^T)_i\big] \\ &= \sum_{i=1}^{m} \sigma_r(i)\,u - \sum_{i=1}^{m} \sigma_r(i)\big(u - (A\sigma_c^T)_i\big) = u - \sum_{i=1}^{m} \sigma_r(i)\big(u - (A\sigma_c^T)_i\big).\end{aligned}$$

Ogni termine $u - (A\sigma_c^T)_i$ è ≥ 0, quindi l'utilità massima possibile è u, e si raggiunge se e solo se la somma è nulla, cioè se ogni azione con $\sigma_r(i) > 0$ ha $(A\sigma_c^T)_i = u$. □

Osservazione (due conseguenze pratiche).

- Per verificare se una strategia è una best response non serve confrontarla con tutte le infinite strategie miste: basta calcolare il vettore $A\sigma_c^T$ (le utilità delle azioni pure) e guardare il supporto. È l'algoritmo del laboratorio.
- All'equilibrio misto ogni giocatore è *indifferente* tra le azioni del proprio supporto. È questa indifferenza che permette di calcolare gli equilibri nei giochi 2×2.

Esempio (best response in sasso-carta-forbici). La colonna non gioca mai forbici e gioca sasso o carta con probabilità 0,5:

$$A = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}, \qquad \sigma_c = \left(\tfrac{1}{2}, \tfrac{1}{2}, 0\right), \qquad A\sigma_c^T = \begin{pmatrix} -0{,}5 \\ 0{,}5 \\ 0 \end{pmatrix} \implies \sigma_r^* = (0, 1, 0).$$

La seconda riga (carta) ha l'utilità massima: l'unica best response è giocare sempre carta.

Esempio (best response nella battaglia dei sessi). Con $A = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$, $B = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$ e $\sigma_c = (\frac{1}{3}, \frac{2}{3})$:

$$A\sigma_c^T = \left(\tfrac{2}{3}, \tfrac{2}{3}\right)^T.$$

Le due azioni della riga danno la stessa utilità, quindi *qualunque* strategia della riga è una best response: $(\frac{2}{3}, \frac{1}{3})$, $(\frac{1}{2}, \frac{1}{2})$, $(1, 0)$, ...

- Con $\sigma_r = (\frac{2}{3}, \frac{1}{3})$: $\sigma_r B = (\frac{2}{3}, \frac{2}{3})$, quindi anche σ_c è una best response a σ_r. Le due strategie sono una best response l'una all'altra: (σ_r, σ_c) è un equilibrio di Nash.

- Con $\sigma_r = (\frac{1}{2}, \frac{1}{2})$: $\sigma_r B = (\frac{1}{2}, 1)$. La colonna dovrebbe giocare solo la seconda azione, ma σ_c mette probabilità $\frac{1}{3}$ sulla prima: $\sigma_c \notin \mathrm{BR}(\sigma_r)$ e il profilo non è un equilibrio, anche se $\sigma_r \in \mathrm{BR}(\sigma_c)$.

3.6 Equilibri misti nei giochi 2×2

Nella battaglia dei sessi

	A	C
A	2, 1	0, 0
C	0, 0	1, 2

si cerca un equilibrio in cui entrambi randomizzano. Per la condizione del supporto, ognuno dev'essere indifferente tra le proprie due azioni.

La colonna rende indifferente la riga. La colonna gioca A con probabilità p. La riga è indifferente se

$$u_r(A) = u_r(C) \iff 2p + 0(1-p) = 0 \cdot p + 1(1-p) \implies p = \tfrac{1}{3}.$$

La riga rende indifferente la colonna. La riga gioca A con probabilità q. La colonna è indifferente se

$$u_c(A) = u_c(C) \iff q \cdot 1 + 0(1-q) = 0 \cdot q + 2(1-q) \implies q = \tfrac{2}{3}.$$

L'equilibrio misto è quindi $\sigma_r = (\frac{2}{3}, \frac{1}{3})$, $\sigma_c = (\frac{1}{3}, \frac{2}{3})$.

Osservazione (il trucco controintuitivo). La probabilità con cui gioca un giocatore è determinata dai payoff dell'*altro*. La riga sceglie q non per massimizzare il proprio payoff (all'equilibrio le sue due azioni le rendono uguale) ma per rendere l'avversario indifferente: se non lo fosse, l'avversario smetterebbe di randomizzare e l'equilibrio crollerebbe.

Approfondimento (quanto vale l'equilibrio misto). All'equilibrio misto della battaglia dei sessi ciascuno ottiene payoff atteso $\frac{2}{3}$: meno di 1, il peggior payoff dei due equilibri puri. I giocatori si mancano con probabilità $\frac{2}{3} \cdot \frac{2}{3} + \frac{1}{3} \cdot \frac{1}{3} = \frac{5}{9}$. L'equilibrio misto è "equo" (simmetrico nei payoff) ma inefficiente: è il motivo per cui si cercano alternative come gli equilibri correlati (lezione 4).

3.7 Giochi a somma zero e minimax

Definizione 3.9 (Gioco a somma zero). Un gioco a due giocatori (A, B) è **a somma zero** se e solo se $A = -B$. Basta quindi una sola matrice A: la riga massimizza $\sigma_r A \sigma_c^T$, la colonna lo minimizza.

Il punto di vista della colonna. Data $A \in \mathbb{R}^{m\times n}$ e la strategia σ_c della colonna, la best response della riga ottiene

$$\max_{\sigma_r \in S_r} \sigma_r A \sigma_c^T = \max_i (A\sigma_c^T)_i = \min\{v \in \mathbb{R} \mid A\sigma_c^T \le \mathbf{1}v\}.$$

La prima uguaglianza è la condizione del supporto (il massimo si raggiunge su un'azione pura); la seconda dice che il massimo di un vettore è il più piccolo v che lo maggiora componente per componente. Con la sua scelta di σ_c la colonna *fissa* questo tetto v al guadagno della riga; essendo il gioco a somma zero, vuole renderlo il più basso possibile.

Definizione 3.10 (I due programmi lineari del minimax).

$$\text{Colonna:} \quad \min_{v,\,\sigma_c} v \quad \text{s.t. } A\sigma_c^T \le \mathbf{1}v, \;\; \sigma_c \in S_c$$

$$\text{Riga:} \quad \max_{u,\,\sigma_r} u \quad \text{s.t. } \sigma_r A \ge u\mathbf{1}^T, \;\; \sigma_r \in S_r$$

La riga cerca la strategia che massimizza il payoff *garantito* u contro qualunque colonna (**maximin**); la colonna quella che minimizza il payoff massimo concesso v (**minimax**).

Teorema 3.11 (Minimax, von Neumann 1928). In un gioco a somma zero $A \in \mathbb{R}^{m\times n}$ con insiemi di strategie miste S_r, S_c esistono $\sigma_r \in S_r$, $\sigma_c \in S_c$ e $v \in \mathbb{R}$ tali che

$$A\sigma_c^T \le \mathbf{1}v, \qquad \sigma_r A \ge v\mathbf{1}^T.$$

Allora v è l'unico **valore del gioco** e σ_r, σ_c sono strategie ottime dei giocatori.

In altre parole max min = min max: il payoff che la riga si può garantire coincide con quello che la colonna può impedirle di superare.

Dimostrazione (traccia delle slide).

1. *Debolezza* ($u \le v$). Per soluzioni ammissibili qualsiasi dei due programmi,

$$u = u\,\mathbf{1}^T\sigma_c^T \le \sigma_r A \sigma_c^T \le \sigma_r \mathbf{1} v = v.$$

   Quindi gli obiettivi sono limitati e si possono restringere agli insiemi compatti $S_c \times [u, v]$ e $S_r \times [u, v]$; per continuità di $\sigma_r A \sigma_c^T$ gli ottimi u e v esistono.

2. *Uguaglianza* ($u = v$), per induzione su $m + n$. Con $m + n = 2$ (matrice 1×1) è ovvio. Siano σ_c, v e σ_r, u ottimi. Se $A\sigma_c^T = \mathbf{1}v$ e $\sigma_r A = u\mathbf{1}^T$, allora $u = \sigma_r A \sigma_c^T = v$. Altrimenti una disuguaglianza è stretta, per esempio la riga k: $(A\sigma_c^T)_k < v$.

3. Si cancella la riga k ottenendo $\bar{A}$ ($(m-1)\times n$), con ottimi $\bar{u}, \bar{v}$. Per induzione $\bar{u} = \bar{v}$. Togliere una riga può solo peggiorare la riga e aiutare la colonna: $\bar{u} \le u$ e $\bar{v} \le v$.

4. Se fosse $\bar{v} < v$, la strategia $\sigma_c' = (1-\epsilon)\sigma_c + \epsilon\bar{\sigma}_c$ darebbe $(A\sigma_c'^T)_i < v$ su tutte le righe diverse da k (per ogni $\epsilon \in (0, 1]$) e anche sulla riga k (per ϵ piccolo, perché lì la disuguaglianza era già stretta): contraddice la minimalità di v. Quindi $\bar{v} = v$ e

$$\bar{u} \le u \le v = \bar{v} = \bar{u} \implies u = v. \qquad \square$$

Attenzione (due dettagli della dimostrazione).

- Al passo 4 le slide scelgono ϵ solo in modo che $v(1-\epsilon) + \bar{v}\epsilon < v$; per la riga cancellata k

serve anche ϵ *abbastanza piccolo*, perché $(A\bar{\sigma}_c^T)_k$ può essere grande. Esiste sempre, dato che $(A\sigma_c^T)_k < v$ strettamente.

- Le slide trattano solo il caso in cui la disuguaglianza stretta è in $A\sigma_c^T \le \mathbf{1}v$. Se tutte quelle sono uguaglianze ma una di $\sigma_r A \ge u\mathbf{1}^T$ è stretta, si ragiona allo stesso modo cancellando una *colonna*.

Approfondimento (il legame con la lezione 1). Minimax per i giochi combinatori (lezione 1) è il caso particolare in cui l'informazione è perfetta e i giocatori muovono a turno: lì il max e il min si scambiano senza bisogno di strategie miste, e il valore si calcola risalendo l'albero. Nei giochi simultanei come sasso-carta-forbici, invece, senza randomizzare il max-min e il min-max *non* coincidono (in pure: -1 contro $+1$); le strategie miste sono proprio ciò che serve a farli coincidere.

3.7.1 Esempio: sasso-carta-forbici come programma lineare

Con $A = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$ e $x = \sigma_r$, il programma della riga $\max u$ s.t. $xA \ge \mathbf{1}u$, $x \in S_r$ diventa:

$$\begin{aligned} &\max_{u,x} u \\ &\text{s.t.} \quad 0x_1 + 1x_2 - 1x_3 \ge u \\ &\qquad -1x_1 + 0x_2 + 1x_3 \ge u \\ &\qquad 1x_1 - 1x_2 + 0x_3 \ge u \\ &\qquad x_1 + x_2 + x_3 = 1 \end{aligned}$$

Ogni vincolo dice "contro la colonna j la riga ottiene almeno u".

Forma standard. I risolutori (come `linprog` di SciPy) vogliono $\min c\,x$ s.t. $M_{ub}\,x \le b_{ub}$, $M_{eq}\,x = b_{eq}$. Si rinomina $u = x_4$, si porta tutto a sinistra e si cambia segno:

$$\begin{aligned} &\min_x \; -x_4 \\ &\text{s.t.} \quad 0x_1 - 1x_2 + 1x_3 + 1x_4 \le 0 \\ &\qquad 1x_1 + 0x_2 - 1x_3 + 1x_4 \le 0 \\ &\qquad -1x_1 + 1x_2 + 0x_3 + 1x_4 \le 0 \\ &\qquad 1x_1 + 1x_2 + 1x_3 + 0x_4 = 1 \end{aligned}$$

$$M_{ub} = \begin{pmatrix} 0 & -1 & 1 & 1 \\ 1 & 0 & -1 & 1 \\ -1 & 1 & 0 & 1 \end{pmatrix},\; b_{ub} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix},$$
$$M_{eq} = (1,1,1,0),\; b_{eq} = 1,\; c = (0,0,0,-1).$$

Per una matrice qualsiasi $A \in \mathbb{R}^{m\times n}$:

$$M_{ub} = \left[\, -A^T \mid \mathbf{1} \,\right] \in \mathbb{R}^{n\times(m+1)}, \quad b_{ub} = \mathbf{0}_n, \quad c = (\underbrace{0,\dots,0}_{m},-1), \quad M_{eq} = (\underbrace{1,\dots,1}_{m},0), \quad b_{eq} = 1.$$

Attenzione (il valore del gioco può essere negativo). Le slide scrivono la forma standard con il vincolo $x \ge 0$ su *tutte* le variabili. Ma $x_4 = u$ è il valore del gioco, che può essere negativo. Il notebook lo dice correttamente ($x_i \ge 0$ solo per $i \le m$), ed è un dettaglio che conta: `linprog` impone di default $x_i \ge 0$ a *tutte* le variabili. Se non si dichiara u libera (`bounds=[(0, None)]*m +`

`[(None, None)]`), su un gioco di valore negativo il programma diventa **impossibile** e il risolutore non restituisce nessuna soluzione. Nel laboratorio lo si verifica su un esempio; tutti i test del notebook hanno valore ≥ 0 e non se ne accorgono.

3.8 Laboratorio: Best response, equilibri e giochi potenziali

LAB

[NonCooperativeGameTheory_part1_blank.ipynb, Potential_Games_Label_Propagation_blank.ipynb]

Obiettivo: implementare gli strumenti della lezione per giochi a due giocatori (verifica di best response ed equilibri, minimax per i giochi a somma zero, equilibri misti 2×2) e poi un'applicazione dei **giochi potenziali**: trovare le comunità di un grafo facendo giocare i nodi. Tutto il codice sotto è stato eseguito: i risultati attesi scritti nel notebook sono riprodotti esattamente.

> **Osservazione (il notebook della lezione 4).** Il notebook del laboratorio 4 riprende integralmente questa prima parte e la estende. Le celle di quel notebook contengono già gli output della soluzione del docente, ed è con quelli che sono stati confrontati i risultati di questa sezione.

3.8.1 Best response ed equilibrio di Nash

Le strategie sono vettori di probabilità (`[0, 1]` = seconda azione pura). La stessa funzione serve per strategie pure e miste, grazie alla condizione del supporto: si calcolano le utilità delle azioni pure contro l'avversario e si controlla che il supporto contenga solo azioni di utilità massima.

```python
import numpy as np

def is_best_response(P1, P2, s1, s2):
    s1, s2 = np.asarray(s1, float), np.asarray(s2, float)
    for s in (s1, s2):
        assert np.all(s >= 0) and np.isclose(s.sum(), 1), "strategia non valida"
    u_row = P1 @ s2               # utilita' di ogni riga contro s2
    u_col = s1 @ P2               # utilita' di ogni colonna contro s1
    br_row = np.all(u_row[s1 > 0] >= u_row.max() - 1e-9)
    br_col = np.all(u_col[s2 > 0] >= u_col.max() - 1e-9)
    return bool(br_row), bool(br_col)

def is_nash_equilibrium(P1, P2, s1, s2):
    return all(is_best_response(P1, P2, s1, s2))
```

Gioco	Profilo	Atteso	Ottenuto
Dilemma del prigioniero	(D, C)	(True, False)	(True, False)
Dilemma del prigioniero	(D, D)	NE	True
Gioco $C/D \times A/B$	(C, A)	(True, False)	(True, False)
Gioco $C/D \times A/B$	(D, B)	NE	True
Battaglia dei sessi	$(\frac{2}{3}, \frac{1}{3}), (\frac{1}{3}, \frac{2}{3})$	(True, True)	(True, True)
Battaglia dei sessi	$(\frac{2}{3}, \frac{1}{3}), (\frac{1}{2}, \frac{1}{2})$	(False, True)	(False, True)
Battaglia dei sessi	(A, A)	NE	True
Battaglia dei sessi	(0,8; 0,8)	errore	AssertionError

Attenzione (confronti esatti con le probabilità). Le strategie del tipo $(\frac{2}{3}, \frac{1}{3})$ non sono rappresentabili esattamente in floating point, e le utilità che devono essere "uguali" possono differire di 10^{-16}. Un test come `u_row[i] == u_row.max()` può allora dare risultati casuali. Per questo il codice confronta con una tolleranza (`- 1e-9`) e verifica la somma con `np.isclose`. Lo stesso vale per qualunque verifica di equilibrio nel progetto.

3.8.2 Minimax nei giochi a somma zero

```python
import scipy.optimize

def minimax_LP(A):
    A = np.asarray(A, float)
    m, n = A.shape
    c = np.zeros(m + 1); c[-1] = -1                  # max u = min -u
    M_ub = np.hstack([-A.T, np.ones((n, 1))])        # u - (xA)_j <= 0
    b_ub = np.zeros(n)
    M_eq = np.hstack([np.ones((1, m)), np.zeros((1, 1))])
    b_eq = [1]
    bounds = [(0, None)] * m + [(None, None)]        # u libera!
    res = scipy.optimize.linprog(c, A_ub=M_ub, b_ub=b_ub,
                                 A_eq=M_eq, b_eq=b_eq, bounds=bounds)
    return res.x[:m]
```

Gioco	Strategia minimax della riga	Valore
Sasso-carta-forbici	$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$	0
RPS modificato (4×3)	(0; 0,125; 0,5; 0,375)	0,125
RPS modificato, A^T (3×4)	(0,25; 0,4167; 0,3333)	0,0833
Matching pennies	$(\frac{1}{2}, \frac{1}{2})$	0
Casuale 5×5, seme 42	(0; 0,5629; 0,4371; 0; 0)	50,28

Attenzione (il commento del notebook sull'RPS modificato). Nel notebook, sopra l'RPS modificato, c'è il commento "It should print [1/3, 1/3, 1/3]". È un residuo copiato dalla cella precedente: la matrice è 4×3 e la soluzione corretta (confermata dall'output del docente nel notebook della lezione 4) è quella in tabella.

Il bug dei limiti, verificato. Il gioco $A = \begin{pmatrix} -2 & -1 \\ -1 & -3 \end{pmatrix}$ ha valore $-\frac{5}{3}$ e strategia ottima $(\frac{2}{3}, \frac{1}{3})$. Con `u` libera, `linprog` restituisce esattamente questo. Con i limiti di default ($u \geq 0$) restituisce `status = 2` (programma impossibile) e `res.x = None`: il codice successivo si rompe con un `TypeError` che non dice nulla sulla causa.

3.8.3 Equilibri misti nei giochi 2×2

Dalla condizione di indifferenza della riga $qA_{11} + (1-q)A_{12} = qA_{21} + (1-q)A_{22}$ si ricava la probabilità q con cui la *colonna* gioca la prima azione, e simmetricamente p per la riga:

$$q = \frac{A_{22} - A_{12}}{A_{11} - A_{12} - A_{21} + A_{22}}, \qquad p = \frac{B_{22} - B_{21}}{B_{11} - B_{12} - B_{21} + B_{22}}.$$

```python
def mixed_nash_equilibrium_2x2(A, B):
    A, B = np.asarray(A, float), np.asarray(B, float)
    den_q = A[0, 0] - A[0, 1] - A[1, 0] + A[1, 1]
    den_p = B[0, 0] - B[0, 1] - B[1, 0] + B[1, 1]
    if den_q == 0 or den_p == 0:
        raise ValueError("nessun equilibrio completamente misto")
    q = (A[1, 1] - A[0, 1]) / den_q      # rende indifferente la riga
    p = (B[1, 1] - B[1, 0]) / den_p      # rende indifferente la colonna
    if not (0 <= p <= 1 and 0 <= q <= 1):
        raise ValueError("niente equilibrio misto interno")
    return np.array([p, 1 - p]), np.array([q, 1 - q])
```

Gioco	Riga	Colonna
Battaglia dei sessi	$(\frac{2}{3}, \frac{1}{3})$	$(\frac{1}{3}, \frac{2}{3})$
Attraversamento ($A = [[-10, 5], [0, -1]]$)	(0,375; 0,625)	(0,375; 0,625)
Matching pennies	$(\frac{1}{2}, \frac{1}{2})$	$(\frac{1}{2}, \frac{1}{2})$
$A = [[4, 5], [6, 3]]$, $B = [[2, 1], [0, 3]]$	(0,75; 0,25)	(0,5; 0,5)
$A = [[3, 1], [0, 2]]$, $B = [[2, 1], [0, 3]]$	(0,75; 0,25)	(0,25; 0,75)
$A = [[2, 3], [3, -1]]$, $B = [[-5, -6], [-2, 0]]$	$(\frac{2}{3}, \frac{1}{3})$	(0,8; 0,2)

Tutti coincidono con il notebook, e `is_nash_equilibrium` li conferma.

Attenzione (la formula non funziona sempre). La formula presuppone che esista un equilibrio *completamente misto*. Non sempre c'è:

- nel **dilemma del prigioniero** il denominatore di q è $-1 + 4 - 0 - 3 = 0$: divisione per zero. Una strategia dominante non lascia spazio all'indifferenza;
- nel gioco $C/D \times A/B$ di inizio lezione esce $p = -0{,}5$: una "probabilità" negativa. L'unico equilibrio è quello puro (D, B).

Una versione da usare nel progetto deve controllare il denominatore e che $p, q \in [0, 1]$, come il codice sopra, e in quei casi cercare gli equilibri puri.

3.8.4 Giochi potenziali e rilevamento di comunità

[Lab03-Non_Cooperative_Game_Theory.pdf]

Definizione 3.12 (Gioco potenziale esatto). Un gioco $\langle N, S, u \rangle$ è un **gioco potenziale esatto** se esiste una **funzione potenziale** $\Phi : S \to \mathbb{R}$ tale che, per ogni giocatore i, ogni profilo s_{-i} degli altri

e ogni coppia di strategie $s_i, s_i' \in S_i$:

$$u_i(s_i', s_{-i}) - u_i(s_i, s_{-i}) = \Phi(s_i', s_{-i}) - \Phi(s_i, s_{-i}).$$

In parole: quando un giocatore cambia strategia da solo, la variazione del *suo* payoff è esattamente la variazione del potenziale *comune* Φ.

- I giochi potenziali (finiti) hanno sempre un **equilibrio di Nash puro**.
- Hanno la **proprietà del miglioramento finito** (FIP): ogni sequenza di deviazioni unilaterali che migliorano il payoff di chi devia è finita.

> **Approfondimento (perché le due proprietà valgono).** Ogni deviazione che migliora il payoff di chi devia aumenta Φ della stessa quantità. Siccome S è finito, Φ assume un numero finito di valori e non può crescere per sempre: ogni sequenza di miglioramenti termina (FIP). Dove termina, nessuno può migliorare da solo: è un equilibrio di Nash puro. In particolare il profilo che *massimizza* Φ è sempre un equilibrio.

Dinamica di best response (BRD). Dalla FIP segue un algoritmo che converge sempre a un equilibrio puro:

```
while not converged:
    for each player i ∈ N:
        if s_i ∉ BR(s_{-i}):
            s_i = BR(s_{-i})
```

Il gioco delle comunità. Dato un grafo $G = (V, E)$, si vogliono trovare gruppi di nodi densamente connessi al loro interno e poco connessi tra loro (**community detection**). Il problema è NP-difficile e ha molte euristiche (Louvain, Infomap, Walktrap, *label propagation*). Il laboratorio mostra che la label propagation è una dinamica di best response:

- ogni nodo è un giocatore: $N = V$;
- ogni nodo sceglie un cluster: $S_i = \{1, 2, \ldots, |V|\}$;
- l'utilità di un nodo è il numero di vicini nello stesso cluster: $u_i(s_i, s_{-i}) = \sum_{j \in \Gamma(i)} \mathbb{I}(s_i = s_j)$;
- il potenziale è $\Phi(s) = \frac{1}{2} \sum_{i \in V} \sum_{j \in \Gamma(i)} \mathbb{I}(s_i = s_j)$, cioè il numero di archi con estremi nello stesso cluster.

> **Approfondimento (verifica che Φ è un potenziale).** Ogni arco $\{i, j\}$ compare due volte nella doppia somma (da i e da j), da cui il $\frac{1}{2}$. Quando i cambia cluster cambiano solo gli archi incidenti a i, e ciascuno contribuisce a Φ con 1 se gli estremi sono nello stesso cluster: la variazione di Φ è la variazione del numero di vicini di i nel suo cluster, cioè esattamente Δu_i.

Non serve calcolare Φ: è solo la garanzia matematica che il ciclo termina. Basta assegnare a ogni nodo un cluster iniziale (il proprio identificativo) e poi, finché qualcosa cambia, spostare ogni nodo nel cluster più frequente tra i suoi vicini.

```
1 import random
2 from collections import Counter
3
```

```python
def best_response(node, G, action_profile):
    counts = Counter(action_profile[nb] for nb in G.neighbors(node))
    if not counts:
        return action_profile[node]
    best = max(counts.values())
    if counts.get(action_profile[node], 0) == best:
        return action_profile[node]          # gia' in un cluster ottimo: resta
    return random.choice([c for c, k in counts.items() if k == best])

def BRD(G):
    profile = {node: node for node in G.nodes()}   # ognuno nel proprio cluster
    nodes = list(G.nodes())
    converged = False
    while not converged:
        converged = True
        random.shuffle(nodes)
        for node in nodes:
            new = best_response(node, G, profile)
            if new != profile[node]:
                profile[node] = new
                converged = False
    return [profile[n] for n in sorted(G.nodes())]
```

Attenzione (i pareggi possono impedire la convergenza). La garanzia di terminazione vale solo se un nodo si sposta quando *migliora strettamente*. Un nodo che ha due cluster ugualmente frequenti tra i vicini e cambia anche a pari merito non aumenta Φ, e la dimostrazione non vale più. Verificato su un grafo con due cricche da 5 nodi e un nodo collegato a entrambe: con la regola "resta se sei già in un cluster ottimo" la dinamica termina al primo turno; con la regola "passa all'altro cluster ottimo" non termina mai. Il controllo nella `best_response` sopra serve a questo.

Risultati. Il grafo di test è un **modello a blocchi stocastico**: 4 comunità da 100 nodi, probabilità di arco 0,2 dentro una comunità e 0,01 tra comunità diverse. Su 20 grafi generati con semi diversi:

- in 19 casi su 20 la dinamica trova *esattamente* le 4 comunità vere, con purezza 1 (nessun nodo assegnato al blocco sbagliato);
- converge in media in 3,8 turni (tra 3 e 5), con 485–595 spostamenti di nodo;
- il potenziale cresce strettamente a ogni spostamento, come previsto (verificato su ogni mossa di ogni esecuzione);
- nel caso rimanente due comunità vengono fuse: 3 cluster, purezza 0,75.

Osservazione (il massimo di Φ è la soluzione sbagliata). Il profilo con il potenziale *massimo* è "tutti i nodi nello stesso cluster": $\Phi = |E|$, perché ogni arco ha gli estremi nello stesso gruppo. È un equilibrio di Nash (nessuno vuole lasciare un gruppo che contiene tutti i suoi vicini), ma non dice nulla sulle comunità. Infatti l'unica esecuzione che ha fuso due comunità è anche quella con il potenziale relativo più alto: l'89% degli archi finisce dentro un cluster, contro l'86–88% delle esecuzioni che trovano le 4 comunità giuste. La label propagation funziona perché si ferma in un *massimo locale* del potenziale, raggiunto partendo da etichette tutte diverse, e non perché lo massimizza. È un buon esempio della differenza tra "trovare un equilibrio" e "trovare l'equilibrio

> giusto".

Riepilogo della lezione

- Un gioco in forma normale è $\langle N, A, u\rangle$; con due giocatori si rappresenta con due matrici A (riga) e B (colonna).
- Una strategia strettamente dominata non è mai giocata. IESDS la elimina; se resta un solo profilo è l'unico equilibrio di Nash. Non tutti i giochi si risolvono così.
- Equilibrio di Nash: ognuno gioca una best response agli altri. Può essere unico, multiplo (battaglia dei sessi) o assente in strategie pure (rigori).
- Tragedia dei beni comuni: all'equilibrio si estrae $\frac{2A}{3B}$ invece dell'ottimo $\frac{A}{2B}$, e ognuno ottiene $\frac{A^2}{9B}$ invece di $\frac{A^2}{8B}$.
- Strategie miste: utilità attesa $\sigma_r A \sigma_c^T$. **Nash (1950)**: ogni gioco finito ha un equilibrio misto.
- Condizione del supporto: una strategia è best response se e solo se mette probabilità solo su azioni pure ottime. All'equilibrio ognuno è indifferente sul proprio supporto; nei 2×2 l'indifferenza dà l'equilibrio ($\frac{2}{3}, \frac{1}{3}$; $\frac{1}{3}, \frac{2}{3}$ nella battaglia dei sessi).
- Somma zero: **teorema minimax**, $\max\min = \min\max$; strategie ottime e valore si calcolano con un programma lineare (attenzione a lasciare libera la variabile del valore).
- **Lab**: verifica di best response ed equilibri, minimax con `linprog`, equilibri 2×2; label propagation come dinamica di best response in un gioco potenziale (4 comunità ritrovate in 19 grafi su 20).

Prossima lezione: esistenza dell'equilibrio di Nash, complessità del calcolo degli equilibri, equilibri correlati e apprendimento nei giochi.

LEZIONE 4

Equilibri alternativi e apprendimento

TEORIA

La lezione 3 ha lasciato quattro concetti di soluzione, ognuno con un limite:

Concetto	Limite
Eliminazione iterata (IESDS)	non tutti i giochi sono risolvibili per dominanza
Equilibrio di Nash puro	non tutti i giochi ne hanno uno
Equilibrio di Nash misto	esiste sempre nei giochi finiti (Nash, 1950), ma è difficile da calcolare oltre i giochi piccoli
Minimax	vale per i giochi a somma zero, dove si calcola in modo efficiente

Questa lezione risponde a tre domande:

1. *perché* l'equilibrio di Nash misto esiste sempre;
2. quanto è *difficile* calcolarlo;
3. che cosa si può fare in alternativa: **equilibri correlati** e **apprendimento** nei giochi ripetuti.

4.1 Esistenza dell'equilibrio di Nash misto

Teorema 4.1 (Nash, 1950). Ogni gioco finito ha un equilibrio di Nash misto.

Idea della dimostrazione: *ridurre* l'esistenza di un equilibrio misto all'esistenza di un **punto fisso** di una funzione.

4.1.1 Il teorema del punto fisso di Brouwer

Teorema 4.2 (Brouwer, 1911). Ogni funzione **continua** f che manda un insieme euclideo **compatto** e **convesso** in sé stesso ha un punto x_0 tale che $f(x_0) = x_0$.

Ricordando le definizioni:

- un insieme di $\mathbb{R}^d$ è **compatto** se è chiuso e limitato;
- un insieme C è **convesso** se per ogni $x, y \in C$ e ogni $\epsilon \in [0, 1]$ vale $\epsilon x + (1 - \epsilon)y \in C$: contiene il segmento tra due suoi punti qualsiasi.

Ogni ipotesi serve. Le slide mostrano un controesempio per ciascuna, sull'intervallo (o sull'anello) e con il grafico di f confrontato con la bisettrice $y = x$: un punto fisso è un punto in cui il grafico incontra la bisettrice.

Ipotesi tolta	Funzione senza punto fisso
Insieme limitato	$f(x) = x + 0{,}1$ su $\mathbb{R}$: una retta parallela alla bisettrice
Insieme chiuso	$f(x) = x^2$ sull'aperto $(0, 1)$: i punti fissi di x^2 sono 0 e 1, entrambi fuori dall'insieme
Insieme convesso	una rotazione dell'anello attorno al centro: il centro, che sarebbe il punto fisso, non sta nell'insieme
Funzione continua	$f(x) = x + \frac{1}{2}$ per $x < \frac{1}{2}$, $f(x) = x - \frac{1}{2}$ per $x \geq \frac{1}{2}$ su $[0, 1]$: il grafico "salta" oltre la bisettrice

4.1.2 La costruzione di Nash

Il piano della dimostrazione:

1. mostrare che l'insieme $S = S_1 \times S_2 \times \cdots \times S_n$ dei profili di strategie miste è compatto e convesso;
2. costruire una funzione continua $f : S \to S$ con $f(\sigma) = \sigma$ se e solo se σ è un equilibrio misto;
3. applicare Brouwer a f.

Passo 1: l'insieme dei profili. Le strategie miste del giocatore i formano il **simplesso standard**

$$S_i = \left\{ x \in \mathbb{R}^{|A_i|} \ : \ x \geq \mathbf{0},\ \mathbf{1}^T x = 1 \right\},$$

che è compatto e convesso. Il prodotto cartesiano di compatti è compatto e il prodotto di convessi è convesso, quindi anche S lo è.

> **Attenzione ("compatto e chiuso").** Le slide scrivono che S è "convesso, compatto e chiuso" nel piano della dimostrazione e poi solo "compatto e chiuso" nel passo 1. In $\mathbb{R}^d$ un compatto è già chiuso; ciò che serve per Brouwer, e che i due teoremi enunciati subito dopo dimostrano, è *compatto e convesso*.

Passo 2: la funzione. L'idea è una funzione che prende un profilo e ne restituisce uno *diverso*, a meno che il profilo non sia un equilibrio. La best response sarebbe la scelta naturale, ma non è una funzione (può avere più valori) e può essere discontinua. Si usa invece il **guadagno** che i otterrebbe passando dalla sua strategia mista all'azione pura a_i^j:

$$g\left(a_i^j, \sigma\right) = \max\left\{0,\ u_i\left(a_i^j, \sigma_{-i}\right) - u_i(\sigma)\right\},$$

e si sposta la probabilità verso le azioni che fanno guadagnare:

$$f(\sigma) = \sigma', \qquad \sigma'_{i,a_j} = \frac{\sigma_{i,a_j} + g(a_i^j, \sigma)}{\sum_{k \in A_i} \left(\sigma_{i,a_k} + g(a_i^k, \sigma)\right)}.$$

La funzione è continua (composizione di funzioni continue, con denominatore ≥ 1), manda S in sé (il risultato è ancora una distribuzione di probabilità) e ha un punto fisso se e solo se σ è un equilibrio misto. Per Brouwer un punto fisso esiste, quindi esiste un equilibrio. □

Approfondimento (il passaggio che le slide non dimostrano). Che un equilibrio sia un punto fisso è facile: all'equilibrio nessuna azione pura migliora, tutti i g sono 0 e $f(\sigma) = \sigma$. Il viceversa è il passo delicato. Sia σ un punto fisso e sia $G_i = \sum_k g(a_i^k, \sigma)$; allora per ogni azione

$$\sigma_{i,a_j} = \frac{\sigma_{i,a_j} + g(a_i^j, \sigma)}{1 + G_i} \quad \Longleftrightarrow \quad \sigma_{i,a_j}\, G_i = g(a_i^j, \sigma).$$

L'utilità $u_i(\sigma)$ è la media delle $u_i(a_i^j, \sigma_{-i})$ pesata con σ_i, quindi almeno un'azione del supporto ha $u_i(a_i^j, \sigma_{-i}) \le u_i(\sigma)$, cioè $g(a_i^j, \sigma) = 0$. Per quell'azione $\sigma_{i,a_j} > 0$ e $\sigma_{i,a_j} G_i = 0$, quindi $G_i = 0$: *nessuna* azione pura fa guadagnare i, e σ_i è una best response. Vale per ogni i, quindi σ è un equilibrio.

Attenzione (refusi). Il nome è *Brouwer*, non "Brower"; e f manda S *in* sé stesso (*into*), non necessariamente *su* tutto S (*onto*): Brouwer non richiede la suriettività.

4.2 Complessità del calcolo degli equilibri

4.2.1 Equilibri misti: PPAD-completezza

- La riduzione appena vista mostra che trovare un equilibrio misto non è più difficile che trovare un punto fisso.
- Una riduzione (molto più elaborata) mostra il contrario: trovare un punto fisso non è più difficile che trovare un equilibrio misto.
- Insieme: trovare un equilibrio misto è *esattamente* difficile quanto trovare un punto fisso. Il problema è **PPAD-completo**.

Definizione 4.3 (La classe PPAD). PPAD (*Polynomial Parity Argument on Directed graphs*) contiene i problemi di punto fisso. Il suo problema rappresentativo è **End of Line**: è dato implicitamente un grafo orientato $G = (V, E)$ con $|V| = 2^n$ nodi, in cui ogni nodo ha grado entrante e uscente al più 1, tramite due circuiti

$$S : \{0,1\}^n \to \{0,1\}^n \text{ (successore)}, \qquad P : \{0,1\}^n \to \{0,1\}^n \text{ (predecessore)}.$$

Dato un nodo sorgente s, si deve trovare un pozzo t (la "fine della linea").

Osservazione (perché è difficile e perché la soluzione esiste). La soluzione esiste sempre: seguendo i successori da s si arriva prima o poi a un nodo senza successore, perché i nodi sono finiti e non si possono ripetere. Ma il grafo ha 2^n nodi e la linea può essere lunga in modo esponenziale; i circuiti S e P non dicono dove finisce. È lo stesso tipo di problema di Nash: si sa che la soluzione c'è, non si sa come trovarla in fretta.

Il lemma di Sperner. È la controparte discreta di Brouwer. Si prende un reticolo triangolato e se ne colorano i nodi di bordo con tre colori secondo la regola delle slide (rosso sul lato sinistro, giallo in basso, blu in alto e a destra). Comunque si colorino i nodi interni, esiste un **triangolo tricromatico**, anzi ne esiste un numero *dispari*. Trovarne uno è PPAD-completo.

L'argomento che lo dimostra è proprio un "End of Line": si considerano i triangoli come nodi e le "porte" rosso-giallo come archi, e si attraversa ogni porta tenendo il rosso a sinistra. Partendo dalla porta sul bordo si cammina di triangolo in triangolo, e il cammino può finire solo in un triangolo tricromatico. Le slide riassumono la catena di riduzioni:

$$\text{EoL} \leq_p \text{Sperner} \leq_p \text{Brouwer} \leq_p \text{Nash}.$$

> **Osservazione (come leggere la catena).** $X \leq_p Y$ significa "X si riduce a Y": chi sa risolvere Y sa risolvere X, quindi Y è almeno difficile quanto X. La catena dice che Nash è almeno difficile quanto End of Line, cioè PPAD-*difficile*. La costruzione di Nash (Brouwer applicato a f) dà la direzione opposta, l'appartenenza a PPAD. Insieme: PPAD-completo.

4.2.2 Equilibri puri: NP-difficoltà

Trovare un equilibrio di Nash puro è NP-difficile. Lo si dimostra riducendo a esso il problema della **3-colorabilità**: dato un grafo $G = (V, E)$, decidere se si possono colorare i nodi con tre colori senza che due nodi adiacenti abbiano lo stesso colore.

Si usano i **giochi su grafo**: i giocatori sono i nodi e il payoff di ciascuno dipende solo dalle interazioni locali (la sua azione e quelle dei vicini).

Primo tentativo. Nodi = giocatori, azioni = $\{R, G, B\}$, payoff $+10$ se il colore scelto non è in conflitto con nessun vicino, -10 se lo è. Se G è 3-colorabile c'è un equilibrio con benessere massimo; se non lo è c'è comunque un equilibrio, solo con benessere più basso. Questo dimostra che trovare l'equilibrio *di benessere massimo* è NP-difficile, ma non che lo sia trovarne *uno*.

La costruzione giusta. Si aggiunge un **nodo sfidante** collegato a tutti gli altri:

- azioni dei nodi del grafo: $\{R, G, B, X, Y\}$; azioni dello sfidante: $\{X, Y\}$;
- payoff di un nodo: $+10$ se sceglie un colore senza conflitti, -10 se il colore è in conflitto, 0 se sceglie X/Y *copiando* lo sfidante, -5 se sceglie X/Y senza copiarlo;
- payoff dello sfidante: $+10$ se nessun nodo lo copia, -10 se qualcuno lo copia.

Allora il gioco ha un equilibrio puro *se e solo se* G è 3-colorabile.

> **Approfondimento (perché funziona).**
>
> - *Se G è 3-colorabile*: i nodi giocano una colorazione corretta ($+10$ ciascuno) e lo sfidante una lettera qualsiasi ($+10$, nessuno lo copia). Nessun nodo ha interesse a passare a una lettera ($\leq 0 < 10$), lo sfidante è già al massimo: equilibrio.
> - *Se non lo è*, si supponga per assurdo un equilibrio. Un nodo che gioca una lettera senza copiare prende -5 e preferirebbe copiare (0); quindi in equilibrio chi gioca una lettera copia lo sfidante. Ma se almeno un nodo lo copia, lo sfidante prende -10 e, cambiando lettera, nessuno lo copierebbe più ($+10$): devierebbe. Quindi nessun nodo gioca lettere: tutti giocano colori. Poiché G non è 3-colorabile, qualche nodo è in conflitto (-10) e preferirebbe copiare lo sfidante (0). Contraddizione.
>
> Verificato con una ricerca esaustiva su tutti i profili: il triangolo e il ciclo da 4 nodi (3-colorabili)

hanno un equilibrio puro, il grafo completo K_4 (non 3-colorabile) no.

Attenzione (NP-difficile rispetto a che cosa). Se un gioco è dato in forma normale *esplicita* (la tabella completa dei payoff), trovare un equilibrio puro è facile: basta controllare ogni cella, in tempo polinomiale nella dimensione della tabella. La difficoltà nasce perché nei giochi su grafo la rappresentazione è *compatta*: con $|V|$ nodi e 5 azioni la tabella avrebbe $5^{|V|} \cdot 2$ celle, ma il gioco si descrive con poche regole locali. Il risultato vale rispetto a questa descrizione compatta, ed è quella che conta nei problemi reali.

4.2.3 Il quadro complessivo

- Trovare un equilibrio di Nash misto è **PPAD-completo**.
- Trovare un equilibrio di Nash puro è **NP-difficile**.
- Trovare l'equilibrio che massimizza il benessere sociale è **NP-difficile**.

Conseguenza: se calcolare un equilibrio è difficile per un algoritmo centralizzato e ottimizzato, non ci si può aspettare che un sistema di agenti autonomi che interagiscono lo raggiunga da solo nei giochi generici. È la motivazione per le nozioni alternative di equilibrio e per l'apprendimento.

4.3 Equilibri correlati

4.3.1 L'intuizione: il semaforo

Il gioco dell'**attraversamento**: due auto arrivano a un incrocio e ciascuna sceglie se passare (*Go*) o fermarsi (*Stop*).

	Go	Stop
Go	$-10, -10$	$5, 0$
Stop	$0, 5$	$-1, -1$

- Due equilibri puri: (Go, Stop) con $u = (5, 0)$ e (Stop, Go) con $u = (0, 5)$. Sbilanciati.
- Un equilibrio misto: entrambi giocano Go con probabilità $\frac{3}{8}$, con $u_1 = u_2 = -\frac{5}{8}$. Equo ma pessimo: ci si scontra con probabilità $\frac{9}{64}$.

Nella realtà il problema si risolve con un **semaforo**: un *dispositivo di coordinamento* che raccomanda un'azione a ciascun giocatore. Lo si modella come una distribuzione di probabilità sui profili:

	Go	Stop
Go	0	$\frac{1}{2}$
Stop	$\frac{1}{2}$	0

Se entrambi seguono la raccomandazione ottengono $u_1 = u_2 = 2{,}5$. E **deviare dalla raccomandazione non conviene**, assumendo che l'altro la segua: chi riceve "Stop" sa che l'altro ha ricevuto "Go" e,

passando, prenderebbe -10 invece di 0; chi riceve "Go" sa che l'altro si ferma e, fermandosi, prenderebbe -1 invece di 5.

4.3.2 Definizione

Definizione 4.4 (Equilibrio correlato). Un **equilibrio correlato** (CE) è una distribuzione di probabilità sui profili di azioni

$$p : A_1 \times A_2 \times \cdots \times A_n \to [0,1]$$

tale che per ogni giocatore i e ogni coppia di azioni $a_i, a_i' \in A_i$

$$\sum_{a_{-i} \in A_{-i}} p(a_i, a_{-i})\, u_i(a_i, a_{-i}) \;\geq\; \sum_{a_{-i} \in A_{-i}} p(a_i, a_{-i})\, u_i(a_i', a_{-i}).$$

In parole: se il dispositivo raccomanda a_i al giocatore i, la probabilità condizionata degli altri profili è proporzionale a $p(a_i, \cdot)$, e contro quella distribuzione seguire la raccomandazione è almeno buono quanto passare a qualunque a_i'.

Teorema 4.5 (Esistenza). Ogni gioco finito ha un equilibrio correlato.

Approfondimento (perché esiste, e che rapporto c'è con Nash). Ogni equilibrio di Nash misto σ è anche un equilibrio correlato: basta prendere $p(a) = \prod_j \sigma_j(a_j)$, la distribuzione dei profili quando ognuno randomizza per conto suo. L'esistenza segue quindi dal teorema di Nash. Il viceversa non vale: il semaforo è un equilibrio correlato ma non è il prodotto di due strategie indipendenti (ha probabilità nulla su (Go,Go) e su (Stop,Stop), cosa impossibile per un prodotto con entrambe le marginali $(\tfrac{1}{2}, \tfrac{1}{2})$). Gli equilibri correlati sono un insieme più grande, e quindi possono contenere esiti migliori.

4.3.3 Le disuguaglianze nell'esempio

Per il giocatore 1 (riga) le due raccomandazioni danno:

$$\begin{aligned} \text{raccomandato Go:} \quad & p(G,G)\cdot(-10) + p(G,S)\cdot 5 \;\geq\; p(G,G)\cdot 0 + p(G,S)\cdot(-1) \\ \text{raccomandato Stop:} \quad & p(S,G)\cdot 0 + p(S,S)\cdot(-1) \;\geq\; p(S,G)\cdot(-10) + p(S,S)\cdot 5 \end{aligned}$$

e per il giocatore 2 (colonna):

$$\begin{aligned} \text{raccomandato Go:} \quad & p(G,G)\cdot(-10) + p(S,G)\cdot 5 \;\geq\; p(G,G)\cdot 0 + p(S,G)\cdot(-1) \\ \text{raccomandato Stop:} \quad & p(G,S)\cdot 0 + p(S,S)\cdot(-1) \;\geq\; p(G,S)\cdot(-10) + p(S,S)\cdot 5 \end{aligned}$$

Semplificando:

$$p(G,S) \geq \tfrac{5}{3}\, p(G,G), \quad p(S,G) \geq \tfrac{3}{5}\, p(S,S), \quad p(S,G) \geq \tfrac{5}{3}\, p(G,G), \quad p(G,S) \geq \tfrac{3}{5}\, p(S,S).$$

Il semaforo $p(G,G) = p(S,S) = 0$, $p(G,S) = p(S,G) = \tfrac{1}{2}$ le soddisfa tutte.

4.3.4 Calcolo con la programmazione lineare

Le condizioni sono **lineari** nelle incognite $p(a)$: un equilibrio correlato si trova risolvendo un programma lineare, cioè in tempo polinomiale nella dimensione del gioco. Gli equilibri correlati sono quindi *più facili* da calcolare di quelli di Nash. Si può scegliere l'obiettivo che si preferisce:

$$\begin{aligned} &\max \text{ [obiettivo a scelta]} \\ &\text{s.t.} \quad \sum_{a_{-i}} p(a_i, a_{-i})\big(u_i(a_i, a_{-i}) - u_i(a_i', a_{-i})\big) \geq 0 \quad \forall i,\ \forall a_i, a_i' \in A_i \\ &\qquad \textstyle\sum_{a \in A} p(a) = 1, \qquad p(a) \geq 0. \end{aligned}$$

Due obiettivi tipici:

- **benessere totale**: $\max \sum_{a \in A} p(a) \sum_i u_i(a)$;
- **regola maximin**: $\max \min_i \sum_{a \in A} p(a)\, u_i(a)$, cioè rendere il più alto possibile il payoff atteso del giocatore che sta peggio.

Attenzione (tre errori nelle slide del programma lineare).

- Nel vincolo, a destra le slide scrivono $p(a_i', a_{-i})\, u_i(a_i', a_{-i})$. La probabilità corretta è $p(a_i, a_{-i})$, la stessa del lato sinistro: si sta confrontando che cosa succede *con la stessa raccomandazione* se si gioca a_i oppure a_i'. Manca anche la sommatoria a destra. La definizione data qualche slide prima è giusta; lo è anche il notebook del docente.
- L'obiettivo "$\max \sum_i p(a)\, u_i(a)$" manca della somma sui profili a.
- "$\max \min_i p(a)\, u_i(a)$" non è un programma lineare così com'è (il minimo non è lineare). Lo diventa con una variabile ausiliaria t: $\max t$ s.t. $t \leq \sum_a p(a)\, u_i(a)$ per ogni i.

4.4 Altri concetti di soluzione

- **Trattare il gioco come a somma zero** e calcolare una strategia minimax: approccio conservativo, utile se si crede che l'avversario voglia danneggiarci o se non si conosce la sua utilità.
- **Minimizzare il massimo rimpianto**. Il **rimpianto** (*regret*) di un'azione è quanto si perde rispetto alla migliore risposta, a posteriori:

$$R(a_i, a_{-i}) = \Big[\max_{a_i' \in A_i} u_i(a_i', a_{-i})\Big] - u_i(a_i, a_{-i}), \qquad a_i^* \in \arg\min_{a_i \in A_i} \max_{a_{-i} \in A_{-i}} R(a_i, a_{-i}).$$

  È meno conservativo del minimax, utile quando l'avversario è imprevedibile più che ostile.

Approfondimento (minimax e minimo rimpianto nell'attraversamento). Per il giocatore 1: contro Go la risposta migliore vale 0 (Stop), contro Stop vale 5 (Go). I rimpianti sono:

	contro Go	contro Stop	rimpianto massimo
Go	$0 - (-10) = 10$	$5 - 5 = 0$	10
Stop	$0 - 0 = 0$	$5 - (-1) = 6$	**6**

Il minimo rimpianto massimo è Stop. Anche il criterio maximin sceglie Stop (caso peggiore -1 contro -10). Qui i due criteri coincidono, ma in generale no: il maximin guarda solo i payoff assoluti,

> il rimpianto li confronta con quello che si sarebbe potuto ottenere.

4.5 Apprendimento nei giochi

Si immagini ora che i giocatori giochino lo stesso gioco G *ripetutamente*, per T turni. Si vogliono algoritmi adattivi con cui i giocatori **imparino** a giocare strategie di equilibrio. Poiché calcolare un equilibrio di Nash è PPAD-completo, non ci si aspetta di impararlo in *ogni* gioco. Due algoritmi classici:

- **fictitious play** (Robinson, 1951);
- **regret matching** (Hart e Mas-Colell, 2000).

4.5.1 Fictitious play

Ogni giocatore stima la strategia degli avversari dalle frequenze osservate e gioca la best response a quella stima.

1. A ogni turno aggiorna il conteggio di quante volte gli avversari hanno giocato ciascuna azione: $\text{count}_{a_{-i}}[a_j]$.
2. Normalizza il conteggio in una distribuzione, la stima della strategia avversaria:
$$\sigma^t_{-i}[a_j] = \frac{\text{count}_{a_{-i}}[a_j]}{t-1}.$$
3. Gioca una best response alla stima: $a^t_i \in \arg\max_{a_i} \mathbb{E}\left[u_i(a_i, a_{-i}) \mid \sigma^t_{-i}\right]$.

Proprietà:

- aggiorna le credenze assumendo che gli altri giocatori siano **stazionari** (giochino sempre la stessa strategia mista), il che non è vero: anche loro imparano;
- non è garantito che converga; ma *se* le frequenze σ^t_{-i} convergono, convergono a un equilibrio di Nash;
- converge sicuramente a un equilibrio di Nash nei giochi a due giocatori a somma zero, nei giochi risolvibili per eliminazione iterata delle strategie strettamente dominate e nei giochi a due giocatori $2 \times N$.

> **Osservazione (che cosa converge).** Le *azioni* giocate in fictitious play sono sempre pure e possono ciclare per sempre: in sasso-carta-forbici l'algoritmo gioca sasso per un po', poi carta, poi forbici, … Ciò che converge all'equilibrio $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ sono le *frequenze empiriche*, cioè le medie nel tempo. Il grafico delle slide mostra proprio queste oscillazioni sempre più strette attorno a $\frac{1}{3}$.

4.5.2 Regret matching

Ogni giocatore decide quanto spesso giocare un'azione in base a *quanto avrebbe guadagnato in più* giocandola sempre, col senno di poi.

1. A ogni turno calcola il **rimpianto cumulato** di ciascuna azione a rispetto a ciò che ha effettivamente giocato:
$$R_i^t(a) = \sum_{s \le t} \Big(u_i(a, \sigma_{-i}^s) - u_i(\sigma_i^s, \sigma_{-i}^s) \Big).$$
2. Aggiorna la strategia in proporzione alla *parte positiva* del rimpianto:
$$\sigma_i^{t+1}(a) = \frac{\max(0, R_i^t(a))}{\sum_{a'} \max(0, R_i^t(a'))},$$
e gioca in modo uniforme se tutti i rimpianti sono ≤ 0.

Attenzione (la formula delle slide è un'altra cosa). Le slide scrivono $R_i^t(a) = \sum_{s \le t} \max\big(0,\ u_i(a, \sigma_{-i}^s) - u_i(\sigma^s)\big)$: la somma delle parti positive di ogni turno, invece della parte positiva della somma. Non è la stessa cosa: con la versione delle slide i rimpianti non possono mai diminuire, e un'azione che è stata buona in passato e poi pessima continua a essere giocata. Il regret matching di Hart e Mas-Colell, quello con le garanzie teoriche, usa $\max(0, \sum_s \dots)$.
L'effetto si misura nel laboratorio. In sasso-carta-forbici entrambe le versioni sembrano funzionare, perché per simmetria finiscono comunque attorno a $\frac{1}{3}$. Su un gioco a somma zero asimmetrico, con strategia minimax (0,33; 0,56; 0,11), la versione corretta converge a (0,34; 0,56; 0,10), quella delle slide a (0,41; 0,35; 0,24), che non è un equilibrio. Le slide inoltre non dicono che cosa fare quando tutti i rimpianti sono nulli (denominatore zero): si gioca uniforme.

Proprietà secondo le slide:

- nei giochi generali la media nel tempo converge all'insieme degli **equilibri correlati**;
- converge a un equilibrio di Nash nelle stesse classi di giochi di fictitious play (somma zero a due giocatori, risolvibili per dominanza, $2 \times N$).

Attenzione (che cosa converge, e a che cosa). Due precisazioni sull'enunciato delle slide, utili se all'esame si chiede "a che cosa converge il regret matching".

- *Che cosa converge.* Le slide parlano della media delle strategie, $\bar{\sigma}^T = \frac{1}{T} \sum_t \sigma_t$. Ma un equilibrio correlato è una distribuzione sui *profili congiunti*, non una strategia per giocatore: ciò che converge è la *distribuzione empirica dei profili giocati*, cioè la frazione di turni in cui si è giocato ciascun $(a_1, \dots, a_n)$.
- *A che cosa.* La versione vista sopra confronta ogni azione con la strategia giocata nel suo complesso (rimpianto *incondizionato*, o "esterno"). La teoria garantisce allora la convergenza agli **equilibri correlati grossolani** (*coarse correlated equilibria*): distribuzioni da cui nessun giocatore vorrebbe sganciarsi *prima* di ricevere la raccomandazione, per giocare sempre una stessa azione fissa. La convergenza agli equilibri correlati veri e propri vale per la variante del paper di Hart e Mas-Colell con rimpianti *condizionati*, che confronta ogni azione effettivamente giocata con ciascuna alternativa ("avrei fatto meglio a giocare k *ogni volta che* ho giocato j?").

Approfondimento (perché nei giochi a somma zero si arriva comunque a Nash). Nei giochi a due giocatori a somma zero la distinzione non conta: le marginali di qualunque equilibrio correlato, anche grossolano, sono strategie minimax. Se nessuno dei due ha rimpianti verso un'azione fissa, ciascuno ottiene almeno il valore del gioco; poiché la somma dei payoff è zero, entrambi ottengono esattamente il valore, e le frequenze di ciascuno sono una strategia ottima. Analogamente, un

> giocatore senza rimpianti gioca sempre più di rado un'azione strettamente dominata da un'altra azione pura (ogni volta che la gioca accumula rimpianto verso quella che la domina), il che spiega il caso dei giochi risolvibili per dominanza.

4.6 Laboratorio: Equilibri correlati e apprendimento

LAB

[Lesson4-NonCooperativeGameTheory_with_Learning_in_Games.ipynb]

Il notebook ripete la prima parte del laboratorio 3 (best response, equilibri, minimax, equilibri 2×2: vedi lezione 3) e aggiunge tre esercizi: il programma lineare per gli equilibri correlati, fictitious play e regret matching. Il notebook contiene gli output della soluzione del docente: tutti i numeri sotto li riproducono.

4.6.1 Equilibri correlati con `linprog`

Le incognite sono le $m \cdot n$ probabilità $p(i, j)$, appiattite riga per riga. Ogni coppia (azione raccomandata, deviazione) dà un vincolo. `linprog` vuole vincoli ≤ 0 e un problema di minimo, quindi si cambia segno.

```
import numpy as np
from scipy.optimize import linprog

def build_ce_lp(A1, A2):
    A1, A2 = np.asarray(A1, float), np.asarray(A2, float)
    m, n = A1.shape
    idx = lambda i, j: i * n + j
    rows = []
    for i in range(m):                    # giocatore 1: raccomandato i, devia a k
        for k in range(m):
            if k != i:
                r = np.zeros(m * n)
                for j in range(n):
                    r[idx(i, j)] = -(A1[i, j] - A1[k, j])
                rows.append(r)
    for j in range(n):                    # giocatore 2: raccomandato j, devia a l
        for l in range(n):
            if l != j:
                r = np.zeros(m * n)
                for i in range(m):
                    r[idx(i, j)] = -(A2[i, j] - A2[i, l])
                rows.append(r)
    A_ub, b_ub = np.array(rows), np.zeros(len(rows))
    A_eq, b_eq = np.ones((1, m * n)), np.array([1.0])
    c = -(A1 + A2).flatten()              # max benessere = min -benessere
    bounds = [(0, None)] * (m * n)
    return c, A_ub, b_ub, A_eq, b_eq, bounds, (m, n)

def solve_ce_max_social_welfare(A1, A2, method='highs'):
    c, A_ub, b_ub, A_eq, b_eq, bounds, (m, n) = build_ce_lp(A1, A2)
    res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method=method)
    dist = res.x.reshape(m, n)
    return {"distribution": dist,
            "social_welfare": float((dist * (A1 + A2)).sum()),
            "linprog_result": res}
```

Risultati.

- *Attraversamento*: benessere massimo 5,0, distribuzione $\left(\begin{smallmatrix}0&1\\0&0\end{smallmatrix}\right)$, cioè l'equilibrio puro (Go, Stop). La matrice dei vincoli è identica a quella del notebook:

$$A_{ub} = \begin{pmatrix} 10 & -6 & 0 & 0 \\ 0 & 0 & -10 & 6 \\ 10 & 0 & -6 & 0 \\ 0 & -10 & 0 & 6 \end{pmatrix} \qquad \text{(variabili: } p_{GG}, p_{GS}, p_{SG}, p_{SS}\text{)}.$$

- *Rigori*: benessere 0, distribuzione uniforme 0,25 su ogni cella.

Osservazione (il programma lineare sceglie un vertice). Anche il semaforo ha benessere 5 (verificato: è un equilibrio correlato), ma il risolutore restituisce (Go, Stop): quando più soluzioni hanno lo stesso valore ottimo, ne restituisce una qualsiasi, di solito un vertice del poliedro. Se si vuole la soluzione *equa* bisogna chiederla, con l'obiettivo maximin: risolvendo $\max t$ s.t. $t \le \mathbb{E}_p[u_i]$ si ottiene esattamente il semaforo, $\left(\begin{smallmatrix}0&\frac{1}{2}\\\frac{1}{2}&0\end{smallmatrix}\right)$, con 2,5 a testa.

Osservazione (il commento sui rigori). Il notebook motiva la distribuzione uniforme con "il suo equilibrio di Nash è $[0{,}5; 0{,}5]$". Il motivo vero è più netto: nei rigori i quattro vincoli diventano $p_{12} \le p_{11} \le p_{21} \le p_{22} \le p_{12}$, una catena chiusa che forza tutte le probabilità a essere uguali. L'equilibrio correlato è *unico*, e l'obiettivo non conta (in un gioco a somma zero il benessere è 0 per qualunque distribuzione).

4.6.2 Fictitious play

```python
def fictitious_play(P, iterations=1000):
    p1_counts, p2_counts = np.ones(3), np.ones(3)   # 1 per evitare divisioni per 0
    sigma_p1, sigma_p2 = [], []
    for _ in range(iterations):
        p1_prob = p1_counts / p1_counts.sum()        # frequenze empiriche
        p2_prob = p2_counts / p2_counts.sum()
        a1 = np.argmax(P @ p2_prob)                  # BR alla stima sull'altro
        a2 = np.argmax(-P.T @ p1_prob)               # somma zero: payoff -P
        p1_counts[a1] += 1
        p2_counts[a2] += 1
        sigma_p1.append(p1_counts / p1_counts.sum())
        sigma_p2.append(p2_counts / p2_counts.sum())
    return p1_prob, p2_prob, np.array(sigma_p1), np.array(sigma_p2)
```

In sasso-carta-forbici le frequenze si avvicinano a $\frac{1}{3}$, con oscillazioni che si stringono:

Turni	Frequenze del giocatore 1	Scarto massimo da $\frac{1}{3}$
100	(0,359; 0,301; 0,340)	0,032
1000	(0,347; 0,319; 0,334)	0,014
10000	(0,332; 0,336; 0,333)	0,002

> **Osservazione (argmax e i pareggi).** `np.argmax` restituisce sempre il *primo* massimo: è una regola di pareggio deterministica. In sasso-carta-forbici la matrice è antisimmetrica ($-P^T = P$), quindi i due giocatori calcolano la stessa best response a partire dagli stessi conteggi: giocano sempre la stessa azione e hanno frequenze identiche. Non è un errore, ma è un artefatto da tenere presente se si usa fictitious play per confronti nel progetto.

4.6.3 Regret matching

```python
def regret_matching(P, T=10000, seed=0):
    rng = np.random.default_rng(seed)
    R1, R2 = np.zeros(3), np.zeros(3)          # rimpianti cumulati
    p1 = p2 = np.ones(3) / 3
    counts1 = np.zeros(3)
    for t in range(T):
        a1, a2 = rng.choice(3, p=p1), rng.choice(3, p=p2)
        counts1[a1] += 1
        R1 += P[:, a2] - P[a1, a2]             # quanto avrei preso con ogni
    azione
        R2 += -P[a1, :] + P[a1, a2]            # payoff della colonna = -P
        pos1, pos2 = np.maximum(R1, 0), np.maximum(R2, 0)
        p1 = pos1 / pos1.sum() if pos1.sum() > 0 else np.ones(3) / 3
        p2 = pos2 / pos2.sum() if pos2.sum() > 0 else np.ones(3) / 3
    return counts1 / T
```

Qui le azioni sono campionate e il rimpianto si calcola sull'azione effettivamente giocata dall'avversario; le slide usano le strategie miste (utilità attese). Il rimpianto campionato è una stima non distorta di quello atteso, quindi le due versioni si comportano allo stesso modo in media.

Risultati. In sasso-carta-forbici, dopo 10 000 turni, la frequenza del giocatore 1 è $(0{,}335;\ 0{,}332;\ 0{,}333)$, con scarto massimo da $\frac{1}{3}$ tra 0,002 e 0,010 su 5 semi. La versione con la formula delle slide dà scarti simili su questo gioco, ed è per questo che il difetto non si vede nel notebook. Il confronto su un gioco asimmetrico

$$M = \begin{pmatrix} 2 & -1 & 1 \\ -1 & 1 & 0 \\ 1 & 0 & -1 \end{pmatrix}, \qquad \text{minimax della riga } (0{,}333;\ 0{,}556;\ 0{,}111),\ \text{valore } 0{,}222,$$

dopo 20 000 turni su 3 semi:

Versione	Frequenze della riga
$\max(0, \sum_s r_s)$ (Hart, Mas-Colell)	$(0{,}34;\ 0{,}56;\ 0{,}10)$, $(0{,}34;\ 0{,}56;\ 0{,}11)$, $(0{,}35;\ 0{,}57;\ 0{,}08)$
$\sum_s \max(0, r_s)$ (slide)	$(0{,}41;\ 0{,}35;\ 0{,}24)$, $(0{,}41;\ 0{,}35;\ 0{,}24)$, $(0{,}41;\ 0{,}35;\ 0{,}24)$

La seconda converge in modo stabile, ma al punto sbagliato: contro la seconda colonna la strategia $(0{,}41;\ 0{,}35;\ 0{,}24)$ ottiene $-0{,}06$, mentre il minimax garantisce 0,222 contro qualunque colonna.

Riepilogo della lezione

- **Nash (1950)** via **Brouwer**: l'insieme dei profili misti è compatto e convesso, la funzione dei "guadagni" è continua e i suoi punti fissi sono esattamente gli equilibri.
- Trovare un equilibrio misto è **PPAD-completo** (End of Line, Sperner); trovare un equilibrio puro, o quello di benessere massimo, è **NP-difficile** (riduzione da 3-colorabilità con un nodo sfidante).
- **Equilibrio correlato**: una distribuzione sui profili da cui a nessuno conviene deviare data la propria raccomandazione. Ogni Nash è un CE; il semaforo è un CE che non è Nash. Si calcola con un programma lineare.
- Altri criteri: minimax (avversario ostile), minimo rimpianto massimo (avversario imprevedibile).
- **Fictitious play**: best response alle frequenze osservate; converge nei giochi a somma zero, risolvibili per dominanza, $2 \times N$. **Regret matching**: si gioca in proporzione al rimpianto cumulato positivo; la distribuzione dei profili converge agli equilibri correlati (grossolani, per la versione con rimpianti incondizionati), e al minimax nei giochi a somma zero.
- **Lab**: CE con `linprog` (semaforo con l'obiettivo maximin), fictitious play e regret matching su RPS; la formula del rimpianto delle slide converge al punto sbagliato su un gioco asimmetrico.

Prossima lezione: giochi coalizionali (core, nucleolo, valore di Shapley).

LEZIONE 5

Giochi coalizionali

TEORIA

Il percorso della lezione:

1. introduzione ai giochi coalizionali;
2. come rappresentarli;
3. quali coalizioni si formano;
4. come dividere il payoff: **stabilità** (core) e **equità** (nucleolo, valore di Shapley).

5.1 Dai giochi non cooperativi a quelli coalizionali

- I giochi non cooperativi modellano giocatori egoisti e in competizione.
- Giochi come il dilemma del prigioniero o l'attraversamento mostrano che la competizione può portare a esiti subottimi.
- Gli equilibri correlati modellano (e rendono possibile) la cooperazione, ma introducono un dispositivo di coordinamento, un *mediatore*.
- Nella realtà, però, le persone formano spontaneamente **coalizioni** e prendono **accordi vincolanti**.

La situazione tipica che si modella con un gioco coalizionale è questa:

- i giocatori possono trarre vantaggio dal formare coalizioni;
- a ogni coalizione è associato un **valore**;
- il valore totale va **distribuito** tra i giocatori.

Servono due scelte di modellazione:

	Domanda	Modello
Coalizioni	il risultato di una coalizione è indipendente dalle altre?	sì → coalizioni singole; no → più coalizioni insieme (giochi in funzione di partizione)
Valore	l'utilità è trasferibile?	sì → un solo numero per coalizione (TU); no → un insieme di vettori di utilità (NTU)

Definizione 5.1 (Utilità trasferibile e non trasferibile). L'utilità è **trasferibile** (TU) se è la stessa per tutti i membri della coalizione e si può ridistribuire tra loro: i payoff sono monetari (o convertibili in denaro), i giocatori possono farsi pagamenti laterali, e la ridistribuzione non cambia il valore

totale. Esempi: dividere profitti, ripartire costi, assegnare seggi in parlamento.
L'utilità è **non trasferibile** (NTU) se ogni membro ha la propria e la coalizione non può convertirla o ridistribuirla liberamente. Esempi: il prestigio di un articolo scientifico scritto insieme, i benefici personali di una collaborazione, i vantaggi legati a indicatori di prestazione individuali.

Attenzione (denaro non vuol dire utilità trasferibile). Il notebook lo dice bene: poter trasferire *denaro* non implica che sia trasferibile l'*utilità*. Un giocatore ricco e uno povero possono trarre utilità molto diverse dalla stessa somma (lezione 2: la funzione di Bernoulli è concava). Assumere TU significa assumere che tutti valutino il denaro allo stesso modo, cioè utilità lineari nel denaro.

5.2 Giochi in funzione caratteristica

Definizione 5.2 (Gioco in funzione caratteristica). Un gioco in funzione caratteristica è una coppia (N, v) dove:

- $N = \{1, \dots, n\}$ è l'insieme dei giocatori, detto anche **grande coalizione**;
- $v : 2^N \to \mathbb{R}$ è la **funzione caratteristica**: per ogni sottoinsieme C di giocatori, $v(C)$ è quanto possono guadagnare i membri di C lavorando insieme. L'utilità $v(C)$ è trasferibile.

Di solito si assume che v sia

- **normalizzata**: $v(\emptyset) = 0$;
- **non negativa**: $v(C) \geq 0$ per ogni $C \subseteq N$;
- **monotona**: $v(C) \leq v(D)$ se $C \subseteq D$.

Esempio (il gelato). Tre persone vogliono comprare del gelato e possono mettere insieme i soldi.

Giocatore	Soldi	Vaschetta	Prezzo
A	\$6	500 g	\$7
B	\$3	750 g	\$9
C	\$3	1000 g	\$11

Il valore di una coalizione è la quantità massima di gelato che può comprare:

$$
\begin{aligned}
v(\emptyset) = v(\{A\}) = v(\{B\}) = v(\{C\}) &= 0 && \text{(nessuno ha \$7 da solo)} \\
v(\{A, B\}) = v(\{A, C\}) &= 750 && \text{(\$9: la vaschetta da 750 g)} \\
v(\{B, C\}) &= 0 && \text{(\$6: non basta per nulla)} \\
v(\{A, B, C\}) &= 1000 && \text{(\$12: la vaschetta da 1000 g).}
\end{aligned}
$$

Oltre la funzione caratteristica. I giochi in funzione caratteristica modellano utilità trasferibile e risultati di una coalizione indipendenti da ciò che fanno gli altri. Se i risultati dipendono dalle altre coalizioni si usano i **giochi in funzione di partizione**, con valori $v(C, P)$ dove C è una coalizione della partizione P; se l'utilità non è trasferibile si usano i **giochi NTU**, dove a ogni coalizione è associato un *insieme* di vettori di utilità. Ogni gioco TU si può rappresentare come un gioco NTU con un continuo di

azioni, quindi

$$\text{NTU} \supset \text{TU} \supset \text{giochi in funzione caratteristica;}$$

nella pratica si usano soprattutto questi ultimi, che spesso si chiamano semplicemente "giochi TU".

Le due domande. I giochi coalizionali servono a rispondere a due domande:

1. quali coalizioni si formeranno?
2. come si divide il payoff all'interno di una coalizione?

Nel gioco del gelato la coalizione con il benessere più alto è $\{A, B, C\}$. Ma dividere in parti uguali non è equo: A è più decisivo di B e C, perché senza di lui non si compra nulla.

5.3 Struttura di coalizione

Definizione 5.3 (Generazione della struttura di coalizione). Dato un gioco $G = (N, v)$ si cerca la **struttura di coalizione** $CS(G)$, cioè una *partizione* di N in coalizioni disgiunte, che massimizza il benessere sociale $\sum_{C \in CS(G)} v(C)$.

Attenzione (partizione, non ricoprimento). Le slide dicono che una struttura di coalizione è un "ricoprimento" (*cover*) di N. Un ricoprimento può avere insiemi sovrapposti, e un giocatore che sta in due coalizioni verrebbe contato due volte. Quello che si intende, e che le slide stesse scrivono più avanti, è una **partizione**: le coalizioni coprono N e sono a due a due disgiunte.

Esempio (quando non conviene la grande coalizione).

$$v(\emptyset) = 0, \quad v(\{A\}) = 0, \quad v(\{B\}) = 300, \quad v(\{C\}) = 0,$$
$$v(\{A, B\}) = 750, \quad v(\{A, C\}) = 750, \quad v(\{B, C\}) = 400, \quad v(\{A, B, C\}) = 1000.$$

La struttura di coalizione che massimizza il benessere è $\{\{A, C\}, \{B\}\}$, con $750+300 = 1050 > 1000$: qui la grande coalizione *non* è la scelta migliore.

5.3.1 Il problema è difficile

Le partizioni di un insieme di n elementi sono il **numero di Bell** B_n, che soddisfa

$$\left(\frac{n}{4}\right)^{n/2} \leq B_n < n^n.$$

Enumerarle tutte è impraticabile. Gli algoritmi usati sono la programmazione dinamica, il grafo delle partizioni e le euristiche per giochi strutturati.

5.3.2 Programmazione dinamica

Sia $v^*(S)$ il massimo benessere ottenibile con una struttura di coalizione dei giocatori di S, e $CS^*(S)$ la struttura che lo realizza: $v(CS^*(S)) = v^*(S)$.

Teorema 5.4 (Equazione della programmazione dinamica). Per ogni $C \subseteq N$:

$$v^*(C) = \max\Big\{v(C),\ \max_{\substack{C' \cup C'' = C \\ C' \cap C'' = \emptyset}} \big(v^*(C') + v^*(C'')\big)\Big\}.$$

O la coalizione C resta intera, o la si divide in due parti, ciascuna delle quali va poi organizzata nel modo migliore.

Dimostrazione. ($\geq$) $v^*(C) \geq v(C)$, perché $\{C\}$ è una struttura di coalizione di C; e $v^*(C) \geq v^*(C') + v^*(C'')$, perché unendo le strutture ottime delle due parti si ottiene una struttura di C. ($\leq$) Se $CS^*(C) = \{C\}$ allora $v^*(C) = v(C)$. Altrimenti sia C' una coalizione di $CS^*(C)$ e $C'' = C \setminus C'$. Il resto, $CS^*(C) \setminus \{C'\}$, è una struttura di coalizione di C'', quindi $v^*(C'') \geq v^*(C) - v(C')$; inoltre $v^*(C') \geq v(C')$. Sommando, $v^*(C') + v^*(C'') \geq v^*(C)$. □

Esempio (la tabella della programmazione dinamica). Con la funzione caratteristica dell'esempio precedente, procedendo per dimensione crescente:

C	$v^*(C)$	$CS^*(C)$
$\{A\}$	0	$\{A\}$
$\{B\}$	300	$\{B\}$
$\{C\}$	0	$\{C\}$
$\{A,B\}$	$\max(v^*(A) + v^*(B),\ v(AB)) = \max(300, 750) = 750$	$\{A,B\}$
$\{A,C\}$	$\max(0, 750) = 750$	$\{A,C\}$
$\{B,C\}$	$\max(300, 400) = 400$	$\{B,C\}$
$\{A,B,C\}$	$\max\big(1000,\ 750+0,\ 750+300,\ 400+0\big) = 1050$	$\{\{A,C\},\{B\}\}$

Per $\{A,B,C\}$ le divisioni in due parti sono tre: $\{A,B\}|\{C\}$, $\{A,C\}|\{B\}$, $\{B,C\}|\{A\}$. La migliore è la seconda.

Attenzione (un errore nella tabella delle slide). Le slide scrivono $v^*(\{B,C\}) = \max(300, 400) = 750$ e riportano 750 in tabella. Il massimo è 400. L'errore non cambia il risultato finale (1050 viene dalla divisione $\{A,C\}|\{B\}$, che non usa $v^*(\{B,C\})$), ma nel passo successivo le slide elencano due volte la divisione $\{A,B\}|\{C\}$ e mai $\{B,C\}|\{A\}$, proprio quella che userebbe il valore sbagliato. Il codice del laboratorio conferma la tabella corretta.

L'approccio è **esatto**, ma il tempo è $O(3^n)$: non è praticabile per $n \geq 20$.

Approfondimento (da dove viene 3^n). Si considerano tutte le coppie (coalizione C, sua parte C'). Ogni giocatore, in una coppia, può stare in C', in $C \setminus C'$ o fuori da C: tre possibilità per ciascuno degli n giocatori, cioè 3^n coppie. Con $n = 20$ sono circa 3,5 miliardi.

Grafo delle partizioni. Le partizioni si possono disporre su livelli, dal livello 1 (la sola grande coalizione) al livello n (tutti singoli); due partizioni sono collegate se una si ottiene dall'altra fondendo due coalizioni. Le slide mostrano il grafo per 4 giocatori senza svilupparlo; l'idea degli algoritmi che lo usano è esplorare per primi pochi livelli scelti bene e fermarsi quando si ha una garanzia sufficiente sulla distanza dall'ottimo.

In pratica. Si usano euristiche e rappresentazioni compatte della funzione caratteristica: funzioni caratteristiche analitiche, algoritmi di clustering (formazione delle coalizioni per similarità o distanza, partizionamento di grafi), metaeuristiche (algoritmi genetici, *simulated annealing*, . . .).

5.4 Dividere il payoff

Definizione 5.5 (Esito di un gioco TU). Un **esito** di un gioco TU $G = (N, v)$ è una coppia (CS, x) dove:

- $CS = (C_1, \ldots, C_k)$ è una struttura di coalizione, cioè una partizione di N: $\bigcup_i C_i = N$, $C_i \cap C_j = \emptyset$ per $i \neq j$;
- $x = (x_1, \ldots, x_n)$ è un **vettore di payoff** che distribuisce il valore di ogni coalizione di CS: $\sum_{i \in C} x_i = v(C)$ per ogni $C \in CS$ (**efficienza**).

Un esito è un'**imputazione** se soddisfa la **razionalità individuale**: $x_i \geq v(\{i\})$ per ogni $i \in N$.

Esempio (esiti). Con $v(\{1,2,3\}) = 9$ e $v(\{4,5\}) = 4$:

- $\big((\{1,2,3\}, \{4,5\}),\ (3,3,3,3,1)\big)$ è un esito: $3 + 3 + 3 = 9$, $3 + 1 = 4$;
- $\big((\{1,2,3\}, \{4,5\}),\ (2,3,2,3,3)\big)$ *non* lo è: la prima coalizione distribuisce 7 e la seconda 6. Non sono ammessi trasferimenti *tra* coalizioni.

5.4.1 Superadditività

Definizione 5.6 (Gioco superadditivo). Un gioco (N, v) è **superadditivo** se

$$v(C_1 \cup C_2) \geq v(C_1) + v(C_2) \qquad \forall C_1, C_2 \subset N \text{ con } C_1 \cap C_2 = \emptyset.$$

Il gioco del gelato è superadditivo: per esempio $v(\{A,B\}) = 750 \geq v(\{A\}) + v(\{B\}) = 0$, $v(\{B,C\}) = 0 \geq 0 + 0$, $v(\{A,B,C\}) = 1000 \geq 750 + 0$.

Nei giochi superadditivi due coalizioni possono sempre fondersi senza perdere nulla: si può assumere che si formi la **grande coalizione**. Non c'è il problema della struttura di coalizione e la notazione si semplifica: un esito è solo un vettore $x = (x_1, \ldots, x_n)$ con $\sum_{i \in N} x_i = v(N)$.

Osservazione (un'avvertenza delle slide). Molti articoli definiscono gli esiti in questo modo, come semplici vettori con $\sum x_i = v(N)$, *anche* quando il gioco non è superadditivo. Va tenuto presente quando si leggono: nel gioco della programmazione dinamica, per esempio, dividere $v(N) = 1000$ sarebbe sbagliato, perché la struttura migliore produce 1050.

Da qui in poi si considerano solo giochi TU superadditivi.

5.5 Stabilità: il core

Nel gioco del gelato si deve dividere 1000. Se la divisione è $(200, 200, 600)$, A e B possono comprare da soli la vaschetta da 750 g e dividersela a metà, 375 a testa: meglio di 200. L'esito $(200, 200, 600)$ **non è stabile**.

Definizione 5.7 (Core). Il **core** di un gioco è l'insieme degli esiti stabili, quelli da cui nessuna coalizione vuole sganciarsi:

$$\text{Core}(G) = \left\{ x \;\middle|\; x_i \geq 0 \;\forall i \in N,\ \sum_{i \in N} x_i = v(N),\ \sum_{i \in C} x_i \geq v(C)\ \forall C \subseteq N \right\}.$$

Ogni coalizione riceve almeno quanto guadagnerebbe da sola.

Esempio (esiti nel core).

- Con $v(\{1,2,3\}) = 9$, $v(\{4,5\}) = 4$ e $v(\{2,4\}) = 7$, l'esito $(3,3,3,3,1)$ *non* è nel core: i giocatori 2 e 4 ricevono $3+3=6<7$ e preferiscono formare la coalizione $\{2,4\}$.
- Nel gelato, $(200,200,600)$ non è nel core perché $v(\{A,B\}) = 750 > x_A + x_B = 400$.
- Nel gelato, $(500,250,250)$ *è* nel core: $x_A + x_B = 750 \geq 750$, $x_A + x_C = 750 \geq 750$, $x_B + x_C = 500 \geq 0$. Nessuna deviazione dà di più.

Osservazione (il primo esempio). L'esempio con $(3,3,3,3,1)$ usa una struttura di coalizione $\{\{1,2,3\},\{4,5\}\}$, mentre la definizione di core appena data è quella per la grande coalizione. Il concetto è lo stesso (nessuna coalizione deve poter fare meglio da sola), ma nella versione con strutture di coalizione l'efficienza si chiede coalizione per coalizione.

Approfondimento (com'è fatto il core del gelato). I vincoli che contano sono due. Da $x_A + x_B \geq 750$ e $x_A + x_B + x_C = 1000$ segue $x_C \leq 250$; analogamente $x_B \leq 250$. Il core è quindi

$$\text{Core} = \{x \geq 0 : x_B \leq 250,\ x_C \leq 250,\ x_A = 1000 - x_B - x_C\},$$

e in particolare $x_A \geq 500$ in ogni esito stabile. Si vede bene il potere di A: se B chiedesse più di 250, A potrebbe accordarsi con C e dividere 750 dando a entrambi più di quanto ricevono; lo stesso vale a parti invertite. B e C sono in concorrenza tra loro per essere il partner di A.

5.5.1 Due cattive notizie

Esempio (il core può essere vuoto).

$$v(\{1\}) = v(\{2\}) = v(\{3\}) = 0, \quad v(\{1,2\}) = 7,\ v(\{1,3\}) = 6,\ v(\{2,3\}) = 5, \quad v(\{1,2,3\}) = 8.$$

Per stare nel core serve stabilità, $x_1 + x_2 \geq 7$, $x_1 + x_3 \geq 6$, $x_2 + x_3 \geq 5$, ed efficienza, $x_1 + x_2 + x_3 = 8$. Sommando le tre disuguaglianze: $2(x_1 + x_2 + x_3) \geq 18$, cioè $x_1 + x_2 + x_3 \geq 9 > 8$. Impossibile: il core è vuoto. E il gioco è superadditivo, quindi la superadditività non basta a garantire un core non vuoto.

Esempio (il core può contenere esiti iniqui). Nel gelato $(1000,0,0)$ è nel core: $x_A + x_B = 1000 \geq 750$, $x_A + x_C = 1000 \geq 750$, $x_B + x_C = 0 \geq 0$, e la somma è 1000. B e C mettono metà dei soldi e non ricevono nulla, ma nessuna coalizione può fare meglio da sola. **Stabilità non vuol dire equità.**

5.6 Il nucleolo

Definizione 5.8 (Eccesso). L'**eccesso** di una coalizione S rispetto all'esito x è

$$e(S, x) = v(S) - \sum_{i \in S} x_i.$$

Misura l'**insoddisfazione** della coalizione: quanto potrebbe guadagnare da sola rispetto a quello che riceve. Un esito è nel core se e solo se tutti gli eccessi sono ≤ 0.

Esempio (eccessi). Con $v(\{i\}) = 0$, $v(\{i, j\}) = 1$ per ogni coppia e $v(\{1, 2, 3\}) = 3$:

- $x = (0, 0, 3)$: $e(\{1, 2\}, x) = 1 - 0 = 1$, la coppia $\{1, 2\}$ è insoddisfatta;
- $x = (1, 2, 0)$: $e(\{1, 2\}, x) = 1 - 3 = -2$, la coppia è più che soddisfatta.

Definizione 5.9 (Nucleolo). Sia $\theta(x)$ il **vettore degli eccessi** di tutte le coalizioni (diverse da $\emptyset$ e da N), ordinato dal più grande al più piccolo. Il **nucleolo** è l'imputazione

$$\eta \in X(G) \ : \ \nexists\, y \in X(G) \text{ con } \theta(y) \prec_{lex} \theta(\eta),$$

dove $X(G)$ è l'insieme delle imputazioni e $\prec_{lex}$ l'ordine lessicografico.

In parole: è l'esito che rende **lessicograficamente minimo** il vettore delle insoddisfazioni. Prima si rende la coalizione più insoddisfatta il meno insoddisfatta possibile; a parità, la seconda; e così via.

Esempio (nucleolo del gioco simmetrico). Nel gioco precedente:

- $x = (1, 2, 0)$: $\theta(x) = (0, 0, -1, -1, -2, -2)$;
- $\eta = (1, 1, 1)$: $\theta(\eta) = (-1, -1, -1, -1, -1, -1)$.

Il primo elemento di $\theta(\eta)$ è $-1 < 0$: η è lessicograficamente migliore, ed è il nucleolo (lo conferma il calcolo con l'algoritmo seguente).

Approfondimento (il nucleolo esiste sempre ed è unico). Il nucleolo esiste ed è unico per ogni gioco con almeno un'imputazione. E se il core non è vuoto il nucleolo sta nel core: un esito del core ha tutti gli eccessi ≤ 0, quindi il nucleolo, che minimizza il più grande, ne ha anch'esso il più grande ≤ 0. È per questo che lo si considera la versione "più centrale" del core.

5.6.1 L'algoritmo di Kopelowitz

Il vettore degli eccessi si minimizza lessicograficamente con una *sequenza* di programmi lineari. Il primo minimizza l'eccesso massimo ϵ_1:

$$\begin{aligned} & \min \epsilon_1 \\ LP_1: \quad & \text{s.t. } v(S) - \textstyle\sum_{i \in S} x_i \leq \epsilon_1 \qquad \forall S \subset N,\ S \notin W_0 = \{\emptyset\} \\ & \quad x_i \geq v(\{i\}), \quad \textstyle\sum_{i \in N} x_i = v(N). \end{aligned}$$

Le coalizioni che all'ottimo hanno eccesso esattamente ϵ_1^* formano l'insieme W_1 e il loro eccesso viene *fissato*. Il programma k-esimo minimizza il massimo eccesso delle coalizioni rimaste:

$$LP_k: \quad \begin{aligned} &\min \epsilon_k \\ &\text{s.t.} \quad v(S) - \textstyle\sum_{i\in S} x_i = \epsilon_r^* && \forall S \in W_r,\ r = 1,\dots,k-1 \\ &\qquad v(S) - \textstyle\sum_{i\in S} x_i \le \epsilon_k && \forall S \notin W_0 \cup W_1 \cup \cdots \cup W_{k-1} \\ &\qquad x_i \ge v(\{i\}), \quad \textstyle\sum_{i\in N} x_i = v(N), \end{aligned}$$

con $W_r = \{S \mid e(S,x) = \epsilon_r^*\}$. Ci si ferma quando tutte le coalizioni hanno l'eccesso fissato.

- Ogni programma ha $O(2^n)$ vincoli, uno per coalizione.
- Nel caso peggiore si fissa una coalizione per volta: fino a 2^n iterazioni.
- In generale è **molto poco pratico**.

Attenzione (quali coalizioni fissare). "Le coalizioni con eccesso pari a ϵ_k^*" va inteso come "le coalizioni il cui eccesso vale ϵ_k^* in *tutte* le soluzioni ottime del programma k". Una soluzione ottima qualsiasi può avere per caso altre coalizioni con eccesso ϵ_k^* che in un'altra soluzione ottima potrebbero scendere: fissarle porterebbe a un risultato sbagliato. Nell'implementazione del laboratorio lo si controlla con un programma lineare in più per ciascuna coalizione candidata (si prova ad abbassarne l'eccesso restando all'ottimo).

5.6.2 Il nucleolo può essere iniquo

Esempio (un gioco in cui conta solo A).

$$v(\emptyset) = v(\{A\}) = v(\{B\}) = v(\{C\}) = 0,$$
$$v(\{A,B\}) = v(\{A,C\}) = 750, \quad v(\{B,C\}) = 0, \quad v(\{A,B,C\}) = 750.$$

Il nucleolo è $\eta = (750, 0, 0)$: assegna tutto ad A, anche se almeno uno tra B e C è necessario per ottenere qualcosa. Anche qui stabilità non implica equità.

Attenzione (il vettore degli eccessi nelle slide). Le slide riportano $\theta(\eta) = (0,0,0,0,0,0)$. Gli eccessi di $\eta = (750,0,0)$ sono: $e(\{B\}) = e(\{C\}) = 0$, $e(\{A,B\}) = e(\{A,C\}) = 750 - 750 = 0$, $e(\{B,C\}) = 0$, ma $e(\{A\}) = 0 - 750 = -750$. Quindi $\theta(\eta) = (0,0,0,0,0,-750)$. Il nucleolo resta quello: il core di questo gioco contiene *solo* $(750,0,0)$ (da $x_A + x_B \ge 750$ e $x_A + x_B + x_C = 750$ segue $x_C = 0$, e lo stesso per x_B), e il nucleolo sta nel core.

5.7 Equità: il valore di Shapley

Idea: un esito equo dovrebbe ricompensare ogni giocatore in proporzione al **contributo** che porta agli altri. Se il giocatore i entra nella coalizione C, il suo **contributo marginale** è il surplus che crea:

$$v(C \cup \{i\}) - v(C).$$

Nel gioco precedente, per esempio, $v(\{A,B\}) - v(\{A\}) = 750$ e $v(\{A,B\}) - v(\{B\}) = 750$: il contributo di un giocatore dipende da chi c'è già. Un esito equo si ottiene premiando ciascuno con il suo contributo *medio*.

Definizione 5.10 (Valore di Shapley). Il valore di Shapley considera tutti gli ordini in cui la grande coalizione si può formare e dà a ciascuno la stessa importanza:

$$\phi(i,v) = \frac{1}{|N|!} \sum_{\pi \in \Pi_N} \Big(v\big(B(\pi,i) \cup \{i\}\big) - v\big(B(\pi,i)\big) \Big),$$

dove Π_N è l'insieme delle permutazioni di N e $B(\pi,i)$ l'insieme dei giocatori che precedono i nella permutazione π.

Esempio (Shapley nel gioco del gelato). Contributi marginali di A nei sei ordini:

Ordine	Contributo di A	
A,B,C	$v(\{A\}) - v(\emptyset)$	$= 0$
A,C,B	$v(\{A\}) - v(\emptyset)$	$= 0$
B,A,C	$v(\{A,B\}) - v(\{B\})$	$= 750$
B,C,A	$v(\{A,B,C\}) - v(\{B,C\})$	$= 1000$
C,A,B	$v(\{A,C\}) - v(\{C\})$	$= 750$
C,B,A	$v(\{A,B,C\}) - v(\{B,C\})$	$= 1000$

$$\phi(A) = \tfrac{1}{6}(0+0+750+1000+750+1000) = 583{,}\overline{3}.$$

Per B i contributi sono $750, 250, 0, 0, 250, 0$, quindi $\phi(B) = \frac{1250}{6} = 208{,}\overline{3}$, e per simmetria $\phi(C) = 208{,}\overline{3}$. Somma: 1000.

Esempio (Shapley nel gioco in cui conta solo A).

Ordine	contributo di A	contributo di B	contributo di C
A,B,C	0	750	0
A,C,B	0	0	750
B,A,C	750	0	0
B,C,A	750	0	0
C,A,B	750	0	0
C,B,A	750	0	0
Valore di Shapley	500	125	125

A differenza del nucleolo, B e C ricevono qualcosa: chi arriva subito dopo A è quello che "sblocca" il valore.

Approfondimento (equo, ma non stabile). $(500, 125, 125)$ non è nel core di questo gioco: il core contiene solo $(750, 0, 0)$, e infatti $x_A + x_B = 625 < 750 = v(\{A,B\})$. A e B da soli prenderebbero di più. Equità e stabilità sono richieste diverse e possono essere incompatibili. Nel gioco del gelato, invece, il valore di Shapley $(583{,}3;\ 208{,}3;\ 208{,}3)$ *è* nel core (rispetta $x_B, x_C \le 250$). Il risultato è coerente con l'esempio delle tre imprese della lezione 1, dove con valore 100 si otteneva $(66{,}67;\ 16{,}67;\ 16{,}67)$: è lo stesso gioco in scala.

5.7.1 Caratterizzazione assiomatica

Teorema 5.11 (Shapley). Il valore di Shapley è l'**unico** schema di ripartizione che soddisfa:

1. **efficienza**: $\phi_1 + \phi_2 + \cdots + \phi_n = v(N)$;
2. **giocatore nullo**: se $v(C \cup \{i\}) = v(C)$ per ogni C, allora $\phi_i = 0$;
3. **simmetria**: se i e j sono intercambiabili ($v(C \cup \{i\}) = v(C \cup \{j\})$ per ogni C che non li contiene), allora $\phi_i = \phi_j$;
4. **additività**: $\phi_i(G_1 + G_2) = \phi_i(G_1) + \phi_i(G_2)$.

Inoltre si può calcolare con la formula sui sottoinsiemi:

$$\phi(i, v) = \sum_{\substack{C \subseteq N \\ i \in C}} \frac{(|N| - |C|)!\,(|C| - 1)!}{|N|!} \big(v(C) - v(C \setminus \{i\})\big).$$

Osservazione (da dove viene il coefficiente). Negli ordini in cui i entra trovando già esattamente la coalizione $C \setminus \{i\}$, i $|C| - 1$ predecessori possono stare in $(|C| - 1)!$ ordini e i $|N| - |C|$ successori in $(|N| - |C|)!$. Il coefficiente è la frazione di permutazioni in cui i dà proprio il contributo $v(C) - v(C \setminus \{i\})$. Si passa da $n!$ permutazioni a 2^{n-1} coalizioni contenenti i: è la formula da usare nel codice.

Attenzione ("giocatore fittizio" e "giocatore nullo"). Le slide chiamano l'assioma 2 *dummy player* e chiedono $\phi_i = 0$. In molti testi il *dummy* è un giocatore che aggiunge sempre esattamente $v(\{i\})$, e riceve $\phi_i = v(\{i\})$; quello che riceve zero è il *null player*, che non aggiunge mai nulla. Con $v(\{i\}) = 0$ le due nozioni coincidono. Le slide scrivono anche "Shapely" invece di Shapley.

5.8 Laboratorio: Core e valore di Shapley

LAB

[CoalitionalGames_Frozensets_blank.ipynb, Lab_5-Coalitional_Game_Theory_Core_Shapley.pdf]

Obiettivo: rappresentare un gioco coalizionale in Python, verificare la superadditività, controllare se un esito sta nel core e calcolare il valore di Shapley. Il notebook contiene gli output attesi; tutti sono riprodotti. In più sono stati implementati e verificati la programmazione dinamica per la struttura di coalizione e il nucleolo.

5.8.1 Rappresentare la funzione caratteristica

Si usa un dizionario indicizzato da `frozenset`: i `frozenset` sono insiemi immutabili, quindi si possono usare come chiavi, e hanno tutte le operazioni degli insiemi (appartenenza, inclusione, unione, intersezione).

```python
from itertools import combinations
from math import factorial

v = {
    frozenset(['A']): 0, frozenset(['B']): 0, frozenset(['C']): 0,
    frozenset(['A', 'B']): 750, frozenset(['A', 'C']): 750,
    frozenset(['B', 'C']): 0,
    frozenset(['A', 'B', 'C']): 1000,
}

def powerset(players):   # tutte le coalizioni NON vuote: 2^n - 1
    return [frozenset(c) for k in range(1, len(players) + 1)
            for c in combinations(players, k)]
```

Attenzione (la coalizione vuota non c'è). Il dizionario del notebook non contiene $\emptyset$, e la `powerset` fornita lo esclude (il commento dice "2^{n-1} frozen sets", ma sono $2^n - 1$). Ogni formula che usa $v(\emptyset)$, come la formula di Shapley con $C = \{i\}$, deve trattarlo a parte: `v.get(S, 0)` invece di `v[S]`, altrimenti un `KeyError`.

5.8.2 Superadditività

```python
def is_superadditive(v):
    for C1 in v:
        for C2 in v:
            if not (C1 & C2):                       # disgiunte
                if v.get(C1 | C2, 0) < v[C1] + v[C2]:
                    return False
    return True
```

Il gioco del gelato dà `True`; il gioco del notebook con $v(A) = 6$, $v(B) = 12$, $v(A, B) = 12$ dà `False` (perché $12 < 6 + 12$). Entrambi come atteso.

5.8.3 Core

```python
def is_stable(x, v, tol=1e-9):
    return all(sum(x[i] for i in C) >= v[C] - tol for C in v)

def is_in_the_core(x, v, tol=1e-9):
    N = max(v)                                          # la grande coalizione
    return (all(xi >= -tol for xi in x.values())
            and abs(sum(x.values()) - v[N]) <= tol
            and is_stable(x, v, tol))
```

Esito	Nel core?	Perché
(333,3334; 333,3333; 333,3333)	no	$x_A + x_B \approx 666{,}7 < 750$
(500; 250; 250)	sì	tutti i vincoli rispettati
(1000; 0; 0)	sì	stabile, anche se iniquo

Attenzione (`max(v)` trova la grande coalizione, ma per un motivo sottile). Il notebook ricava la grande coalizione con `max(characteristic_function)`. Funziona, ma non perché confronti le dimensioni: per i `frozenset` l'operatore `>` significa "sovrainsieme proprio". `max` tiene il primo elemento e lo sostituisce con ogni elemento che lo contiene strettamente; la grande coalizione contiene tutte le altre, quindi appena la incontra non la lascia più. Se la grande coalizione mancasse dal dizionario, `max` restituirebbe una coalizione qualsiasi senza segnalare nulla.

Osservazione (perché non si calcola il core). Il notebook si limita a *verificare* se un esito sta nel core. Calcolarlo tutto richiederebbe di trovare i vertici del poliedro definito dai vincoli, con una sequenza di programmi lineari. Per sapere solo se il core è *vuoto* basta però un programma lineare: si cerca un x qualsiasi che soddisfi i vincoli. Applicato al gioco con $v(\{1,2\}) = 7$, $v(\{1,3\}) = 6$, $v(\{2,3\}) = 5$, $v(N) = 8$ il risolutore conferma che non esiste.

5.8.4 Valore di Shapley

```python
def shapley_value(player, v):
    N = max(v)
    n = len(N)
    phi = 0.0
    for C in v:
        if player in C:
            s = len(C)
            weight = factorial(n - s) * factorial(s - 1) / factorial(n)
            phi += weight * (v[C] - v.get(C - {player}, 0))
    return phi

def shapley(v):
    return {p: shapley_value(p, v) for p in max(v)}
```

Risultato sul gelato: $\{A : 583{,}33;\ B : 208{,}33;\ C : 208{,}33\}$, come atteso. Una seconda implementazione con le permutazioni dà gli stessi valori.

Sottoinsiemi o permutazioni. Il notebook raccomanda la formula sui sottoinsiemi: $O(2^n)$ invece di $O(n!)$. Su giochi casuali con n giocatori, tempi misurati per calcolare il valore di *tutti* i giocatori:

n	Sottoinsiemi	Permutazioni
6	0,1 ms	3,8 ms
8	0,8 ms	304 ms
9	1,9 ms	3 409 ms
10	5,5 ms	3 628 800 ordini, non eseguito

I valori calcolati con i due metodi coincidono ogni volta. Il codice sopra scorre tutte le coalizioni per ogni giocatore, quindi il costo totale è $n \cdot 2^n$: comunque esponenziale. Per giochi con molti giocatori si usano approssimazioni (per esempio campionando permutazioni a caso) o rappresentazioni compatte, come anticipa la fine del notebook.

> **Attenzione (“il valore di Shapley è individualmente razionale”).** Il notebook elenca tra le proprietà $\phi(i, v) \geq v(\{i\})$. Vale per i giochi *superadditivi*: lì ogni contributo marginale è $v(C \cup \{i\}) - v(C) \geq v(\{i\})$, e quindi lo è anche la media. Senza superadditività può fallire. Il notebook stesso ricorda subito dopo l’altro limite: il valore di Shapley non è necessariamente nel core.

5.8.5 In più: struttura di coalizione e nucleolo

Programmazione dinamica. Implementazione diretta dell’equazione della teoria, coalizione per coalizione in ordine di dimensione:

```python
def coalition_structure_dp(v):
    N = max(v)
    best, struct = {}, {}
    for C in sorted(powerset(sorted(N)), key=len):
        best[C], struct[C] = v[C], [C]
        members = sorted(C)
        for k in range(1, len(C)):
            for part in combinations(members, k):
                C1 = frozenset(part); C2 = C - C1
                if min(C1) != min(C):          # ogni divisione una volta sola
                    continue
                if best[C1] + best[C2] > best[C]:
                    best[C] = best[C1] + best[C2]
                    struct[C] = struct[C1] + struct[C2]
    return best, struct
```

Sul gioco dell’esempio restituisce esattamente la tabella corretta: $v^*(\{B, C\}) = 400$ e $v^*(N) = 1050$ con $CS^* = \{\{A, C\}, \{B\}\}$.

Nucleolo. Implementato con l’algoritmo di Kopelowitz e `linprog`, fissando a ogni passo solo le coalizioni il cui eccesso non si può abbassare. Risultati:

Gioco	Nucleolo	Vettore degli eccessi
Simmetrico ($v(ij) = 1$, $v(N) = 3$)	$(1, 1, 1)$	$(-1, -1, -1, -1, -1, -1)$
Conta solo A ($v(N) = 750$)	$(750, 0, 0)$	$(0, 0, 0, 0, 0, -750)$
Gelato ($v(N) = 1000$)	$(750, 125, 125)$	$(-125, -125, -125, -125, -250, -750)$

Nel gelato nucleolo e Shapley sono diversi: $(750; 125; 125)$ contro $(583{,}3; 208{,}3; 208{,}3)$. Entrambi stanno nel core, ma il nucleolo "bilancia" le insoddisfazioni (le coppie $\{A, B\}$ e $\{A, C\}$ e i singoli B e C hanno tutti eccesso -125), mentre Shapley premia i contributi marginali.

Riepilogo della lezione

- Gioco in funzione caratteristica (N, v): $v(C)$ è quanto guadagna la coalizione C; utilità trasferibile.
- Struttura di coalizione: partizione di N che massimizza $\sum v(C)$. Le partizioni sono il numero di Bell; la programmazione dinamica la trova in $O(3^n)$.
- Superadditività: fondersi non fa mai perdere, quindi si forma la grande coalizione e un esito è un vettore x con $\sum x_i = v(N)$.
- **Core**: esiti in cui ogni coalizione riceve almeno $v(C)$. Può essere vuoto e può contenere esiti iniqui.
- **Nucleolo**: minimizza lessicograficamente gli eccessi $e(S, x) = v(S) - x(S)$; esiste, è unico, sta nel core se questo non è vuoto; si calcola con una sequenza di programmi lineari (Kopelowitz). Può essere iniquo.
- **Shapley**: contributo marginale medio su tutti gli ordini di ingresso; unico a soddisfare efficienza, giocatore nullo, simmetria, additività. Equo ma non necessariamente stabile.
- **Lab**: `frozenset` come chiavi, superadditività, verifica del core, Shapley sui sottoinsiemi ($n \cdot 2^n$ contro $n!$); in più programmazione dinamica e nucleolo.

LEZIONE 6

Giochi coalizionali: aspetti computazionali

TEORIA

La lezione 5 si chiudeva con un problema: la funzione caratteristica ha 2^n valori e il valore di Shapley li usa tutti. Questa lezione mostra tre modi per scalare a molti giocatori. Il percorso:

1. il problema computazionale;
2. una classe speciale di giochi: il **gioco dell'aeroporto**;
3. i **giochi su sottografo indotto** (ISG);
4. una rappresentazione compatta universale: le **MC-Nets**;
5. l'**approssimazione Monte Carlo** del valore di Shapley.

6.1 Il problema computazionale

Un gioco coalizionale è rappresentato da una funzione caratteristica $v : 2^N \to \mathbb{R}$, che ha un valore per ogni sottoinsieme di giocatori: la sua dimensione è **esponenziale** nel numero di giocatori. Già con tre giocatori (il gioco del gelato della lezione 5) servono otto numeri:

$$v(\emptyset) = v(\{A\}) = v(\{B\}) = v(\{C\}) = 0,$$
$$v(\{A,B\}) = v(\{A,C\}) = 750, \quad v(\{B,C\}) = 0, \quad v(\{A,B,C\}) = 1000.$$

Di conseguenza anche i concetti di soluzione richiedono un numero esponenziale di calcoli. La formula di Shapley somma su tutte le permutazioni:

$$\phi(i,v) = \frac{1}{|N|!} \sum_{\pi \in \Pi_N} \Big(v\big(B(\pi,i) \cup \{i\}\big) - v\big(B(\pi,i)\big) \Big).$$

La versione sui sottoinsiemi scende a 2^{n-1} termini per giocatore, ma resta esponenziale.

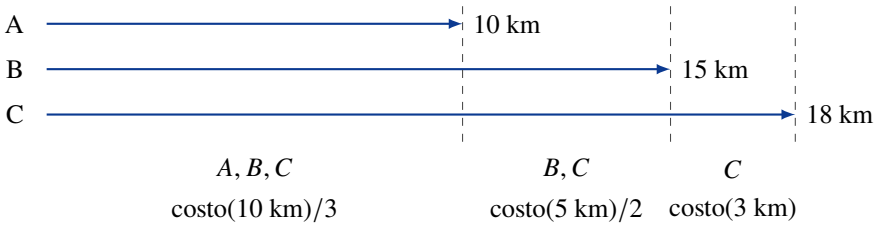
Tre strade.

Strada	Che cosa si ottiene
Classi speciali di giochi	formule chiuse per il valore di Shapley, calcolabili in tempo polinomiale (gioco dell'aeroporto)
Rappresentazione compatta	funzione caratteristica in spazio polinomiale e valore di Shapley in tempo polinomiale (ISG, MC-Nets)
Approssimazione Monte Carlo	tempo polinomiale per *qualunque* gioco, con garanzie sull'errore

6.2 Il gioco dell'aeroporto

Ci sono n compagnie aeree. Ognuna ha bisogno di una pista di una certa lunghezza per i propri aerei. Una pista può essere condivisa, quindi conviene costruirne *una sola*, abbastanza lunga per tutti, e dividerne il costo. Come lo si divide in modo equo?

Intuizione. La compagnia A ha bisogno di 10 km, B di 15 km, C di 18 km. Si costruisce una pista di 18 km, ma i suoi tratti servono a gruppi diversi:



I primi 10 km servono a tutti e tre, quindi il loro costo si divide per 3. I 5 km successivi servono solo a B e C, che se li dividono in due. Gli ultimi 3 km servono solo a C, che li paga da solo.

Definizione 6.1 (Gioco dell'aeroporto). I giocatori $N = \{1, \dots, n\}$ hanno costi ordinati $c_1 < c_2 < \cdots < c_n$, dove c_i è il costo della pista che serve al giocatore i. La funzione caratteristica è

$$v(S) = \max_{i \in S} c_i \qquad \forall S \subseteq N :$$

una coalizione paga la pista del suo membro più esigente. È un gioco di **costo**: $v(S)$ è una spesa da ripartire, non un guadagno.

Teorema 6.2 (Littlechild e Owen, 1973). Il valore di Shapley del gioco dell'aeroporto è

$$\phi_i = \sum_{j=1}^{i} \frac{c_j - c_{j-1}}{n - j + 1} \qquad \forall i \in N, \qquad c_0 = 0.$$

Lettura della formula. Il tratto di pista tra c_{j-1} e c_j costa $c_j - c_{j-1}$ e serve ai giocatori $j, j+1, \dots, n$, che sono $n - j + 1$: se lo dividono in parti uguali. Il giocatore i paga la sua quota di ogni tratto fino al proprio, cioè per $j = 1, \dots, i$.

Dimostrazione. Si definisce $P_j = \{j, \dots, n\}$, l'insieme dei giocatori che hanno bisogno del tratto j, e per ogni j il gioco

$$v_j(S) = \begin{cases} 0 & \text{se } S \cap P_j = \emptyset, \\ c_j - c_{j-1} & \text{se } S \cap P_j \neq \emptyset. \end{cases}$$

Una coalizione S con membro più esigente k interseca esattamente $P_1, \dots, P_k$, quindi $\sum_j v_j(S) = \sum_{j=1}^{k} (c_j - c_{j-1}) = c_k = v(S)$: il gioco dell'aeroporto è la somma dei giochi v_j. In v_j i giocatori fuori da P_j sono **nulli** (non cambiano mai il valore), quelli dentro P_j sono **simmetrici**; per efficienza si dividono

in parti uguali $c_j - c_{j-1}$:

$$\phi_i(v_j) = \begin{cases} \dfrac{c_j - c_{j-1}}{n-j+1} & \text{se } i \in P_j \text{ (cioè } j \le i\text{)}, \\ 0 & \text{altrimenti.} \end{cases}$$

Per **additività**, $\phi_i(v) = \sum_j \phi_i(v_j) = \sum_{j=1}^{i} \frac{c_j - c_{j-1}}{n-j+1}$. □

Osservazione (gli assiomi come strumento di calcolo). La dimostrazione non tocca mai le $n!$ permutazioni: scompone il gioco in pezzi semplici e usa i quattro assiomi della lezione 5 (nullo, simmetria, efficienza, additività). È lo stesso trucco che rende veloci ISG e MC-Nets: *additività* vuol dire che basta calcolare Shapley su ogni pezzo e sommare.

Esempio (quattro compagnie, costi $(8, 11, 13, 18)$**).**

	Tratto 1	Tratto 2	Tratto 3	Tratto 4	**Shapley**
Costo marginale	8	3	2	5	
Quota di P1	2				**2**
Quota di P2	2	1			**3**
Quota di P3	2	1	1		**4**
Quota di P4	2	1	1	5	**9**

Il primo tratto (8) si divide fra quattro, il secondo ($11 - 8 = 3$) fra tre, il terzo ($13 - 11 = 2$) fra due, l'ultimo ($18 - 13 = 5$) lo paga solo P4. Totale $2 + 3 + 4 + 9 = 18 = v(N)$.

Complessità. La formula come scritta richiede $O(n^2)$ operazioni (una somma di i termini per ogni giocatore). Ma ogni riga della tabella è la riga precedente più un termine: con una somma cumulativa si scende a $O(n)$, più $O(n \log n)$ se i costi vanno ordinati. Contro $O(n \cdot 2^n)$ della formula generale.

Applicazioni. Car sharing e ride sharing, capacità di banda in una rete, una condotta d'acqua o un oleodotto, spazio di archiviazione in cloud con utenti che chiedono quote diverse, livelli di abbonamento a un software.

Osservazione (ricetta per riconoscere un gioco dell'aeroporto). Servono quattro ingredienti:

- una sola struttura o risorsa **condivisa**;
- una struttura **annidata**: chi chiede di più ha bisogno anche di tutto ciò che serve a chi chiede di meno;
- il costo **cresce** con la richiesta;
- l'obiettivo è dividere i costi in modo **equo**.

Approfondimento (il valore di Shapley dell'aeroporto è nel core). Nel gioco dell'aeroporto una coalizione S con membro più esigente k paga, con Shapley, $\sum_{i \in S} \phi_i \le$ la somma delle quote di tutti i giocatori fino a k, cioè al più $c_k = v(S)$. Nessun gruppo pagherebbe meno costruendo la pista da solo: la ripartizione è anche *stabile*. In un gioco di costo la condizione del core ha il verso

opposto rispetto alla lezione 5: $\sum_{i \in S} x_i \leq v(S)$. Il gioco è infatti *concavo* (per i costi), e per questi giochi Shapley sta sempre nel core.

6.3 Giochi su sottografo indotto

Definizione 6.3 (Induced subgraph game, ISG). I giocatori sono i nodi di un grafo pesato. Un arco $\{i, j\}$ con peso w_{ij} rappresenta il valore della coppia; un cappio con peso w_{ii} il valore del giocatore da solo. Il valore di una coalizione è la somma dei pesi degli archi che hanno *entrambi* gli estremi nella coalizione, cioè il peso del sottografo indotto da C:

$$v(C) = \sum_{\{i,j\} \subseteq C} w_{ij}.$$

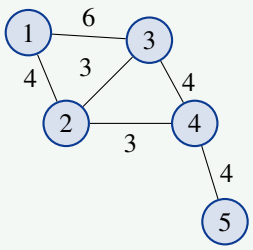
Teorema 6.4 (Shapley negli ISG).

$$\phi_i = w_{ii} + \frac{1}{2} \sum_{j \in \Gamma(i)} w_{ij},$$

dove $\Gamma(i)$ è l'insieme dei vicini di i.

Perché. Di nuovo additività: ogni arco $\{i, j\}$ è un gioco a sé, in cui una coalizione vale w_{ij} se contiene sia i sia j, altrimenti 0. Lì i e j sono simmetrici e tutti gli altri sono nulli, quindi si dividono w_{ij} a metà. Il cappio è un gioco in cui conta solo i, che si prende tutto w_{ii}.

Esempio (ISG con cinque giocatori).



Valori di alcune coalizioni:

$$v(\{1,2,4\}) = 4 + 3 = 7,$$
$$v(\{1,2,5\}) = 4,$$
$$v(\{1,2,3,4\}) = 4 + 6 + 3 + 4 + 3 = 20.$$

Valori di Shapley (nessun cappio):

$$\phi_1 = \tfrac{1}{2}(6+4) = 5, \qquad \phi_2 = \tfrac{1}{2}(4+3+3) = 5,$$
$$\phi_3 = \tfrac{1}{2}(6+3+4) = 6{,}5, \qquad \phi_4 = \tfrac{1}{2}(4+3+4) = 5{,}5,$$
$$\phi_5 = \tfrac{1}{2} \cdot 4 = 2.$$

Controllo di efficienza: $5 + 5 + 6{,}5 + 5{,}5 + 2 = 24$, la somma di tutti i pesi.

Applicazioni. Gli ISG modellano reti sociali e relazioni fra giocatori: pesi **positivi** per le sinergie, pesi **negativi** per i conflitti. Sono anche un modello naturale per la formazione di coalizioni, che si possono cercare con tecniche di clustering di grafi (modularità, partizionamento spettrale con il vettore di Fiedler, conduttanza, Infomap, Walktrap, modelli a blocchi stocastici).

Attenzione (gli ISG non rappresentano tutti i giochi). Il valore di una coalizione è determinato dalle sole coppie: non c'è modo di esprimere un effetto che compare solo quando ci sono *tre* giocatori

insieme. Esempio dal notebook: il gioco dell'aeroporto con costi $(5, 7, 10)$. Singoli e coppie fissano i pesi ($w_{AA} = 5$, $w_{BB} = 7$, $w_{CC} = 10$, $w_{AB} = -5$, $w_{AC} = -5$, $w_{BC} = -7$), ma allora l'ISG darebbe $v(\{A, B, C\}) = 5$, mentre il valore vero è 10.

6.4 Rappresentazioni compatte

Una rappresentazione compatta riduce la memoria occupata dalla funzione caratteristica, di solito usando una struttura di rete. Il prezzo:

- il valore di una coalizione non si legge più in $O(1)$ da una tabella, ma si *calcola* in tempo polinomiale (sommando pesi);
- in cambio, sfruttando l'additività del valore di Shapley, anche Shapley si calcola in tempo polinomiale.

Gli ISG sono già una rappresentazione compatta: n nodi e al più $n(n+1)/2$ archi (cappi compresi) invece di 2^n valori. Le MC-Nets la generalizzano fino a rappresentare qualunque gioco.

6.4.1 Marginal Contribution Nets

Definizione 6.5 (MC-Net). Una MC-Net rappresenta la funzione caratteristica come un insieme di **regole**

$$\text{pattern} \longrightarrow \text{valore},$$

dove il pattern è una formula booleana sui giocatori e il valore è il **contributo marginale** del pattern. Nella forma base il pattern è una congiunzione $\{a \wedge b \wedge \cdots \wedge c\}$, e la regola **si applica** a S se S contiene tutti i giocatori del pattern. Il valore di una coalizione è la somma dei valori delle regole che si applicano:

$$v(S) = \sum_{\varphi \to x \,:\, \varphi \subseteq S} x.$$

Se i valori possono essere sia positivi sia negativi, già le congiunzioni bastano a rappresentare *qualunque* gioco.

Esempio (due regole).

$$\{a \wedge b\} \to 5, \qquad \{b\} \to 2.$$

- $v(\{a\}) = 0$: nessuna regola si applica ($\{a\}$ non contiene b);
- $v(\{b\}) = 2$: si applica solo la seconda;
- $v(\{a, b\}) = 5 + 2 = 7$: si applicano entrambe.

ISG come MC-Net. Ogni arco $\{i, j\}$ di peso w diventa la regola $\{i \wedge j\} \to w$ e ogni cappio diventa $\{i\} \to w_{ii}$. Il grafo dell'esempio precedente diventa

$$\{1 \wedge 3\} \to 6, \quad \{1 \wedge 2\} \to 4, \quad \{2 \wedge 3\} \to 3, \quad \{2 \wedge 4\} \to 3, \quad \{3 \wedge 4\} \to 4, \quad \{4 \wedge 5\} \to 4.$$

Una MC-Net è in sostanza un **ipergrafo**: ogni regola è un iperarco che collega tutti i giocatori del pattern, pesato con il suo valore.

Attenzione (errore nella slide "Induced subgraph games as MC-Nets"). La slide scrive $\{4 \wedge 5\} \to 5$, ma nel grafo l'arco $\{4,5\}$ pesa 4. Con la regola della slide verrebbero $\phi_4 = 6$, $\phi_5 = 2{,}5$ e $v(N) = 25$, invece di 5,5, 2 e 24 (verificato al calcolatore).

Teorema 6.6 (Shapley nelle MC-Nets con congiunzioni).

$$\phi_i = \sum_{\varphi \to x \,:\, i \in \varphi} \frac{x}{|\varphi|}.$$

Ogni regola è un gioco a sé in cui i giocatori del pattern sono simmetrici e gli altri nulli: il valore x si divide in parti uguali fra i $|\varphi|$ giocatori del pattern. Poi additività. Nell'esempio a due regole:

$$\phi_a = \frac{5}{2} = 2{,}5, \qquad \phi_b = \frac{5}{2} + \frac{2}{1} = 4{,}5,$$

e $2{,}5 + 4{,}5 = 7 = v(\{a,b\})$.

Approfondimento (i valori delle regole sono i dividendi di Harsanyi). Per un gioco qualsiasi i valori delle regole con congiunzioni si ricavano con l'inclusione-esclusione (trasformata di Möbius):

$$d(S) = \sum_{T \subseteq S} (-1)^{|S|-|T|}\, v(T),$$

e la MC-Net $\{S \to d(S)\}$ rappresenta esattamente v. È per questo che le congiunzioni con valori di segno qualunque sono complete. La formula di Shapley qui sopra dice allora che ogni giocatore riceve una quota uguale di ogni dividendo delle coalizioni a cui appartiene.
Ma "completa" non vuol dire "compatta". Per il gioco dell'aeroporto il calcolo dà $d(S) = (-1)^{|S|+1} \min_{i \in S} c_i$, mai zero: servono tutte le $2^n - 1$ regole (verificato fino a $n = 10$: 1023 regole). Con costi $(5, 7, 10)$ si ottiene proprio la MC-Net del notebook, con valori $5, 7, 10, -5, -5, -7, +5$.

6.4.2 MC-Nets con formule generali

Le regole possono contenere formule logiche arbitrarie, con letterali **positivi** e **negativi**. Una regola $\varphi \to x$ si applica a S se S **soddisfa** φ, scritto $S \models \varphi$: assegnando "vero" ai giocatori in S e "falso" a tutti gli altri, la formula risulta vera. Il valore di una coalizione è

$$v(S) = \sum_{\varphi \to x \,:\, S \models \varphi} x.$$

Esempio (MC-Net con un letterale negativo).

$$x_1 \wedge x_2 \to 5 \qquad x_2 \to 2 \qquad x_3 \to 4 \qquad x_2 \wedge \neg x_3 \to -2$$

S	regole applicate	$v(S)$
$\{x_1\}$	nessuna	0
$\{x_2\}$	2 e 4	$2-2=0$
$\{x_3\}$	3	4
$\{x_1,x_2\}$	1, 2, 4	$5+2-2=5$
$\{x_1,x_3\}$	3	4
$\{x_2,x_3\}$	2, 3	$2+4=6$
$\{x_1,x_2,x_3\}$	1, 2, 3	$5+2+4=11$

La regola 4 si applica solo quando c'è x_2 *e manca* x_3.

Teorema 6.7 (Shapley nelle MC-Nets generali). Per una regola $\varphi \to x$ con p letterali positivi e n letterali negativi, ogni giocatore i che compare in forma positiva e ogni giocatore j che compare in forma negativa ricevono

$$\phi_i = \frac{(p-1)!\,n!}{(p+n)!}\,x = \frac{x}{p\binom{p+n}{n}}, \qquad \phi_j = \frac{p!\,(n-1)!}{(p+n)!}\,(-x) = -\frac{x}{n\binom{p+n}{p}}.$$

Gli altri giocatori sono nulli per quella regola. Per additività il valore di Shapley è la somma su tutte le regole.

Da dove vengono i coefficienti. Si guardano solo i $p+n$ giocatori della regola, perché gli altri non contano. Un giocatore positivo i "accende" la regola se arriva *dopo* tutti gli altri positivi e *prima* di tutti i negativi; la frazione di ordini in cui succede è $\frac{(p-1)!\,n!}{(p+n)!}$. Un negativo j "spegne" la regola (contributo $-x$) se arriva dopo tutti i positivi e prima degli altri negativi: frazione $\frac{p!\,(n-1)!}{(p+n)!}$. Senza negativi ($n=0$) si ritrova x/p.

Esempio (Shapley nella MC-Net con letterale negativo).

$$\phi_{x_1} = \tfrac{1!\,0!}{2!}\cdot 5 = 2{,}5 \qquad \text{(solo regola 1)}$$
$$\phi_{x_2} = \tfrac{1!\,0!}{2!}\cdot 5 + 2 + \tfrac{1!\,0!}{2!}\cdot(-2) = 2{,}5+2-1 = 3{,}5 \qquad \text{(regole 1, 2, 4)}$$
$$\phi_{x_3} = 4 + \tfrac{1!\,0!}{2!}\cdot\big(-(-2)\big) = 4+1 = 5 \qquad \text{(regola 3; negativo nella 4)}$$

Somma $2{,}5+3{,}5+5 = 11 = v(N)$. Il calcolo esponenziale sulla funzione caratteristica della tabella dà gli stessi valori.

Approfondimento (l'aeroporto con n regole). Con le formule generali il gioco dell'aeroporto torna compatto. La dimostrazione del teorema di Littlechild e Owen è già una MC-Net: per ogni tratto j la regola

$$(x_j \vee x_{j+1} \vee \cdots \vee x_n) \to c_j - c_{j-1}$$

("almeno uno di quelli che hanno bisogno del tratto j"). Sono n regole invece di 2^n-1; verificato al calcolatore che riproducono v. La scelta del linguaggio delle formule decide quanto è compatta la rappresentazione.

Riassunto MC-Nets. Rappresentazione compatta e **universale**; il valore di una coalizione si calcola in modo efficiente; il valore di Shapley si calcola in tempo **lineare** nella dimensione dell'insieme delle regole.

6.5 Approssimazione Monte Carlo del valore di Shapley

L'idea Monte Carlo. Se si vuole conoscere una probabilità $P(x)$ ma la si sa solo campionare, la si approssima con le frequenze: si genera un campione D di dimensione M da $P(X)$ e si stima

$$P_D(X = x) = \frac{M_{X=x}}{M},$$

dove $M_{X=x}$ è il numero di volte in cui è uscito x.

Applicata a Shapley. Il valore di Shapley è una *media* del contributo marginale su tutte le $n!$ permutazioni, cioè il valore atteso del contributo marginale di i quando la permutazione è scelta uniformemente a caso. Invece di sommare su tutte, se ne campionano m:

$$\tilde{\phi}(i, v) = \frac{1}{m} \sum_{\pi \in \mathcal{P}} \Big(v\big(B(\pi, i) \cup \{i\}\big) - v\big(B(\pi, i)\big) \Big),$$

dove $\mathcal{P} \subset \Pi_N$ è un insieme di m permutazioni estratte a caso.

```
# Input: v funzione caratteristica; m numero di campioni
def MC_Shapley(v, m):
    φ̃_i = 0 for each i in N
    for k = 1 ... m:
        π_k = random permutation of N
        for i = 1 ... n:
            sv = v(B(π_k, i) ∪ {i}) - v(B(π_k, i))
            φ̃_i += sv
    for i = 1 ... n:
        φ̃_i = φ̃_i / m
    return φ̃_1, ..., φ̃_n
```

> **Attenzione (indice nello pseudocodice delle slide).** Nell'ultimo ciclo le slide scrivono "**for** $k = 1 \dots n$" ma poi dividono $\tilde{\phi}_i$: l'indice giusto è i, come sopra.

Proprietà.

- **Calcolo efficiente**: m permutazioni, n valutazioni di v ciascuna; se v si calcola in tempo polinomiale, tutto è polinomiale.
- L'**errore** diminuisce come $1/\sqrt{m}$: per dimezzarlo servono quattro volte tanti campioni.
- Molto efficace in pratica; si applica a *qualunque* gioco e a qualunque rappresentazione.
- Esistono tecniche di campionamento che riducono la varianza più in fretta: *antithetic sampling*, estensione multilineare, metodi kernel (riferimento: Mitchell et al., 2022, *Sampling Permutations for Shapley Value Estimation*, JMLR).

Approfondimento (perché $1/\sqrt{m}$, e una garanzia esplicita). Ogni permutazione dà un contributo marginale X_k indipendente, con media esatta ϕ_i: la stima è *non distorta*. La sua varianza è σ_i^2/m, quindi l'errore tipico (deviazione standard) è $\sigma_i/\sqrt{m}$. Se i contributi marginali stanno in un intervallo di ampiezza r, la disuguaglianza di Hoeffding dà

$$P\big(|\tilde{\phi}_i - \phi_i| \geq \varepsilon\big) \leq 2\exp\left(-\frac{2m\varepsilon^2}{r^2}\right),$$

cioè con $m \geq \frac{r^2}{2\varepsilon^2} \ln \frac{2}{\delta}$ campioni l'errore è sotto ε con probabilità almeno $1-\delta$. È la "garanzia sulla soluzione" promessa all'inizio della lezione; le slide dicono invece che non c'è modo di fissare m esattamente e che in pratica $m \in [1000, 10000]$ funziona bene.

6.6 Riepilogo delle tecniche

Tecnica	Pro	Contro
Classi speciali (aeroporto)	rappresentazione compatta; Shapley veloce	espressività limitata
Monte Carlo	Shapley veloce; proprietà di convergenza; si applica a qualunque gioco e rappresentazione	non esatto; non c'è un modo esatto di scegliere m (in pratica 1000–10000)
ISG	rappresenta molti giochi; Shapley veloce; poca memoria	non completo: non rappresenta qualunque gioco
MC-Nets	completo; Shapley veloce; di solito poca memoria	memoria $O(2^n)$ nel caso peggiore; sposta il lavoro sul calcolo dei contributi marginali, che vanno precalcolati

6.7 Laboratorio: Aeroporto, Monte Carlo, ISG e MC-Nets

LAB

[CoalitionalGamesComputationalIssues_Blank.ipynb, Lab06-Coalitional_Game_Theory_Computational_Issues.pdf]

Obiettivo: implementare le tre strade della teoria e verificare ogni volta che il risultato coincide con la formula esponenziale della lezione 5. Il notebook è vuoto (non ci sono output attesi): tutte le soluzioni qui sotto sono state scritte ed eseguite, e i numeri riportati sono misurati. Le slide del lab sono un sottoinsieme di quelle della lezione.

Il notebook riparte dal codice della lezione 5, con una differenza: ora `powerset` aggiunge anche `frozenset()` con valore 0, quindi la formula di Shapley può leggere `v[C - {i}]` senza `KeyError` (il problema segnalato nella lezione 5).

6.7.1 Il gioco dell'aeroporto

Il notebook racconta l'origine del problema (Littlechild e Owen, 1973) con un costo di €1000 al km e piste da 8, 11, 13, 18 km. La funzione caratteristica è già fornita:

```python
def airport_game_characteristic_function(cost_list):
    player_list = [str(i) for i in range(len(cost_list))]
    return {subset: (max([cost_list[int(i)] for i in subset])
                     if subset != frozenset() else 0)
            for subset in powerset(player_list)}
```

Da scrivere: Shapley con la formula dedicata. La versione lineare accumula la quota del tratto corrente:

```python
def airport_game_shapley_value(cost_list):
    c = sorted(cost_list)
    n = len(c)
    phi, acc, prev = [], 0.0, 0
    for j, cj in enumerate(c):          # restano n - j giocatori
        acc += (cj - prev) / (n - j)
        phi.append(acc)
        prev = cj
    return phi
```

Sui costi $(8000, 11000, 13000, 18000)$ restituisce $(2000, 3000, 4000, 9000)$, identico alla formula esponenziale `shapley(v)`.

> **Attenzione (un "/2" di troppo nel notebook).** Nel calcolo passo per passo il notebook scrive per il giocatore 4 "(€18000 − €13000)/2 = €5000". L'ultimo tratto lo paga solo lui: si divide per 1, e infatti il risultato €5000 è giusto.

Formula dedicata contro formula generale. Giochi casuali con costi distinti fra 1 e 99:

n	Coalizioni	Costruire v	Shapley generale	Shapley aeroporto
12	4 096	0,01 s	0,03 s	10 µs
16	65 536	0,16 s	0,79 s	19 µs
20	1 048 576	3,7 s	19,3 s	16 µs

I valori coincidono sempre. Con $n = 20$ il vettore dei costi occupa 216 byte, il dizionario 42 MB secondo `sys.getsizeof`.

> **Attenzione (`sys.getsizeof` misura solo il contenitore).** `getsizeof` non conta gli oggetti contenuti: per il dizionario misura la tabella hash ma non il milione di `frozenset` che fanno da chiavi, quindi la memoria vera è molto più grande di 42 MB. Lo stesso vale più avanti per il grafo: `getsizeof(G)` restituisce 48 byte qualunque sia il grafo. Il confronto del notebook va preso come indicativo.

6.7.2 Approssimazione Monte Carlo

Prima versione, ingenua: si parte dalla funzione caratteristica completa.

```python
def permutation_sampling(player_list, m):
    return [random.sample(player_list, len(player_list)) for _ in range(m)]

def predecessors(player, player_list, permutation):
    return frozenset(permutation[:permutation.index(player)])

def shapley_montecarlo(v, m):
    players = list(max(v))
    phi = {p: 0.0 for p in players}
    for perm in permutation_sampling(players, m):
        for p in players:
            B = predecessors(p, players, perm)
            phi[p] += v[B | {p}] - v[B]
    return {p: phi[p] / m for p in players}
```

Sull'aeroporto con 20 giocatori e $m = 1000$ impiega 0,09 s (contro 19 s dell'esatto), con un errore medio percentuale (MAPE) del 9,3%.

> **Attenzione (il test del notebook ordina le stime).** La cella di test confronta le stime *ordinate per valore* con i valori esatti. Funziona solo perché nell'aeroporto, con costi distinti, ϕ_i cresce con i; ma ordinando si riassegnano le stime ai giocatori sbagliati quando due sono vicine, e l'errore risulta *più basso* del vero: sullo stesso campione 6,8% ordinando, 9,3% confrontando giocatore per giocatore. Il confronto corretto è per nome del giocatore.

Versione migliore. Il notebook chiede di non passare la funzione caratteristica completa (2^n valori: già costruirla costa più che calcolare Shapley esatto con la formula dedicata). Nell'aeroporto i contributi marginali di una permutazione si calcolano in un'unica passata: chi entra paga la differenza fra il proprio costo e il massimo raggiunto finora.

```python
def airport_mc_fast(cost_list, m):
    n = len(cost_list)
```

```
    phi = [0.0] * n
    idx = list(range(n))
    for _ in range(m):
        random.shuffle(idx)
        cur = 0                          # pista costruita finora
        for i in idx:
            if cost_list[i] > cur:
                phi[i] += cost_list[i] - cur
                cur = cost_list[i]
    return [x / m for x in phi]
```

Costo $O(m \cdot n)$, senza mai costruire v.

Esperimento facoltativo: errore contro m. MAPE medio su 20 ripetizioni, costi distinti fra 1 e 999:

m	MAPE ($n = 20$)	MAPE$\cdot\sqrt{m}$	MAPE ($n = 100$)	Tempo ($n = 100$)
10	91,1%	2,88	152,9%	0,3 ms
100	27,0%	2,70	58,9%	2,8 ms
1 000	8,0%	2,52	19,1%	33 ms
10 000	2,8%	2,84	6,2%	323 ms

La pendenza in scala log-log è $-0{,}505$ per $n = 20$ e $-0{,}467$ per $n = 100$: l'errore scende come $1/\sqrt{m}$, come dice la teoria (MAPE$\cdot\sqrt{m}$ resta circa costante). Il grafico della slide "Error reduces as sample increases" mostra la stessa retta.

> **Osservazione (l'errore relativo pesa sui giocatori piccoli).** Con $n = 20$ e $m = 1000$ il giocatore con la pista più corta ha un errore relativo del 24%, quello con la pista più lunga dell'1,1%. Il giocatore più piccolo paga poco (contribuisce solo quando arriva per primo, cioè in $1/n$ delle permutazioni), quindi il suo contributo è quasi sempre zero e raramente grande: varianza alta rispetto alla media. Per questo con $n = 100$ il MAPE è più alto a parità di m: ci sono più giocatori con quote piccole.

6.7.3 Giochi su sottografo indotto

Il notebook usa `networkx`: un grafo casuale di Erdős–Rényi con 10 nodi e 25 archi, più un cappio per nodo, con pesi interi casuali fra 1 e 100.

```
def ISG_coalition_value(graph, coalition):
    return graph.subgraph(coalition).size(weight='weight')

def ISG_shapley_value(graph):
    phi = {}
    for i in graph.nodes():
        w_ii = graph.edges[i, i]['weight'] if graph.has_edge(i, i) else 0
        phi[i] = w_ii + 0.5 * sum(graph.edges[i, j]['weight']
                                  for j in graph.neighbors(i) if j != i)
    return phi
```

Il valore del sottografo indotto è già in `networkx`: `subgraph` tiene solo gli archi interni alla coalizione (cappi compresi) e `size(weight=...)` ne somma i pesi. Il confronto con la funzione caratteristica completa (1024 coalizioni) dà valori identici; la formula ISG impiega 25 µs, quella esponenziale 9 ms. Sull'esempio della teoria si ritrovano $v(\{1,2,4\}) = 7$, $v(\{1,2,3,4\}) = 20$ e $\phi = (5,5,6{,}5,5{,}5,2)$.

> **Attenzione (con i cappi, `neighbors(i)` contiene i stesso).** In `networkx` un nodo con un cappio è vicino di se stesso. Scrivendo la formula alla lettera, $w_{ii} + \frac{1}{2}\sum_{j \in \texttt{neighbors}(i)} w_{ij}$, il cappio viene contato una volta e mezza. Sul grafo del lab la somma dei valori ingenui è 1920,5 invece di $v(N) = 1633$: l'efficienza fallisce, e la cella di confronto del notebook non stampa il messaggio "The Shapley values are the same!". Serve il filtro `if j != i`.

> **Attenzione (quanti archi può avere il grafo).** Un commento del notebook dice che un grafo non orientato con N nodi ha al più $N(N+1)/2$ archi senza cappi, e che con 10 nodi e i cappi si arriva a 65. I conti giusti sono $N(N-1)/2 = 45$ senza cappi e $N(N+1)/2 = 55$ con i cappi. Il conteggio successivo "35 archi" (25 più 10 cappi) è invece corretto.

6.7.4 MC-Nets

Il notebook implementa le MC-Nets nella forma base: una lista di coppie (pattern, valore), con il pattern come insieme di giocatori in congiunzione. L'esempio è il gioco dell'aeroporto con costi $(5, 7, 10)$, che come ISG non si può rappresentare:

```python
MC_net = [({"A"}, 5), ({"B"}, 7), ({"C"}, 10),
          ({"A", "B"}, -5), ({"A", "C"}, -5), ({"B", "C"}, -7),
          ({"A", "B", "C"}, 5)]

def MC_net_coalition_value(MC_net, coalition):
    return sum(x for P, x in MC_net if set(coalition).issuperset(P))

def MC_net_shapley_value(MC_net):
    phi = {}
    for P, x in MC_net:
        for i in P:
            phi[i] = phi.get(i, 0) + x / len(P)
    return phi
```

Risultati: $v(\{A,C\}) = 5 + 10 - 5 = 10$; la funzione caratteristica ricostruita è quella dell'aeroporto ($v = 10$ per ogni coalizione che contiene C, $v(\{A,B\}) = 7$); il valore di Shapley è (1,67; 2,67; 5,67) con tutti e tre i metodi (MC-Net, formula esponenziale, formula dell'aeroporto). L'esempio a due regole della teoria dà $\phi_a = 2{,}5$, $\phi_b = 4{,}5$.

> **Attenzione (`max(MC_net)` trova i giocatori solo per caso).** `from_MC_net_to_CFG` ricava l'insieme dei giocatori con `max(MC_net)[0]`. Le regole sono tuple (pattern, valore) e le tuple si confrontano sul primo elemento: per gli insiemi `>` vuol dire "sovrainsieme proprio", come per `max(v)` nella lezione 5. Funziona perché la MC-Net del notebook ha una regola con *tutti* i giocatori. Senza quella regola si perdono giocatori senza alcun errore: con le regole $\{A,B\} \to 5$, $\{B\} \to 2$, $\{C\} \to 1$ il risultato è $\{A,B\}$ e C sparisce. Il modo robusto è l'unione dei pattern: `set().union(*(P for P, _ in MC_net))`.

In più: MC-Nets con letterali negativi. Una regola diventa una terna (positivi, negativi, valore), e si applica se `pos <= S and not (neg & S)`. Con la formula di Shapley generale della teoria, sull'esempio $x_1 \wedge x_2 \to 5, x_2 \to 2, x_3 \to 4, x_2 \wedge \neg x_3 \to -2$ si ottiene (2,5; 3,5; 5), identico al calcolo esponenziale sulla funzione caratteristica ricostruita, che coincide con la tabella delle slide.

Riepilogo della lezione

- La funzione caratteristica ha 2^n valori e Shapley li usa tutti: servono scorciatoie.
- **Aeroporto**: $v(S) = \max_{i \in S} c_i$. Shapley: $\phi_i = \sum_{j \le i} \frac{c_j - c_{j-1}}{n-j+1}$, ogni tratto di pista diviso fra chi lo usa. Si dimostra per additività; si calcola in $O(n)$.
- **ISG**: $v(C)$ è la somma dei pesi degli archi interni a C; $\phi_i = w_{ii} + \frac{1}{2} \sum_{j \in \Gamma(i)} w_{ij}$. Non completo (solo interazioni a coppie).
- **MC-Nets**: regole pattern $\to$ valore; $v(S)$ è la somma delle regole soddisfatte. Con congiunzioni: $\phi_i = \sum x/|\varphi|$. Con letterali negativi: $\frac{(p-1)!\,n!}{(p+n)!} x$ ai positivi, $\frac{p!\,(n-1)!}{(p+n)!}(-x)$ ai negativi. Complete, ma $O(2^n)$ regole nel caso peggiore.
- **Monte Carlo**: media dei contributi marginali su m permutazioni casuali; stima non distorta, errore $\propto 1/\sqrt{m}$; vale per qualunque gioco.
- Il filo comune: *additività* del valore di Shapley (scomporre il gioco in pezzi semplici) oppure *campionamento*.
- **Lab**: aeroporto lineare contro esponenziale (16 µs contro 19 s con $n = 20$), Monte Carlo con pendenza misurata $-0{,}5$, ISG con `networkx`, MC-Nets; tutto verificato contro la formula esatta.